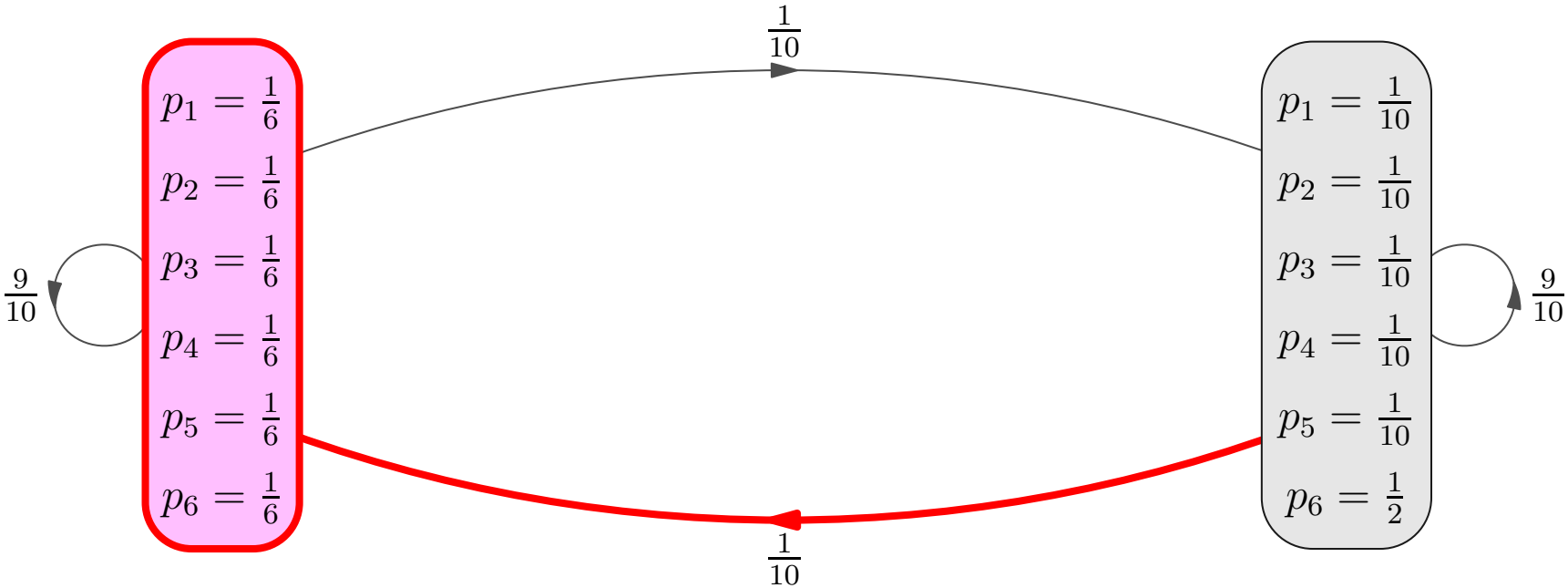
# Lecture 7: Hidden Markov Models

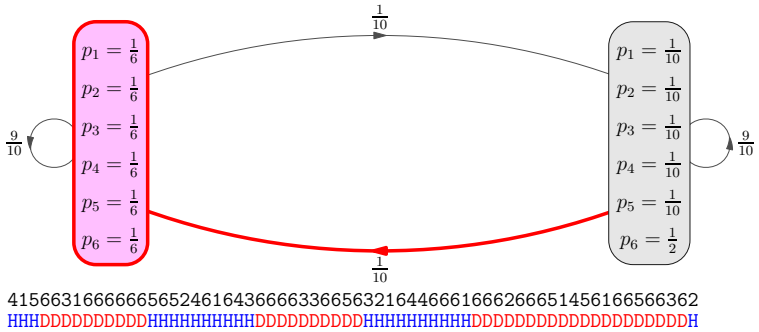



415663166666656524616436666336656321644666166626665145616656636 2
HHHDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDDDDDDDDDDDH

*Generative Models, Markov Chains, FSA, HMMs*

# Outline

1. **Probabilistic Models**
2. Hidden Markov Models
3. Summing Over Paths



# Probabilistic Models

- Probability is the natural language for talking about uncertainty
- Reason about biological sequences where we have uncertainty
- Can we develop methods to answer questions such as how likely is a certain sequence to be a coding region?
- Or what is the probability that this part of a protein forms a beta sheet?

# Probabilistic Models

- Probability is the natural language for talking about uncertainty
- Reason about biological sequences where we have uncertainty
- Can we develop methods to answer questions such as how likely is a certain sequence to be a coding region?
- Or what is the probability that this part of a protein forms a beta sheet?

# Probabilistic Models

- Probability is the natural language for talking about uncertainty
- Reason about biological sequences where we have uncertainty
- Can we develop methods to answer questions such as how likely is a certain sequence to be a coding region?
- Or what is the probability that this part of a protein forms a beta sheet?

# Probabilistic Models

- Probability is the natural language for talking about uncertainty
- Reason about biological sequences where we have uncertainty
- Can we develop methods to answer questions such as how likely is a certain sequence to be a coding region?
- Or what is the probability that this part of a protein forms a beta sheet?

# Probabilistic Models and ML

- In machine learning we usually have a data set consisting of pairs of features, $\boldsymbol{x}$, and targets, $y$
- Many machine learning problems comes down to determining the probability of the target $y$ given the features, $\boldsymbol{x}$

$$\mathbb{P}\left(y|\boldsymbol{x}\right)$$

- This is a discrimination task
- However, we usually just use a model with lots of parameters, $\boldsymbol{\theta}$ and train on data to find $\mathbb{P}\left(y|\boldsymbol{x},\boldsymbol{\theta}\right)$

# Probabilistic Models and ML

- In machine learning we usually have a data set consisting of pairs of features, $\boldsymbol{x}$, and targets, $y$
- Many machine learning problems comes down to determining the probability of the target $y$ given the features, $\boldsymbol{x}$

$$\mathbb{P}\left(y|\boldsymbol{x}\right)$$

- This is a discrimination task
- However, we usually just use a model with lots of parameters, $\boldsymbol{\theta}$ and train on data to find $\mathbb{P}\left(y|\boldsymbol{x},\boldsymbol{\theta}\right)$

# Probabilistic Models and ML

- In machine learning we usually have a data set consisting of pairs of features, $\boldsymbol{x}$, and targets, $y$
- Many machine learning problems comes down to determining the probability of the target $y$ given the features, $\boldsymbol{x}$

$$\mathbb{P}\left(y|\boldsymbol{x}\right)$$

- This is a discrimination task
- However, we usually just use a model with lots of parameters, $\boldsymbol{\theta}$ and train on data to find $\mathbb{P}\left(y|\boldsymbol{x},\boldsymbol{\theta}\right)$

# Probabilistic Models and ML

- In machine learning we usually have a data set consisting of pairs of features, $\boldsymbol{x}$, and targets, $y$
- Many machine learning problems comes down to determining the probability of the target $y$ given the features, $\boldsymbol{x}$

$$\mathbb{P}\left(y|\boldsymbol{x}\right)$$

- This is a discrimination task
- However, we usually just use a model with lots of parameters, $\boldsymbol{\theta}$ and train on data to find $\mathbb{P}\left(y|\boldsymbol{x},\boldsymbol{\theta}\right)$

# Generative Models

- If we really want to use a principled model then we need to construct probabilistic models by thinking about how the data is generated

- This involves computing the joint probability

$$\mathbb{P}\left(y, \boldsymbol{x}\right)$$

- We can still do discrimination since

$$\mathbb{P}\left(y|\boldsymbol{x}\right) = \frac{\mathbb{P}\left(y, \boldsymbol{x}\right)}{\mathbb{P}\left(\boldsymbol{x}\right)} = \frac{\mathbb{P}\left(y, \boldsymbol{x}\right)}{\sum_y \mathbb{P}\left(y, \boldsymbol{x}\right)}$$

- The difference is we have to consider the distribution of the features $\mathbb{P}\left(\boldsymbol{x}\right)$

# Generative Models

- If we really want to use a principled model then we need to construct probabilistic models by thinking about how the data is generated

- This involves computing the joint probability

$$\mathbb{P}\left(y, \boldsymbol{x}\right)$$

- We can still do discrimination since

$$\mathbb{P}\left(y|\boldsymbol{x}\right) = \frac{\mathbb{P}\left(y, \boldsymbol{x}\right)}{\mathbb{P}\left(\boldsymbol{x}\right)} = \frac{\mathbb{P}\left(y, \boldsymbol{x}\right)}{\sum_y \mathbb{P}\left(y, \boldsymbol{x}\right)}$$

- The difference is we have to consider the distribution of the features $\mathbb{P}\left(\boldsymbol{x}\right)$

# Generative Models

- If we really want to use a principled model then we need to construct probabilistic models by thinking about how the data is generated

- This involves computing the joint probability

$$\mathbb{P}\left(y, \boldsymbol{x}\right)$$

- We can still do discrimination since

$$\mathbb{P}\left(y|\boldsymbol{x}\right) = \frac{\mathbb{P}\left(y, \boldsymbol{x}\right)}{\mathbb{P}\left(\boldsymbol{x}\right)} = \frac{\mathbb{P}\left(y, \boldsymbol{x}\right)}{\sum_y \mathbb{P}\left(y, \boldsymbol{x}\right)}$$

- The difference is we have to consider the distribution of the features $\mathbb{P}\left(\boldsymbol{x}\right)$

# Generative Models

- If we really want to use a principled model then we need to construct probabilistic models by thinking about how the data is generated

- This involves computing the joint probability

$$\mathbb{P}\left(y, \boldsymbol{x}\right)$$

- We can still do discrimination since

$$\mathbb{P}\left(y|\boldsymbol{x}\right) = \frac{\mathbb{P}\left(y, \boldsymbol{x}\right)}{\mathbb{P}\left(\boldsymbol{x}\right)} = \frac{\mathbb{P}\left(y, \boldsymbol{x}\right)}{\sum_y \mathbb{P}\left(y, \boldsymbol{x}\right)}$$

- The difference is we have to consider the distribution of the features $\mathbb{P}\left(\boldsymbol{x}\right)$

# Markov Models

- A lot of biological data involves sequences, e.g. DNA, proteins
- The simplest model for a sequence is a Markov model, where the probability of the next element in the sequence just depends on the current state

$$\mathbb{P}\left(x_t | x_{t-1},\, x_{t-2},\, \ldots,\, x_1\right) = \mathbb{P}\left(x_t | x_{t-1}\right)$$

- However, this is an overly simple model for biological sequences
- There are far too few parameter in $\mathbb{P}\left(x_t | x_{t-1}\right)$ to obtain a useful model

# Markov Models

- A lot of biological data involves sequences, e.g. DNA, proteins
- The simplest model for a sequence is a Markov model, where the probability of the next element in the sequence just depends on the current state

$$\mathbb{P}\left(x_t | x_{t-1},\, x_{t-2},\, \ldots,\, x_1\right) = \mathbb{P}\left(x_t | x_{t-1}\right)$$

- However, this is an overly simple model for biological sequences
- There are far too few parameter in $\mathbb{P}\left(x_t | x_{t-1}\right)$ to obtain a useful model

# Markov Models

- A lot of biological data involves sequences, e.g. DNA, proteins
- The simplest model for a sequence is a Markov model, where the probability of the next element in the sequence just depends on the current state

$$\mathbb{P}\left(x_t | x_{t-1},\, x_{t-2},\, \ldots,\, x_1\right) = \mathbb{P}\left(x_t | x_{t-1}\right)$$

- However, this is an overly simple model for biological sequences
- There are far too few parameter in $\mathbb{P}\left(x_t | x_{t-1}\right)$ to obtain a useful model

# Markov Models

- A lot of biological data involves sequences, e.g. DNA, proteins
- The simplest model for a sequence is a Markov model, where the probability of the next element in the sequence just depends on the current state

$$\mathbb{P}\left(x_t | x_{t-1},\, x_{t-2},\, \ldots,\, x_1\right) = \mathbb{P}\left(x_t | x_{t-1}\right)$$

- However, this is an overly simple model for biological sequences
- There are far too few parameter in $\mathbb{P}\left(x_t | x_{t-1}\right)$ to obtain a useful model

# Finite State Automaton

- As we saw in the last lecture, one mechanism to make more complex models is to construct a finite state automaton
- For sequences we can make the probability of generating a character from some alphabet depend on the state
- We can then build a model describing the probability of transitioning between states
- Note, that this is also a Markov model—the probability of the next symbol in the sequence depends only on the current state

# Finite State Automaton

- As we saw in the last lecture, one mechanism to make more complex models is to construct a finite state automaton
- For sequences we can make the probability of generating a character from some alphabet depend on the state
- We can then build a model describing the probability of transitioning between states
- Note, that this is also a Markov model—the probability of the next symbol in the sequence depends only on the current state

# Finite State Automaton

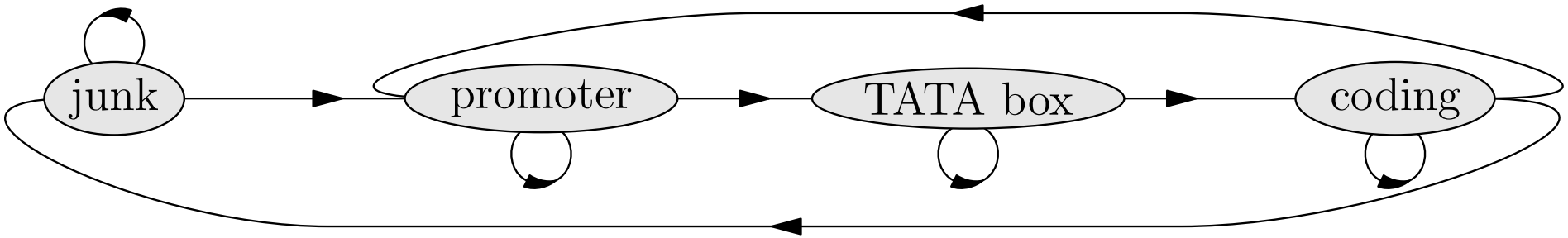
- As we saw in the last lecture, one mechanism to make more complex models is to construct a finite state automaton
- For sequences we can make the probability of generating a character from some alphabet depend on the state
- We can then build a model describing the probability of transitioning between states
- Note, that this is also a Markov model—the probability of the next symbol in the sequence depends only on the current state

# Finite State Automaton

- As we saw in the last lecture, one mechanism to make more complex models is to construct a finite state automaton
- For sequences we can make the probability of generating a character from some alphabet depend on the state
- We can then build a model describing the probability of transitioning between states
- Note, that this is also a Markov model—the probability of the next symbol in the sequence depends only on the current state

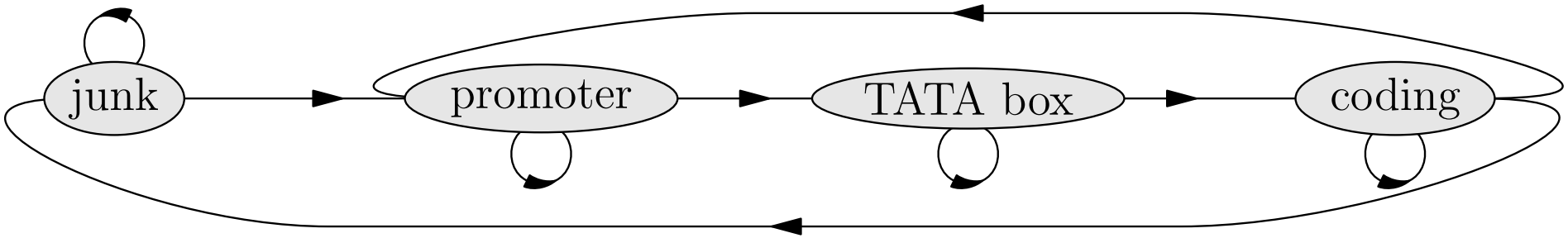
# Markov Model for DNA

- We could imagine in modelling a DNA sequence we might have different emission probabilities depending on whether we are in a
  - ⋆ Coding region
  - ⋆ The TATA box before a coding region
  - ⋆ The promoter region in front of the TATA box
  - ⋆ In junk DNA



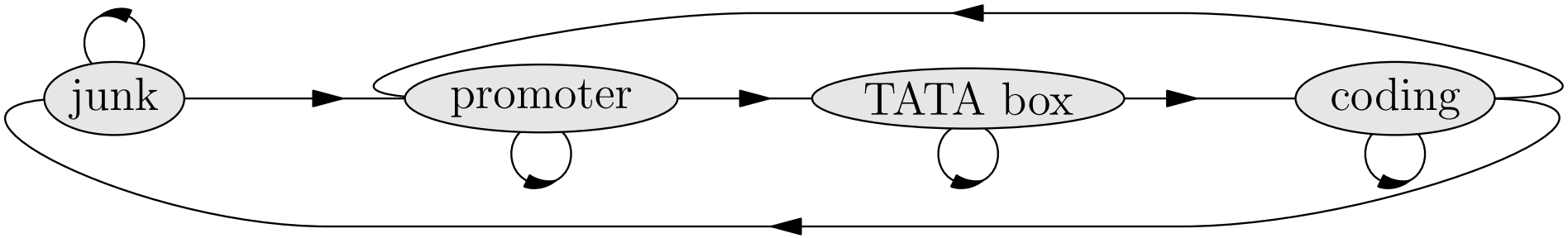
# Markov Model for DNA

- We could imagine in modelling a DNA sequence we might have different emission probabilities depending on whether we are in a
  - ⋆ Coding region
  - ⋆ The TATA box before a coding region
  - ⋆ The promoter region in front of the TATA box
  - ⋆ In junk DNA



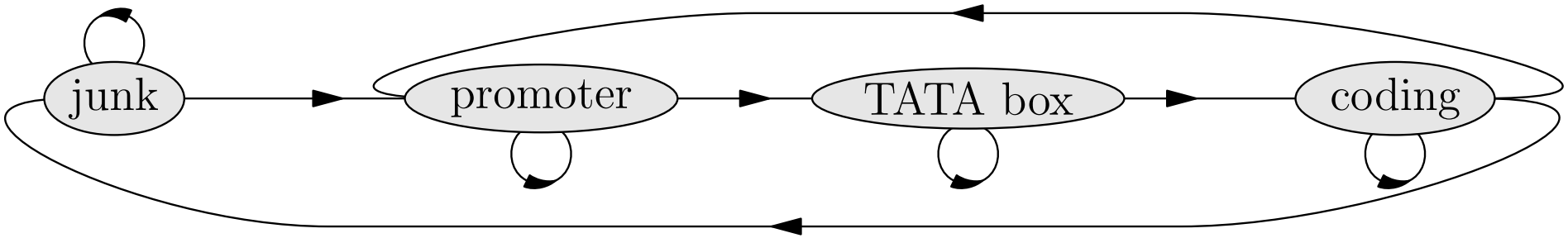
# Markov Model for DNA

- We could imagine in modelling a DNA sequence we might have different emission probabilities depending on whether we are in a
  - ⋆ Coding region
  - ⋆ The TATA box before a coding region
  - ⋆ The promoter region in front of the TATA box
  - ⋆ In junk DNA



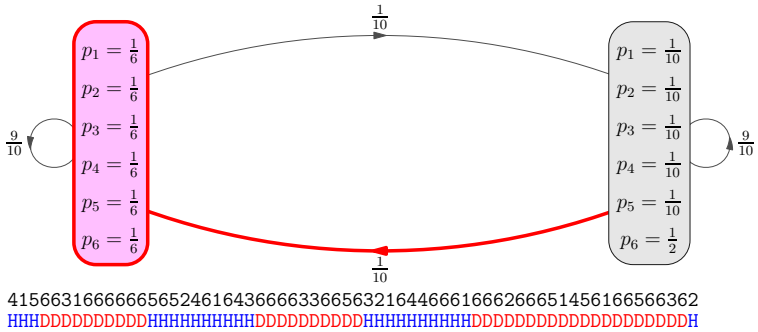
# Markov Model for DNA

- We could imagine in modelling a DNA sequence we might have different emission probabilities depending on whether we are in a
  - ⋆ Coding region
  - ⋆ The TATA box before a coding region
  - ⋆ The promoter region in front of the TATA box
  - ⋆ In junk DNA



# Outline

1. Probabilistic Models
2. **Hidden Markov Models**
3. Summing Over Paths



# Hidden Markov Model

- Although we have a generative model we want to use it as a descriminative model
- The problem is we usually don't know which region of DNA we are in (that is often what we are trying to find out)
- Thus we have a sequence, but we don't know which state we're in
- This model where we have a **probabilistic FSA** where we emit a letter in each state according to a probability which depends on the state and then we make a transition to a new state (possibly the same) is known as a **hidden Markov model**

# Hidden Markov Model

- Although we have a generative model we want to use it as a descriminative model
- The problem is we usually don't know which region of DNA we are in (that is often what we are trying to find out)
- Thus we have a sequence, but we don't know which state we're in
- This model where we have a **probabilistic FSA** where we emit a letter in each state according to a probability which depends on the state and then we make a transition to a new state (possibly the same) is known as a **hidden Markov model**

# Hidden Markov Model

- Although we have a generative model we want to use it as a descriminative model
- The problem is we usually don't know which region of DNA we are in (that is often what we are trying to find out)
- Thus we have a sequence, but we don't know which state we're in
- This model where we have a **probabilistic FSA** where we emit a letter in each state according to a probability which depends on the state and then we make a transition to a new state (possibly the same) is known as a **hidden Markov model**

# Hidden Markov Model

- Although we have a generative model we want to use it as a descriminative model
- The problem is we usually don't know which region of DNA we are in (that is often what we are trying to find out)
- Thus we have a sequence, but we don't know which state we're in
- This model where we have a **probabilistic FSA** where we emit a letter in each state according to a probability which depends on the state and then we make a transition to a new state (possibly the same) is known as a **hidden Markov model**

# Dishonest Casino

- Consider a dishonest casino where we have two dice
- An honest dice with $p_i = \frac{1}{6}$
- And a dishonest dice where $p_6 = \frac{1}{2}$ while $p_i = \frac{1}{10}$ for $i = \{1, 2, 3, 4, 5\}$
- With a probability of $\frac{1}{10}$ the roller of the dice switches dice

# Dishonest Casino

- Consider a dishonest casino where we have two dice
- An honest dice with $p_i = \frac{1}{6}$
- And a dishonest dice where $p_6 = \frac{1}{2}$ while $p_i = \frac{1}{10}$ for $i = \{1, 2, 3, 4, 5\}$
- With a probability of $\frac{1}{10}$ the roller of the dice switches dice

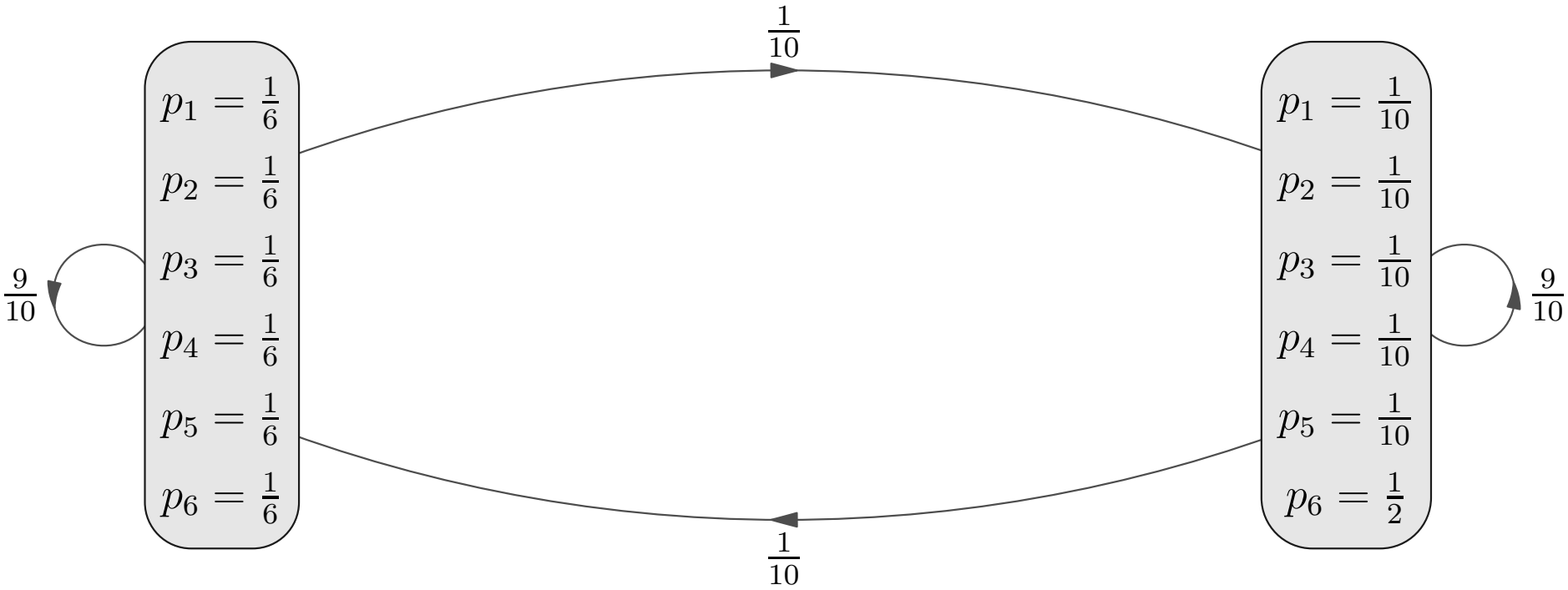
# Dishonest Casino

- Consider a dishonest casino where we have two dice
- An honest dice with $p_i = \frac{1}{6}$
- And a dishonest dice where $p_6 = \frac{1}{2}$ while $p_i = \frac{1}{10}$ for $i = \{1, 2, 3, 4, 5\}$
- With a probability of $\frac{1}{10}$ the roller of the dice switches dice

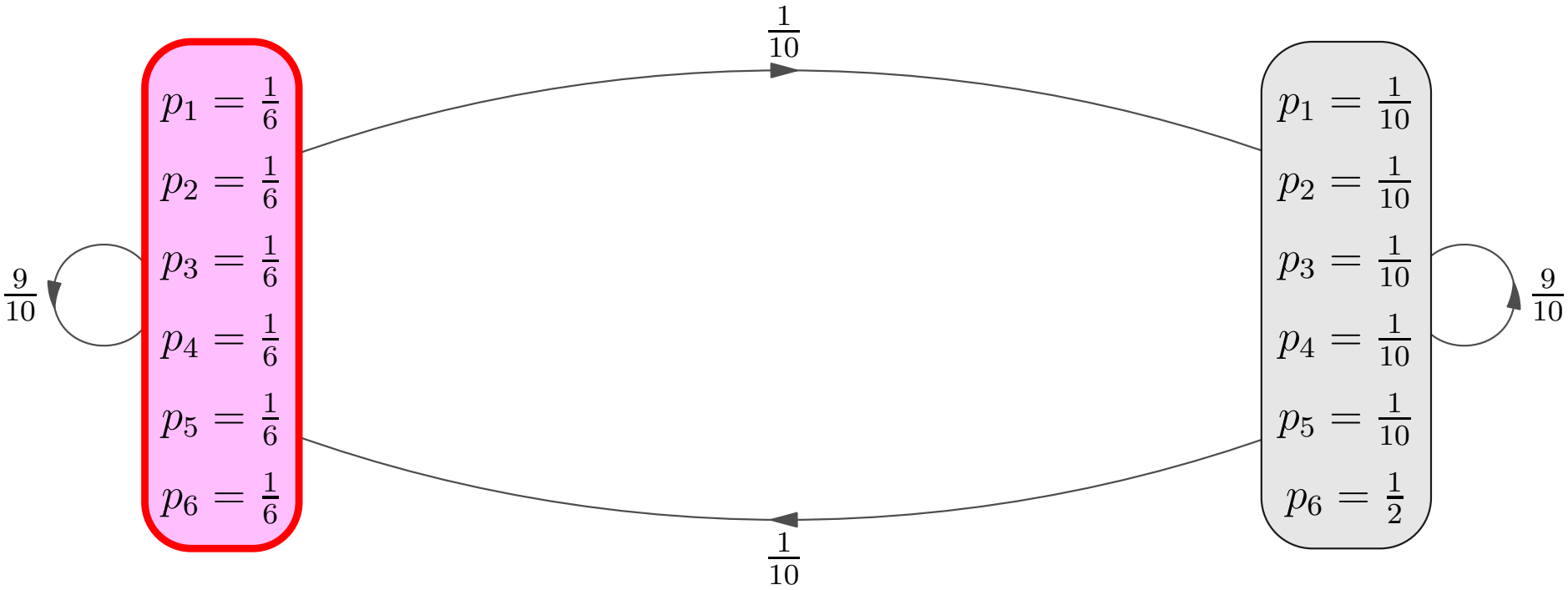
# Dishonest Casino

- Consider a dishonest casino where we have two dice
- An honest dice with $p_i = \frac{1}{6}$
- And a dishonest dice where $p_6 = \frac{1}{2}$ while $p_i = \frac{1}{10}$ for $i = \{1, 2, 3, 4, 5\}$
- With a probability of $\frac{1}{10}$ the roller of the dice switches dice

# Instance of a Dishonest Casino

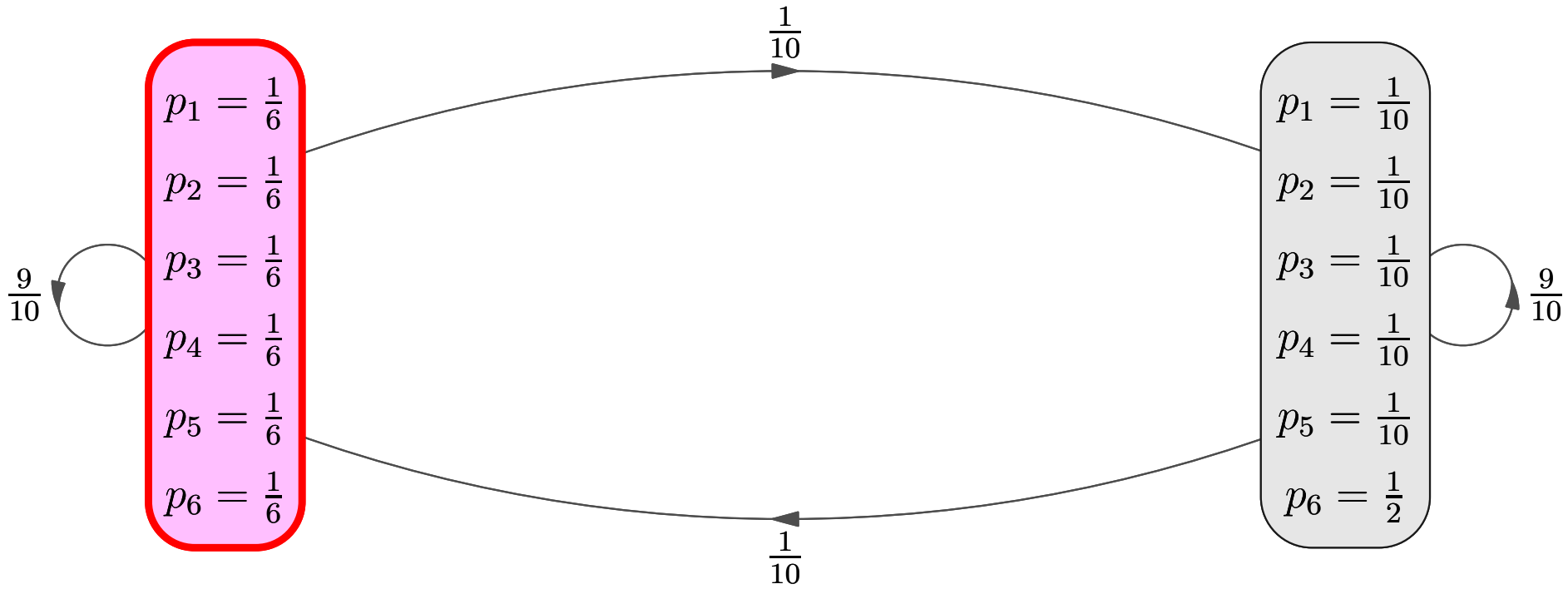


# Instance of a Dishonest Casino



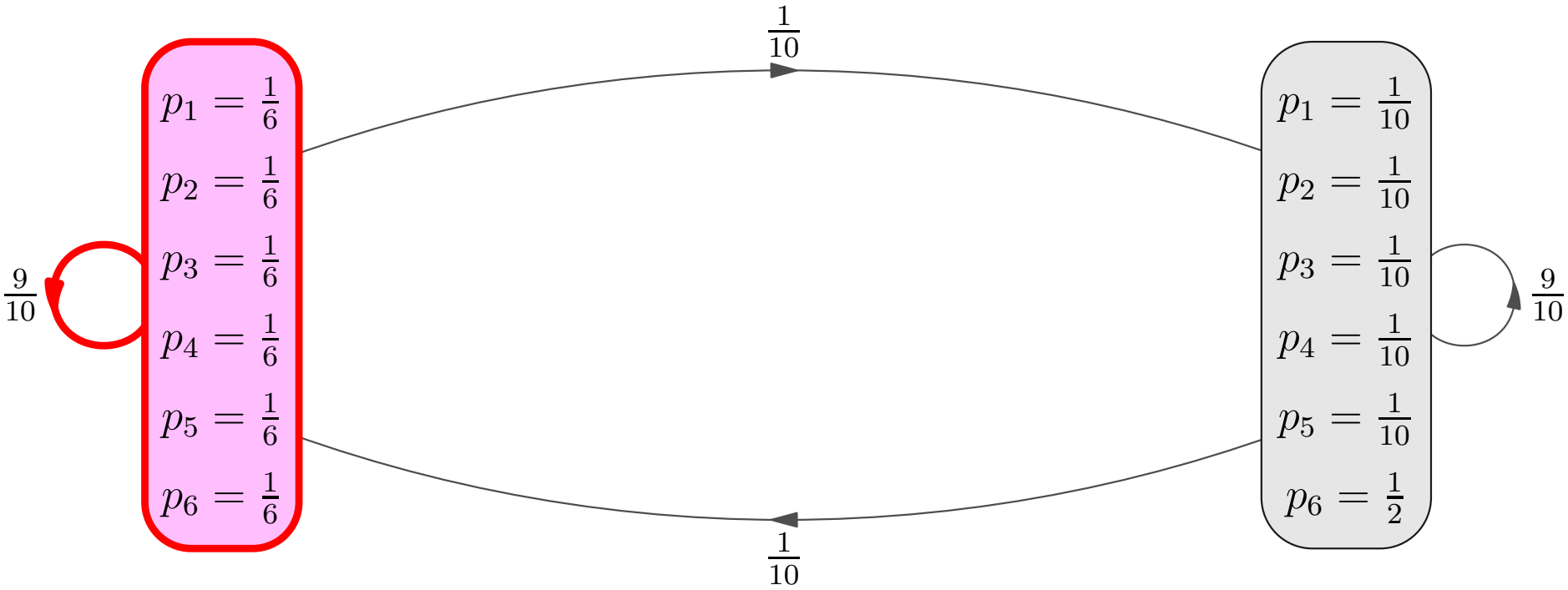

H

# Instance of a Dishonest Casino



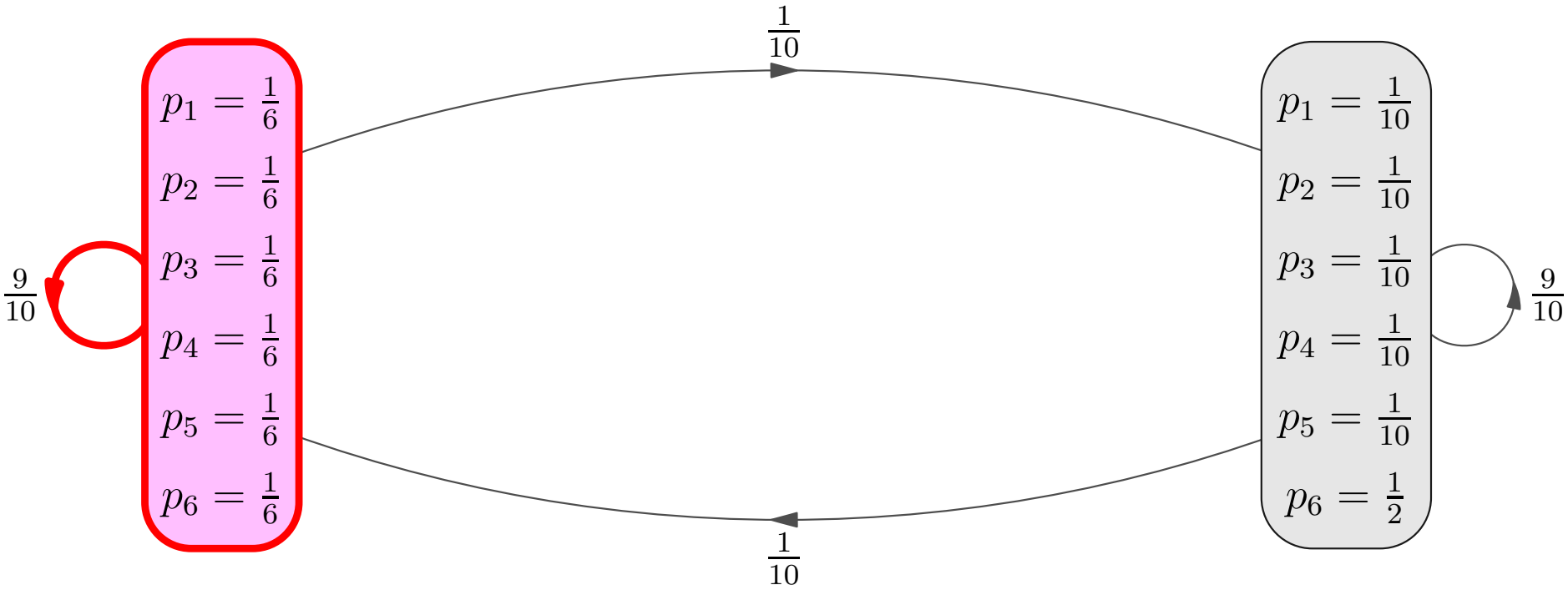

4
H

# Instance of a Dishonest Casino



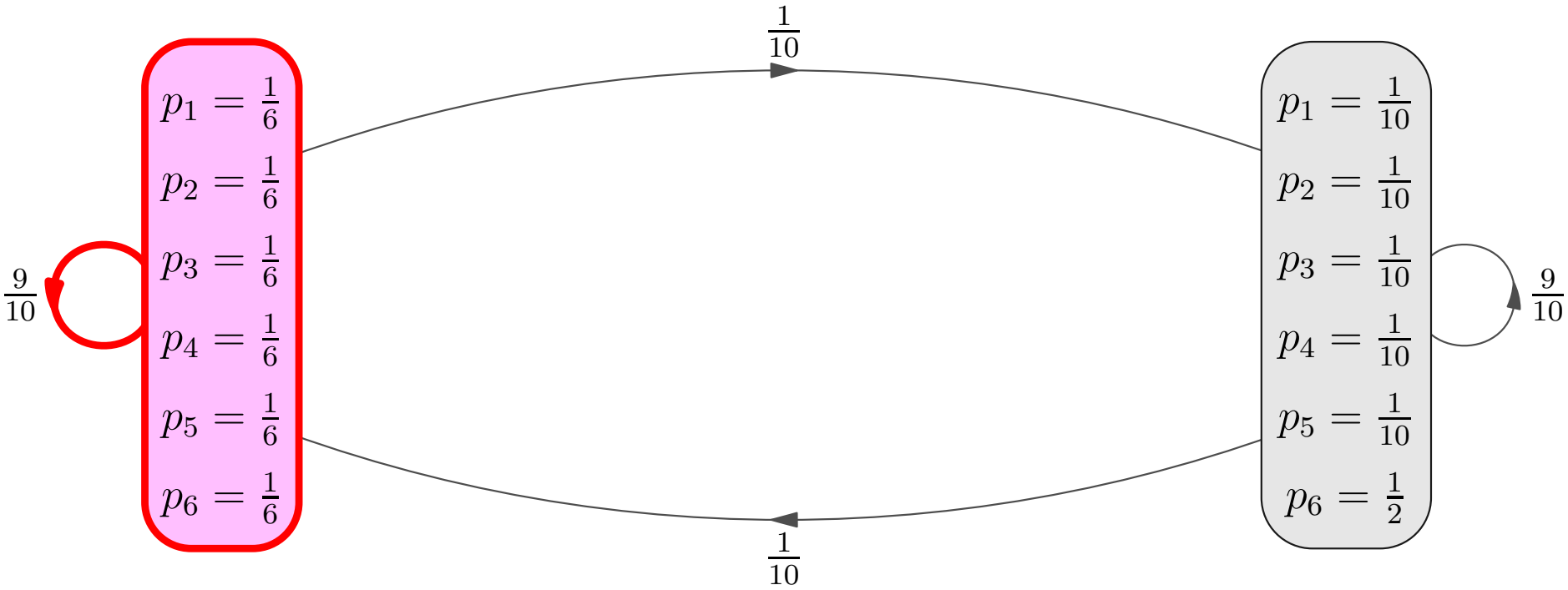

4
HH

# Instance of a Dishonest Casino



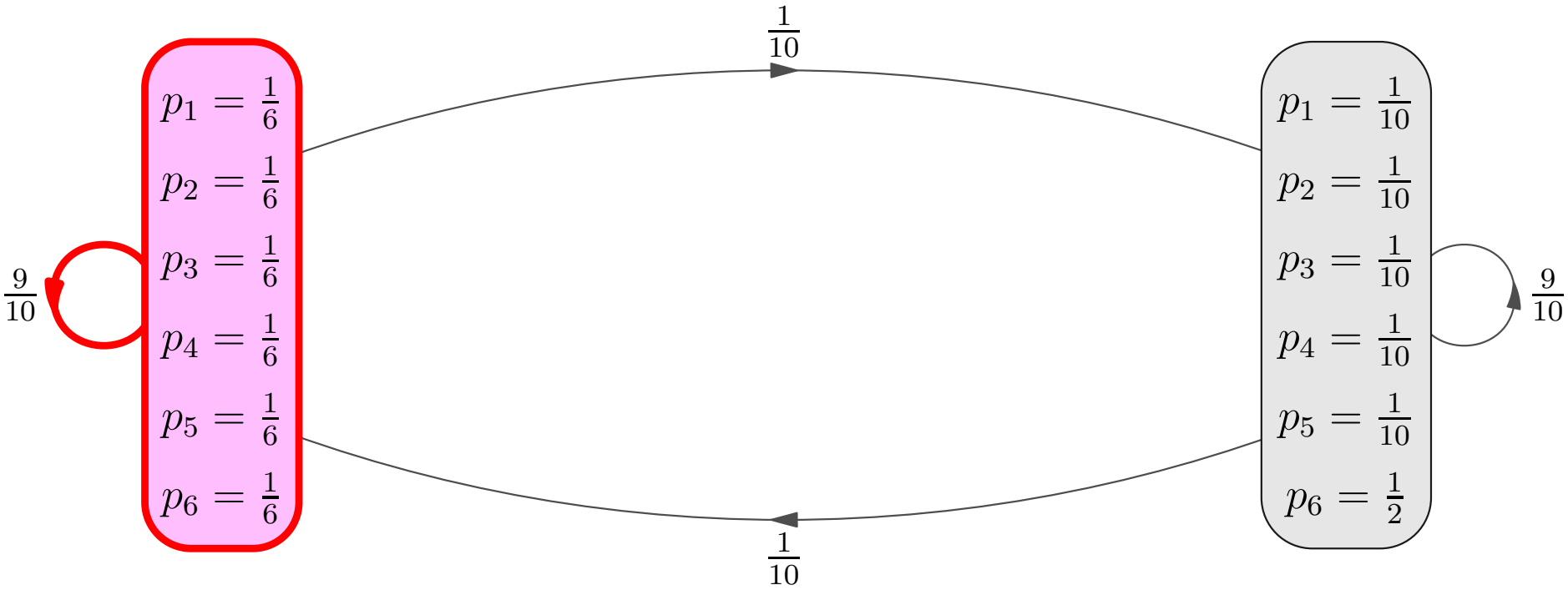

41
HH

# Instance of a Dishonest Casino



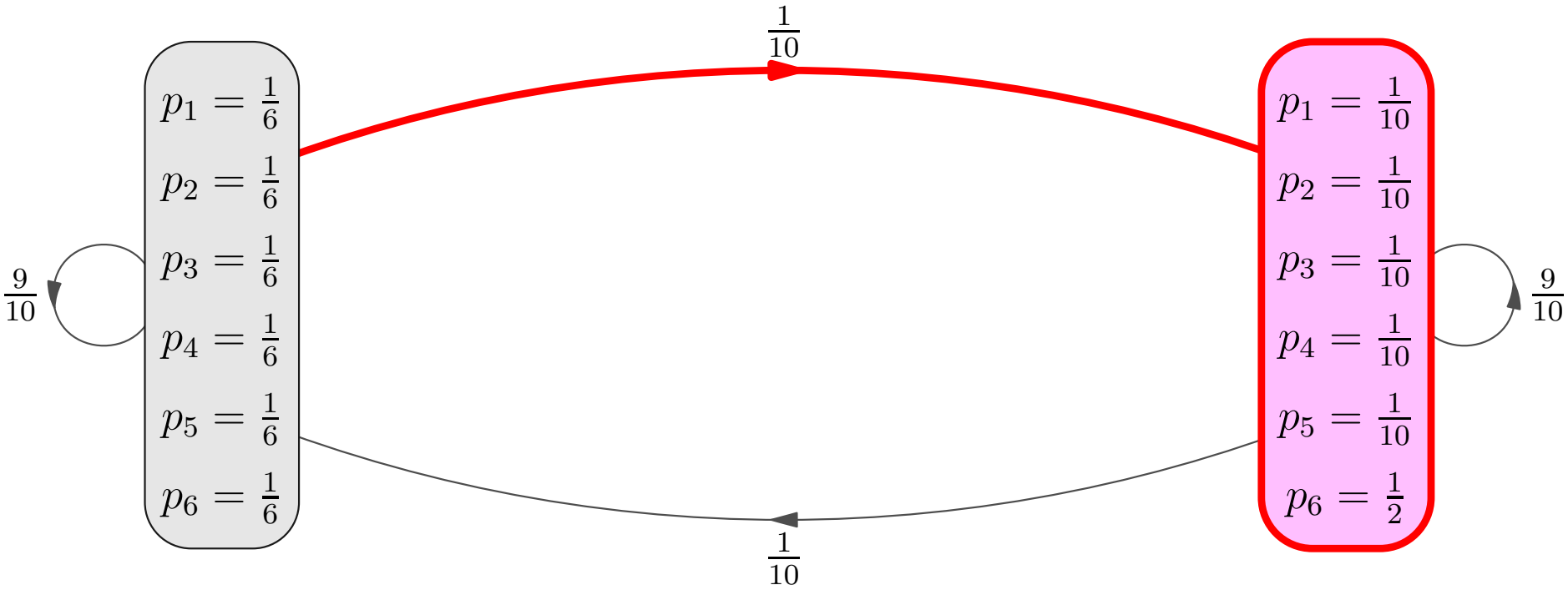

41
HHH

# Instance of a Dishonest Casino



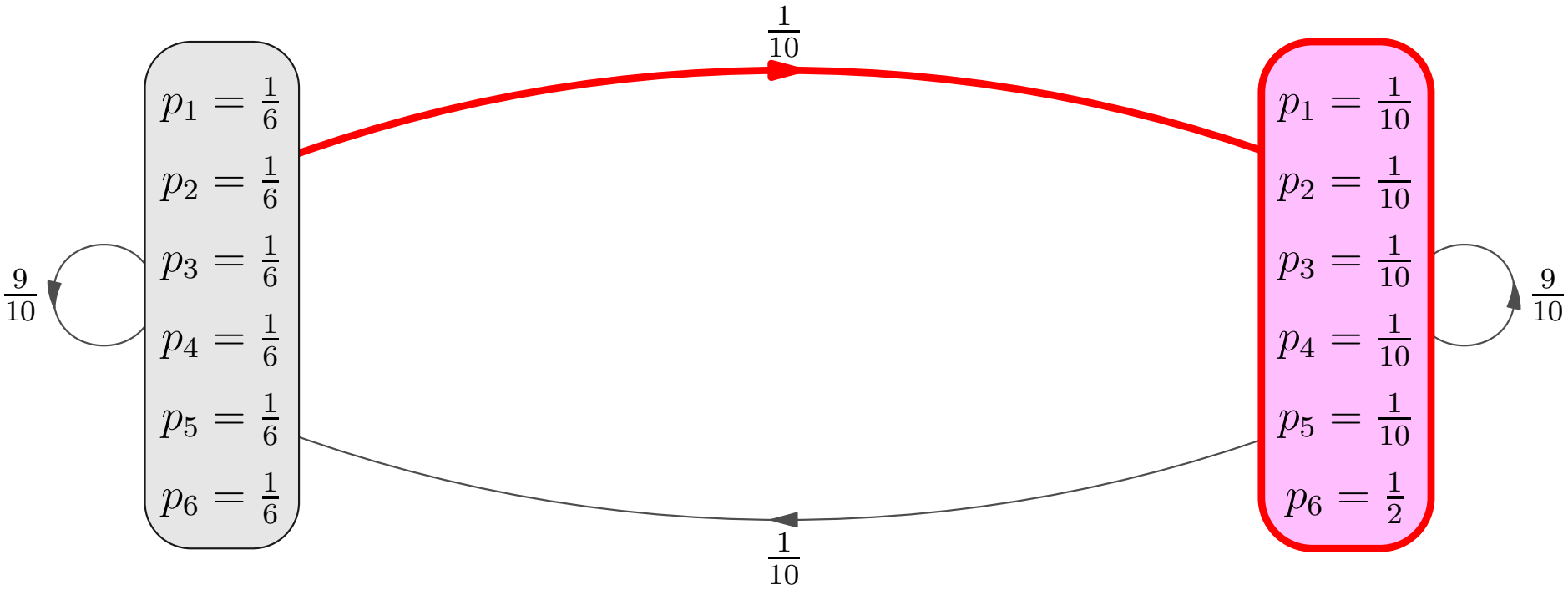

415
HHH

# Instance of a Dishonest Casino



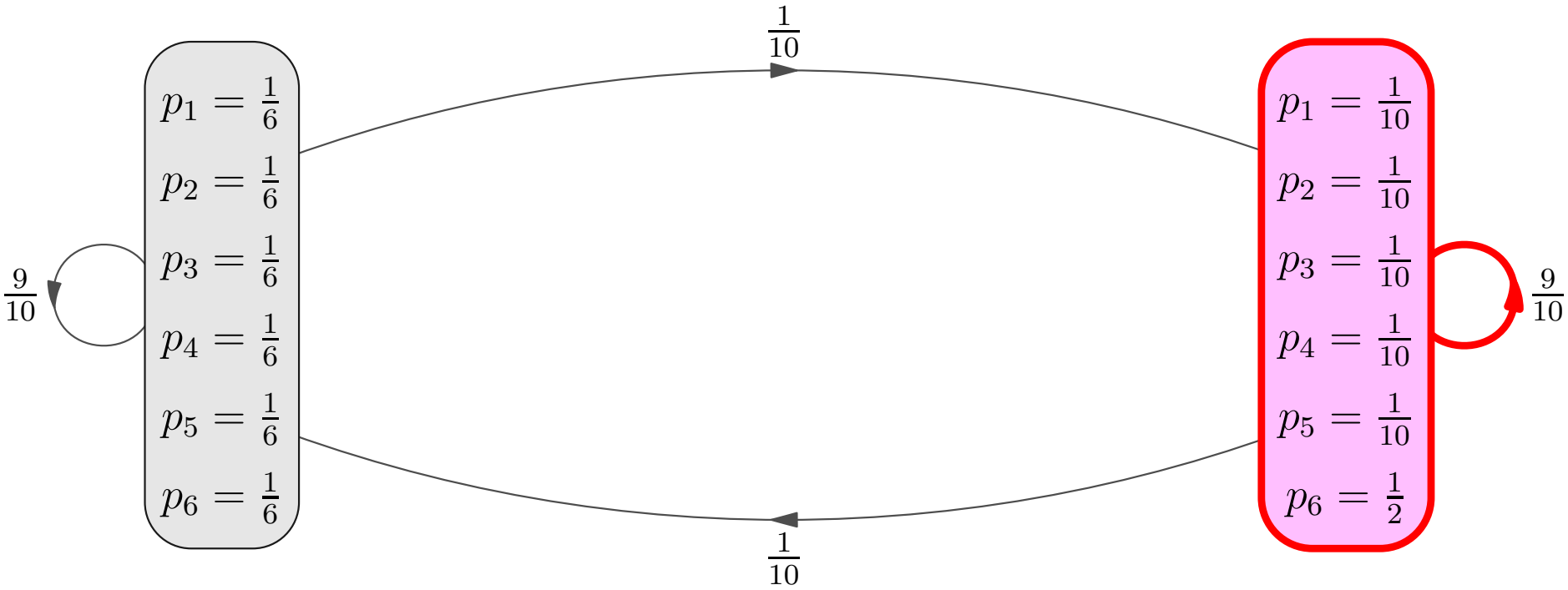

415
HHHD

# Instance of a Dishonest Casino



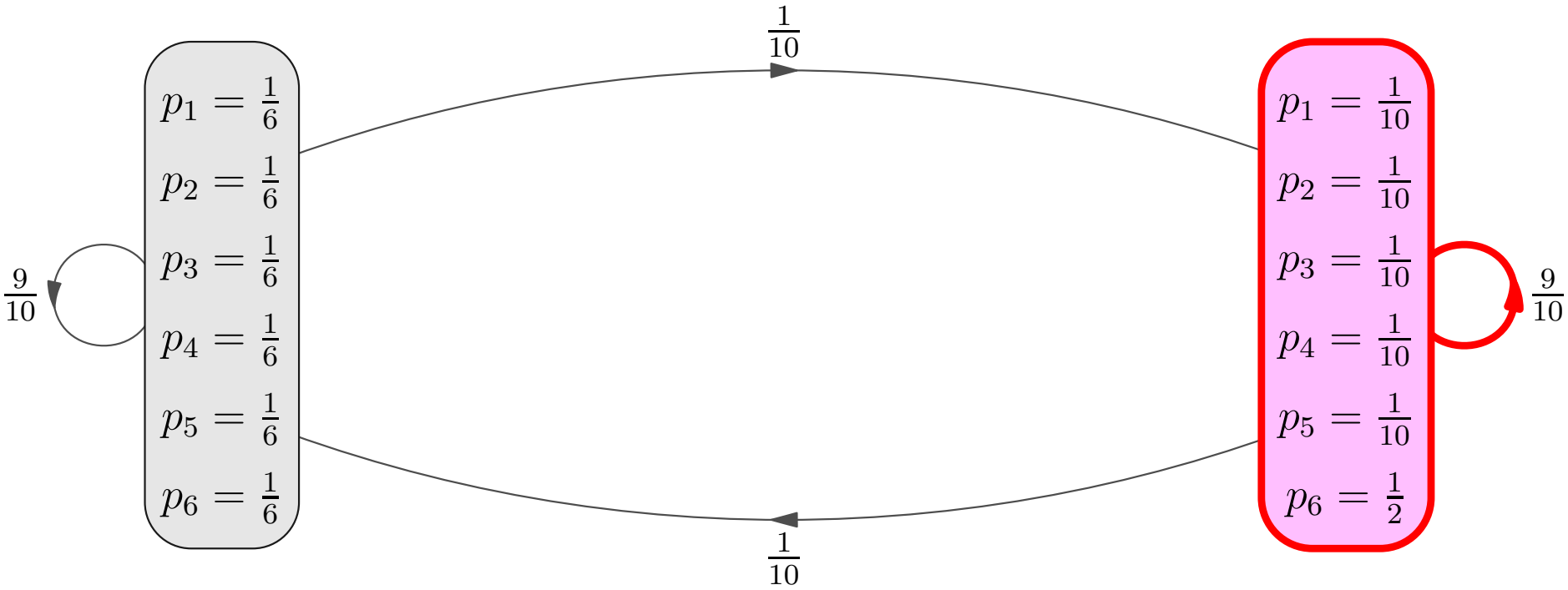

4156
HHHD

# Instance of a Dishonest Casino



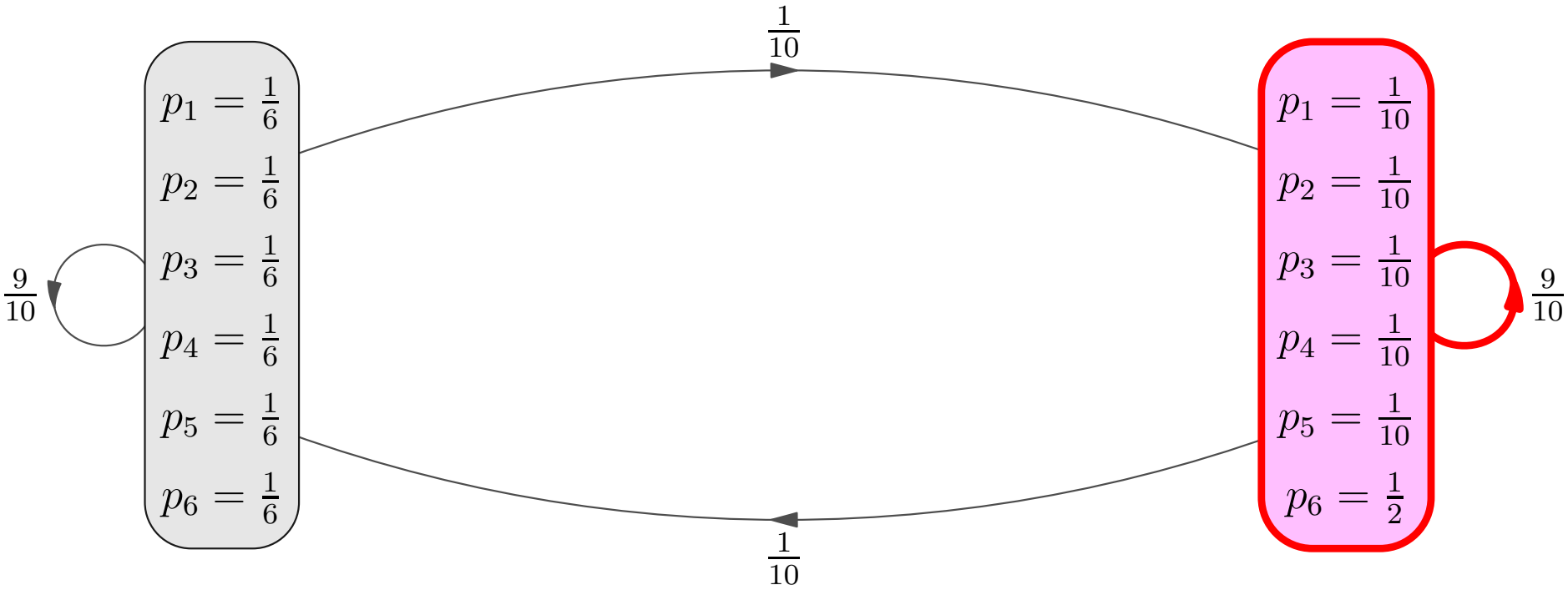

41566
HHHDD

# Instance of a Dishonest Casino



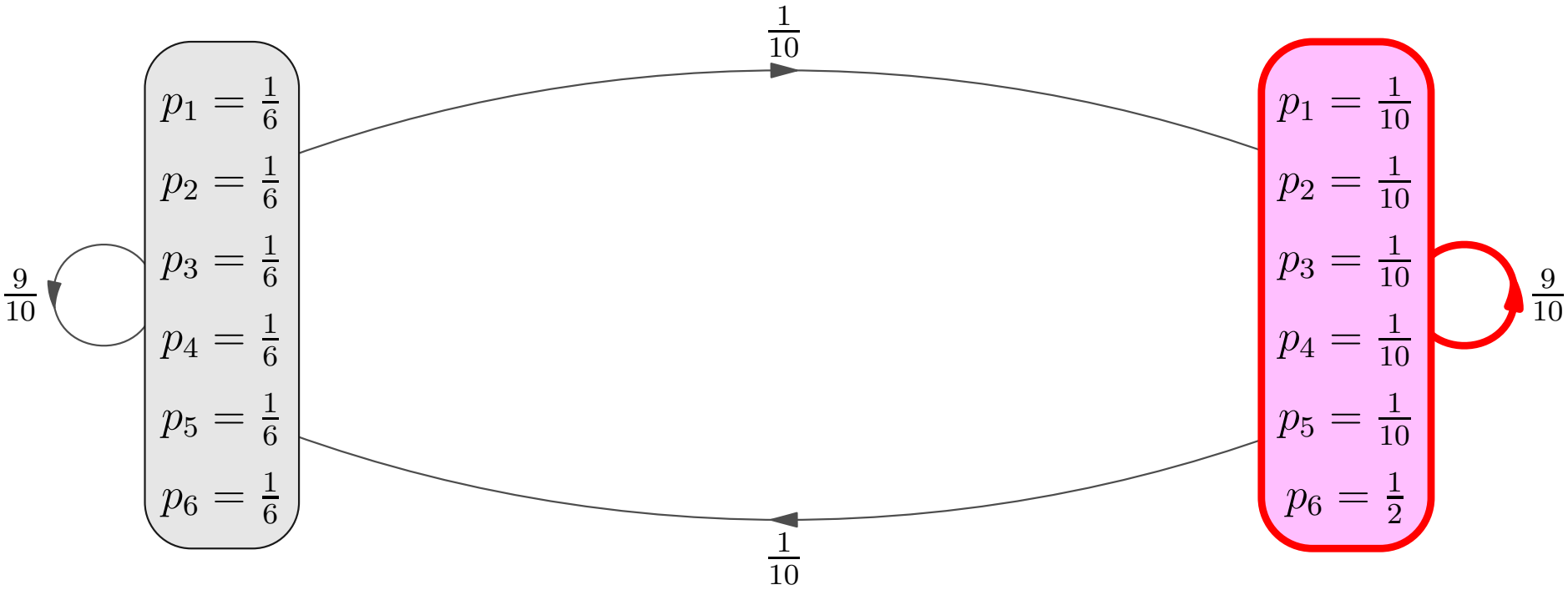

415663
HHHDDD

# Instance of a Dishonest Casino



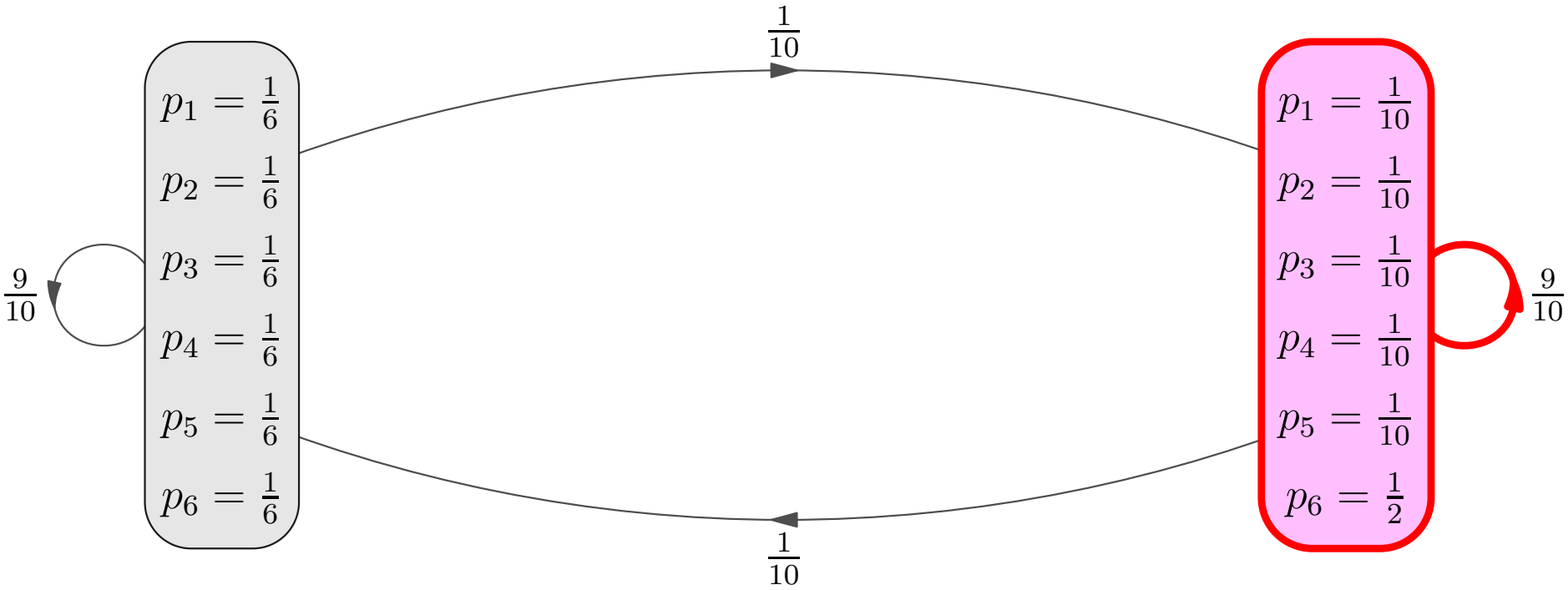

4156631
HHHDDDD

# Instance of a Dishonest Casino



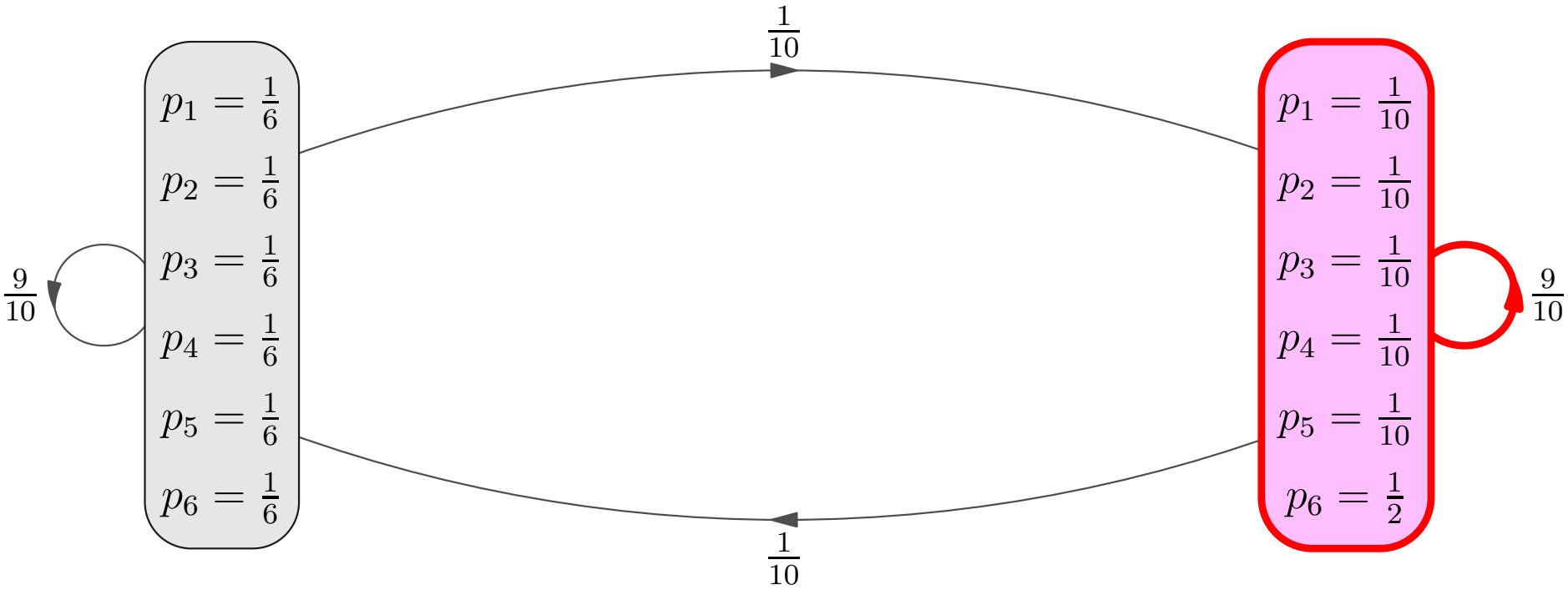

41566316
HHHDDDDD

# Instance of a Dishonest Casino



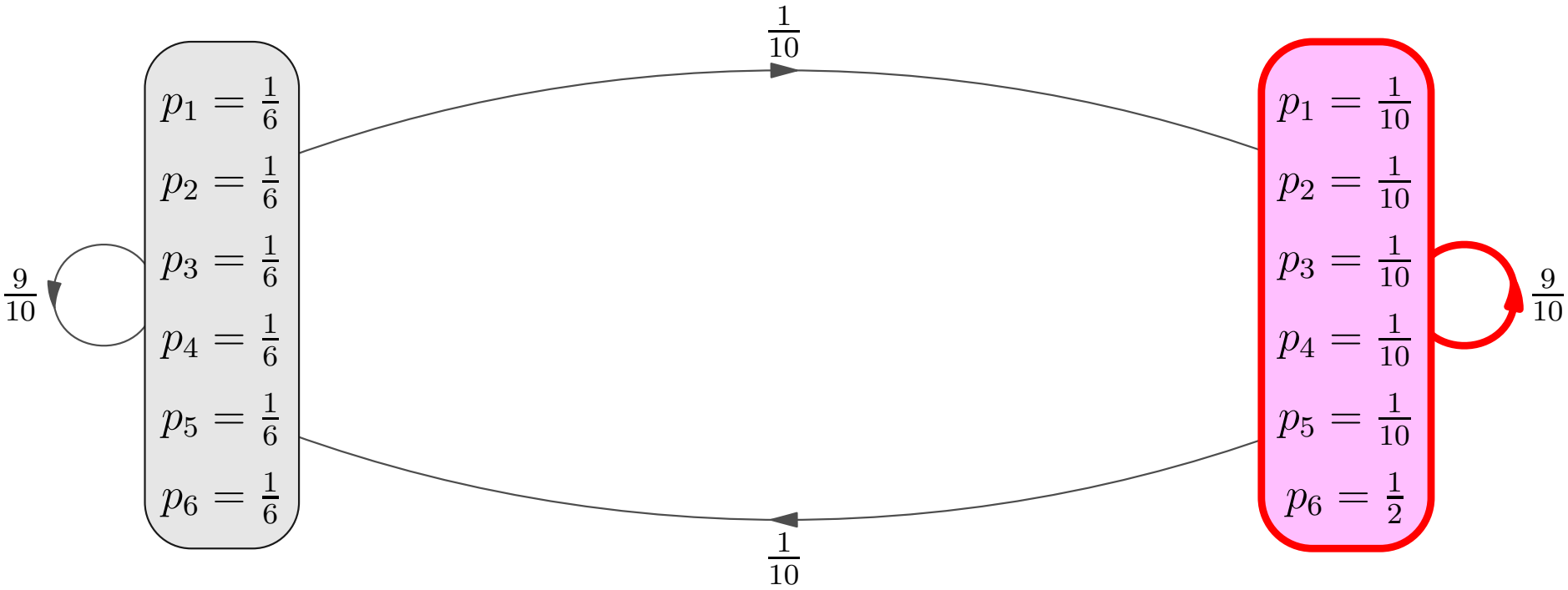

415663166
HHHDDDDDD

# Instance of a Dishonest Casino



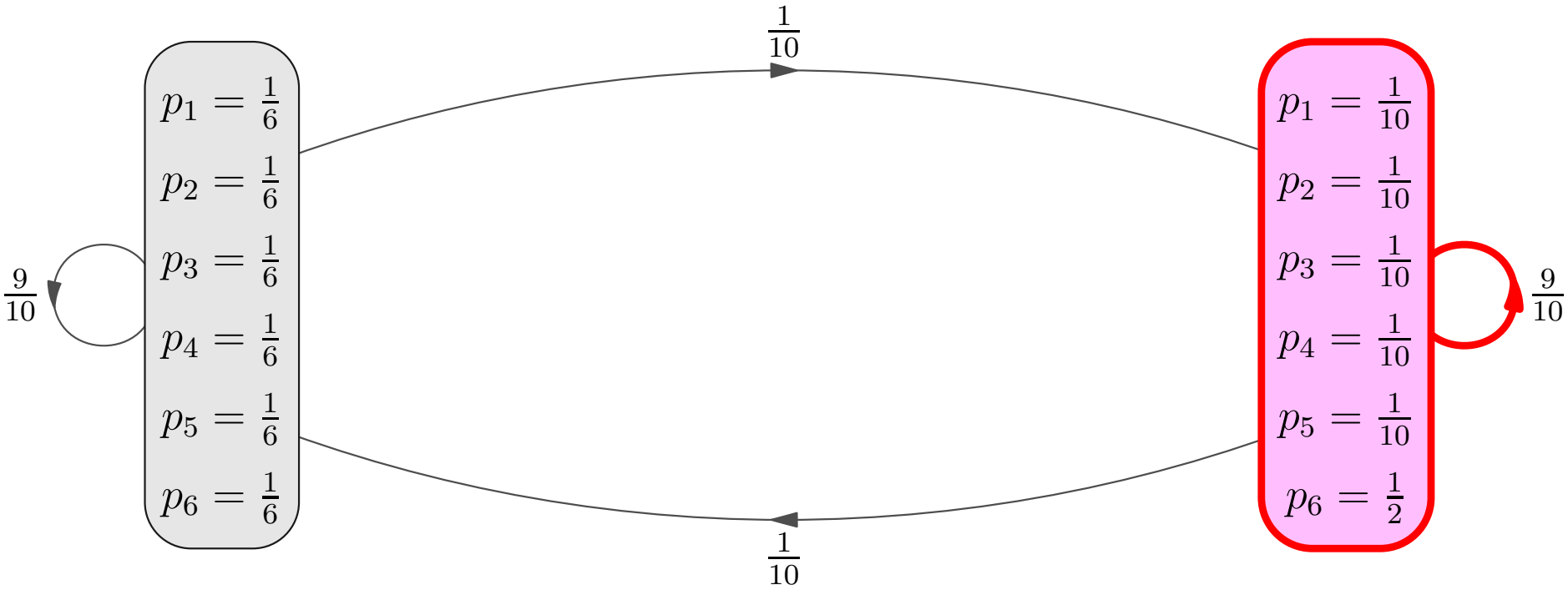

4156631666
HHHDDDDDDD

# Instance of a Dishonest Casino



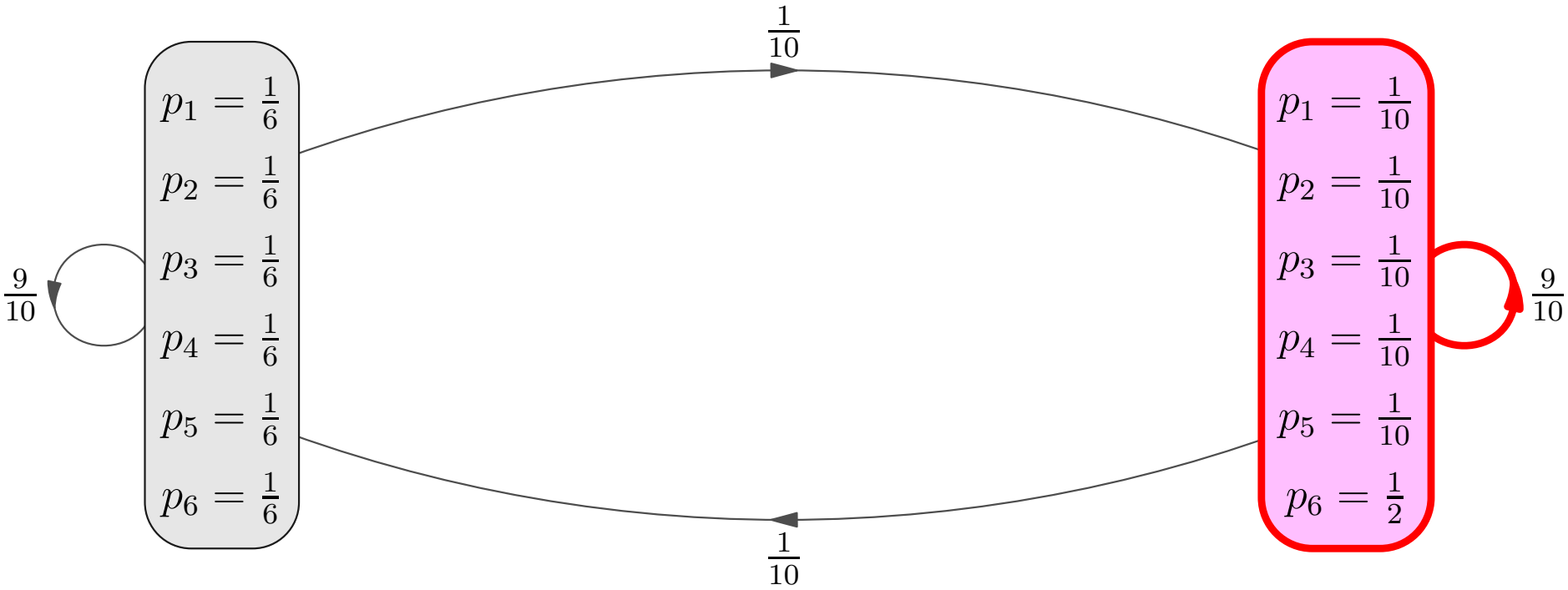

41566316666
HHHDDDDDDDD

# Instance of a Dishonest Casino



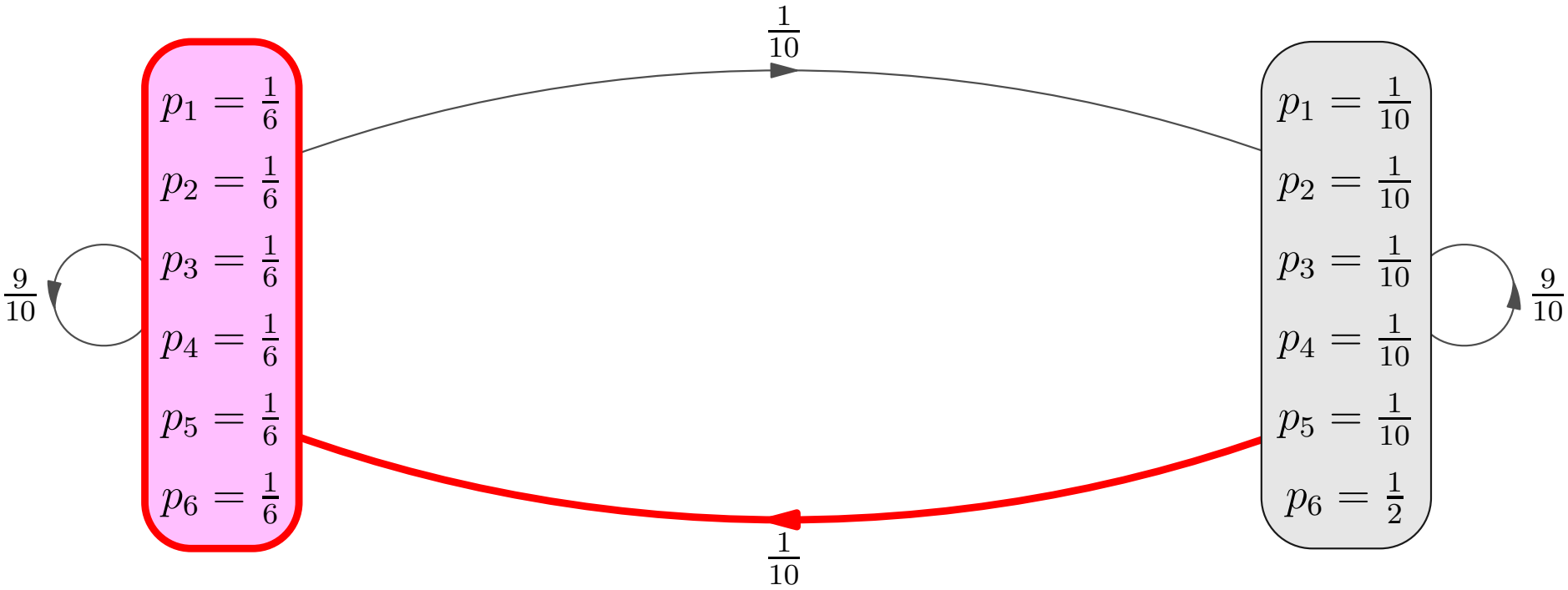

415663166666
HHHDDDDDDDDD

# Instance of a Dishonest Casino



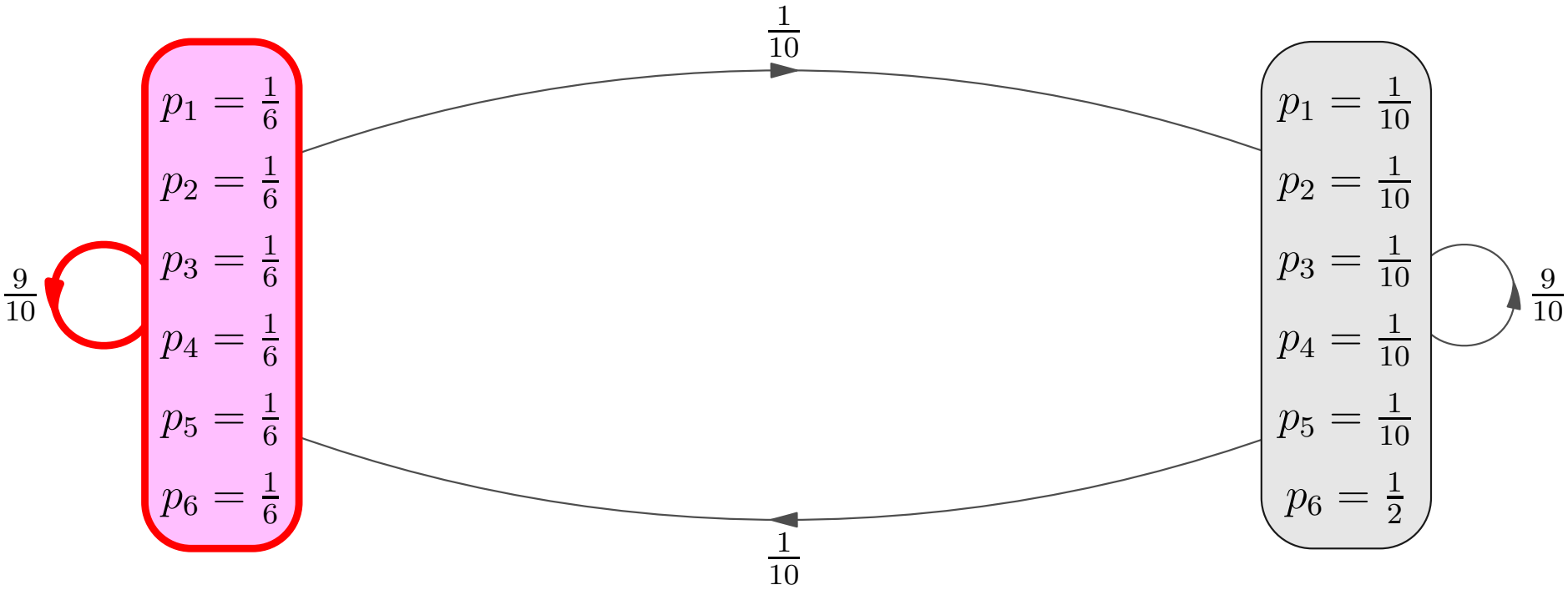

4156631666666
HHHDDDDDDDDDD

# Instance of a Dishonest Casino



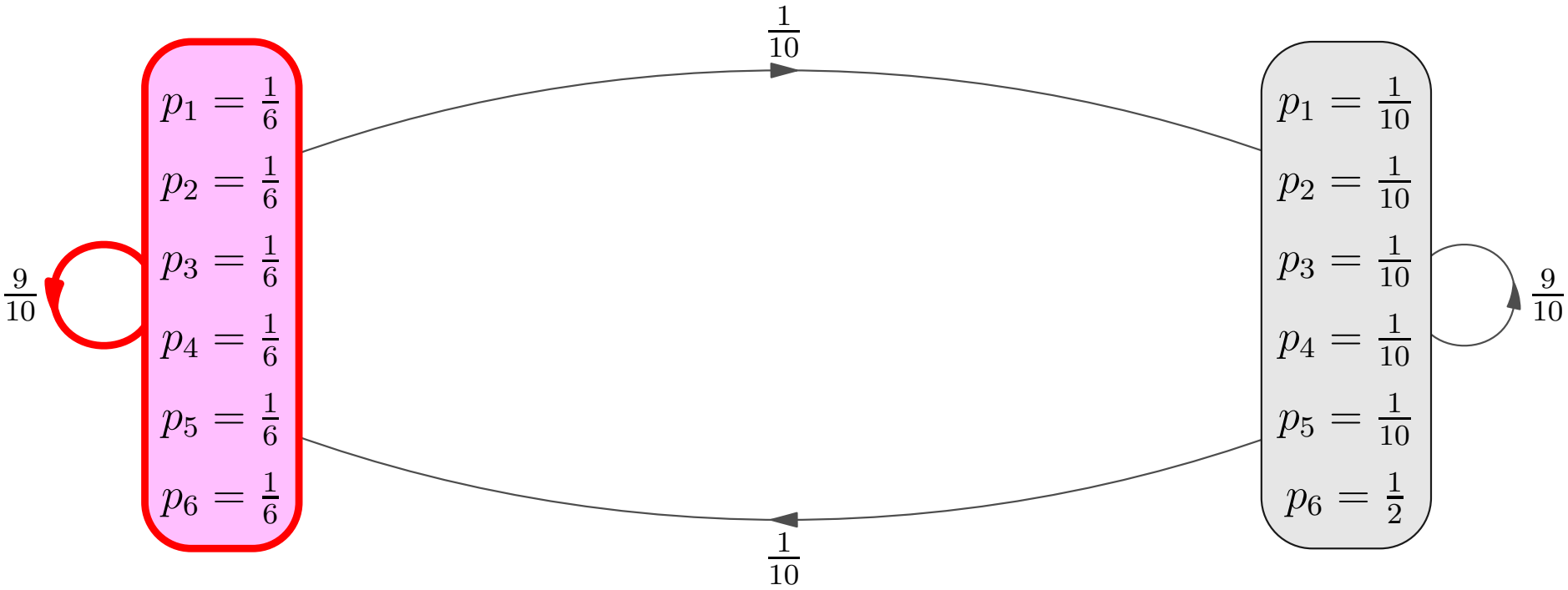

41566316666665
HHHDDDDDDDDDDH

# Instance of a Dishonest Casino



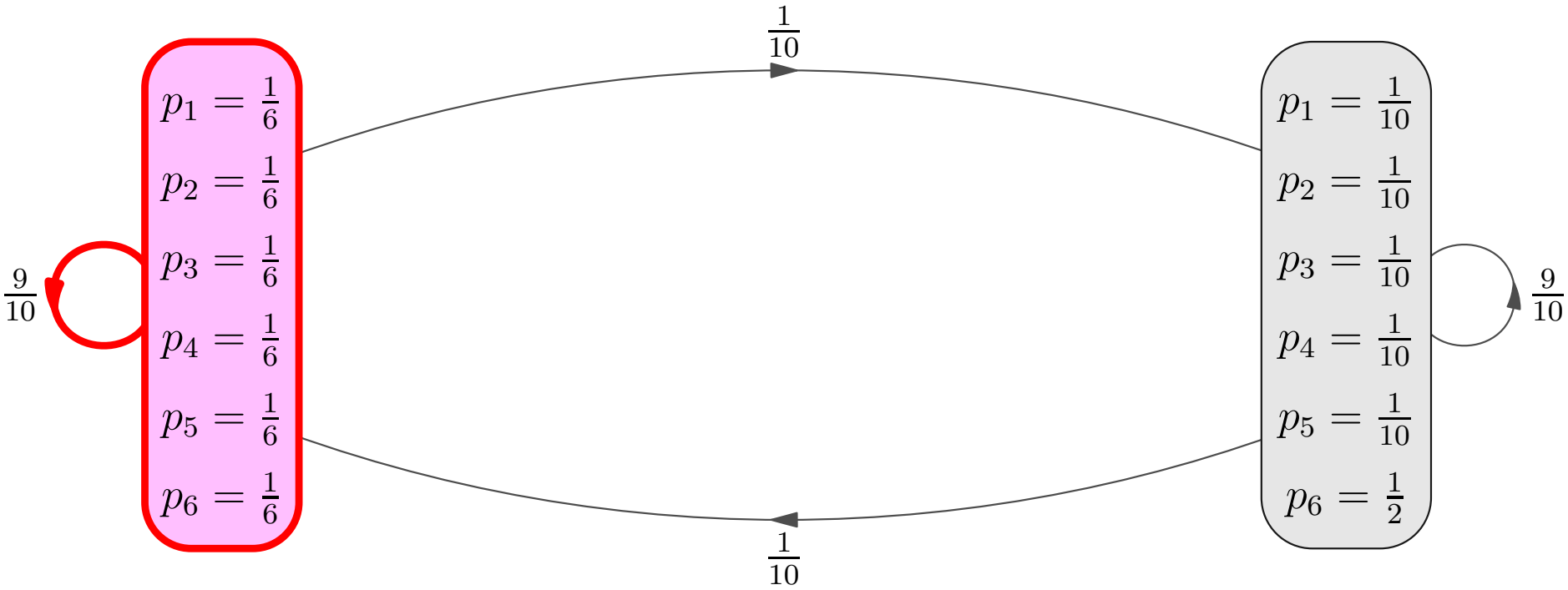

415663166666656
HHHDDDDDDDDDDHH

# Instance of a Dishonest Casino



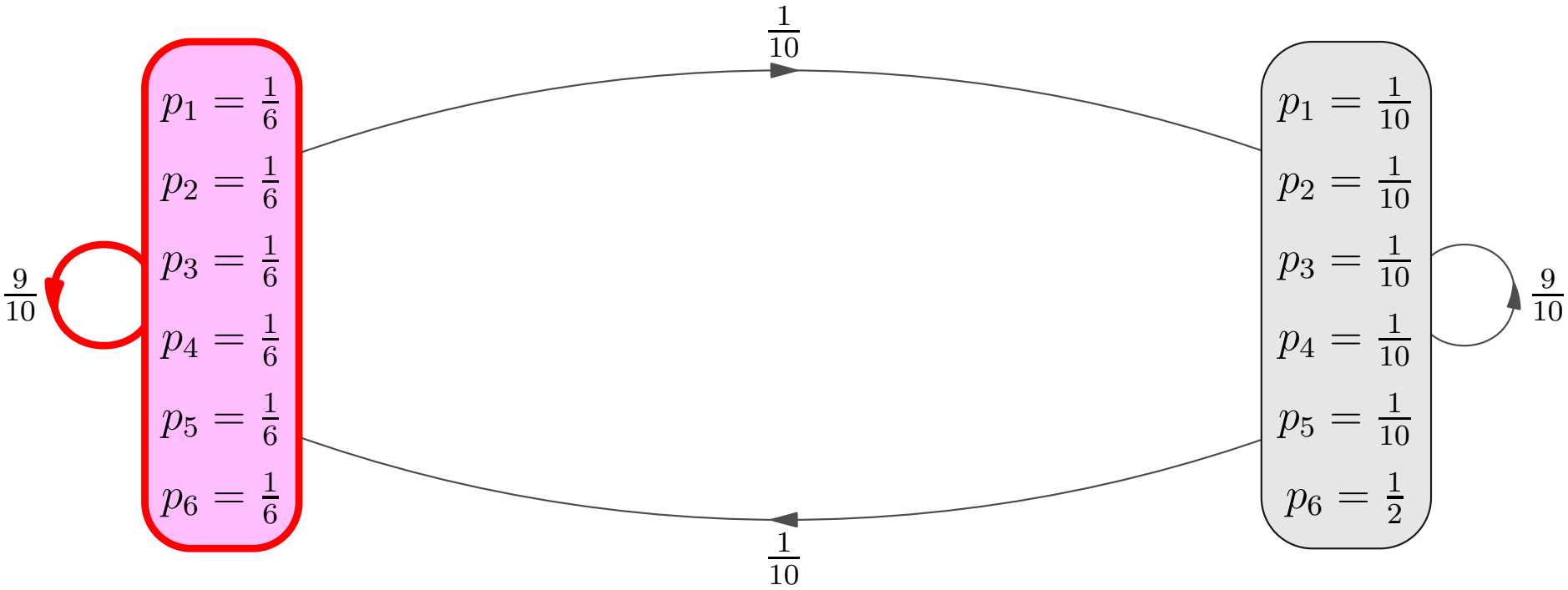

4156631666666565
HHHDDDDDDDDDDHHH

# Instance of a Dishonest Casino



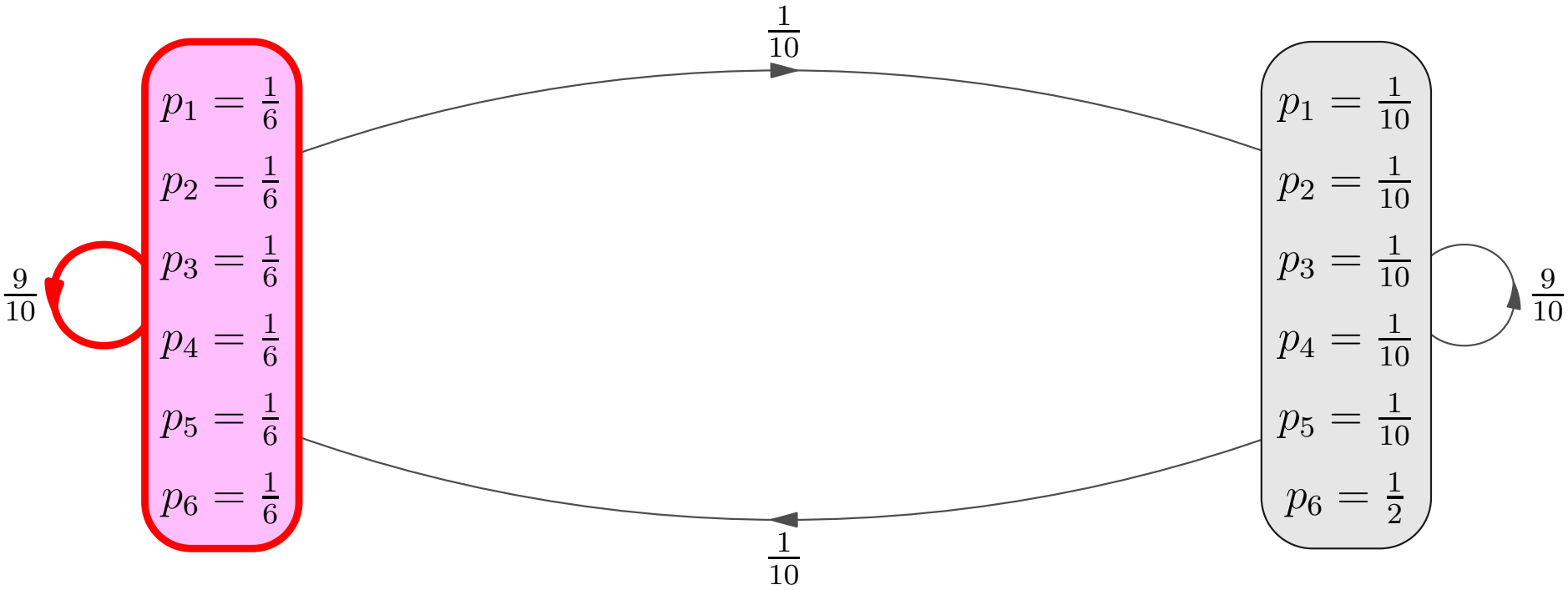

41566316666665652
HHHDDDDDDDDDDHHHH

# Instance of a Dishonest Casino



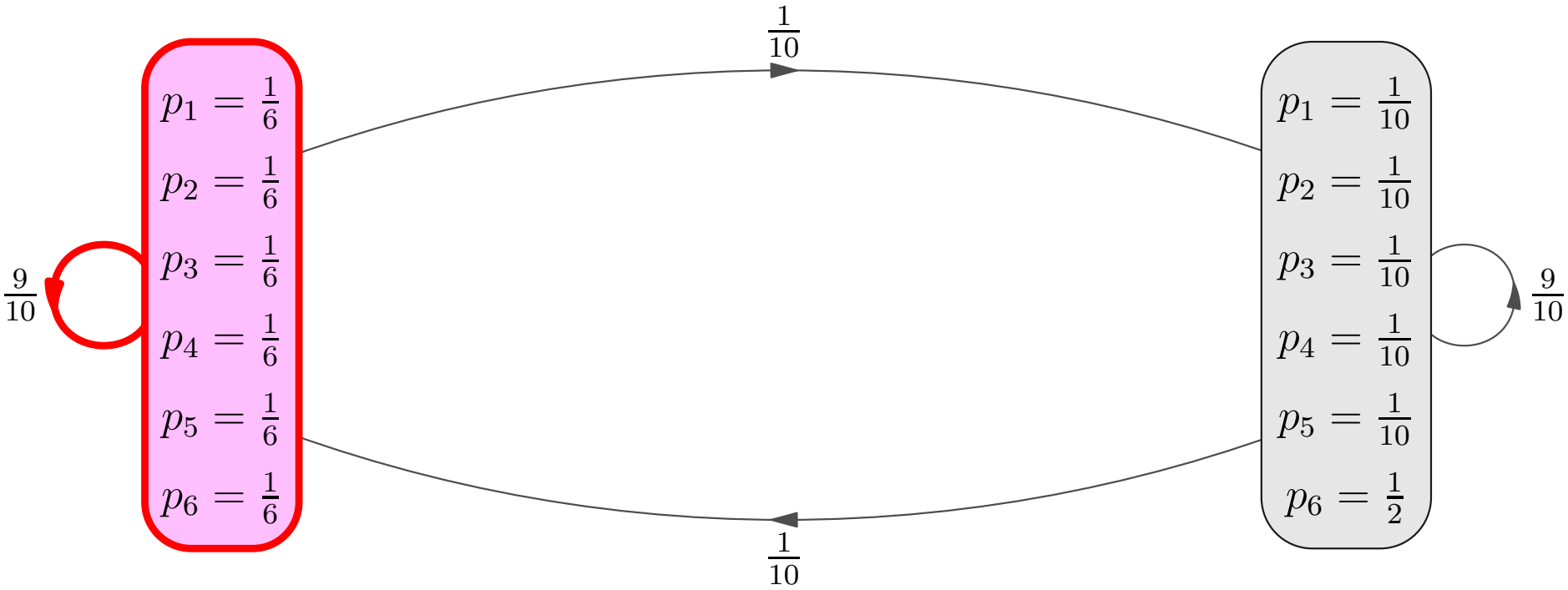

415663166666656524
HHHDDDDDDDDDDHHHHH

# Instance of a Dishonest Casino



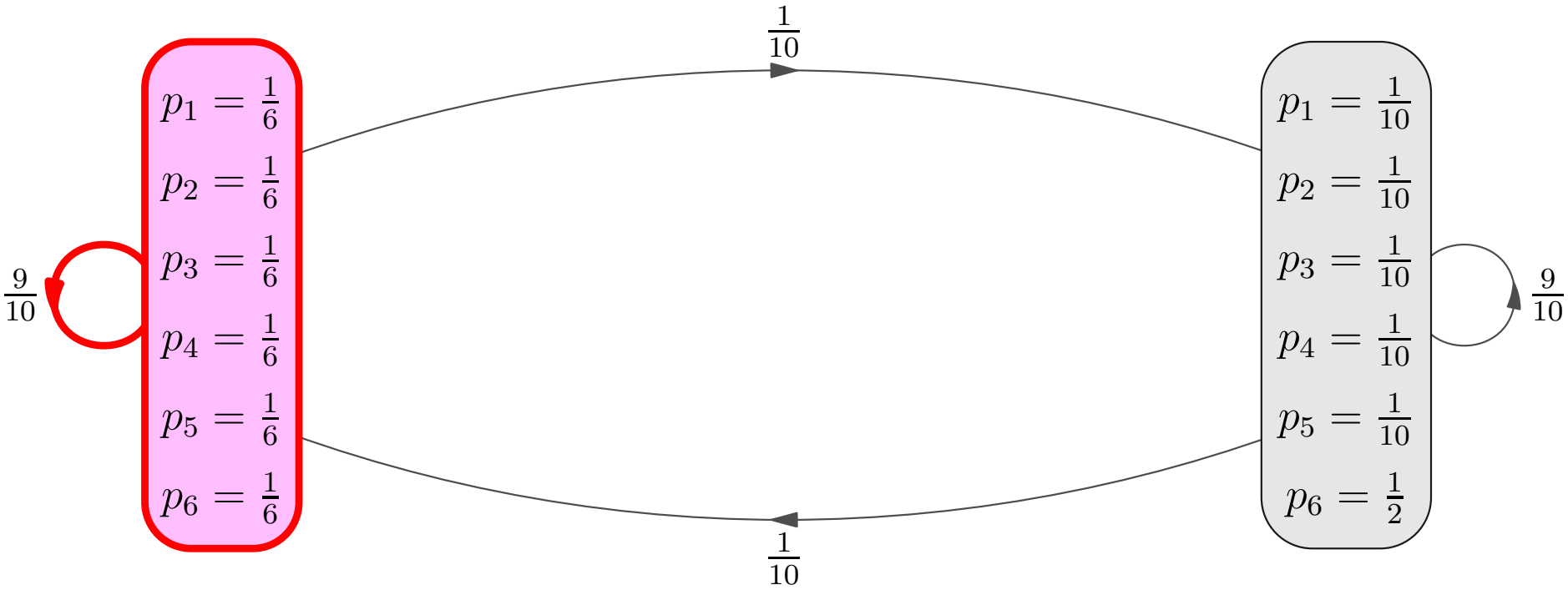

4156631666666565246
HHHDDDDDDDDDDHHHHHH

# Instance of a Dishonest Casino



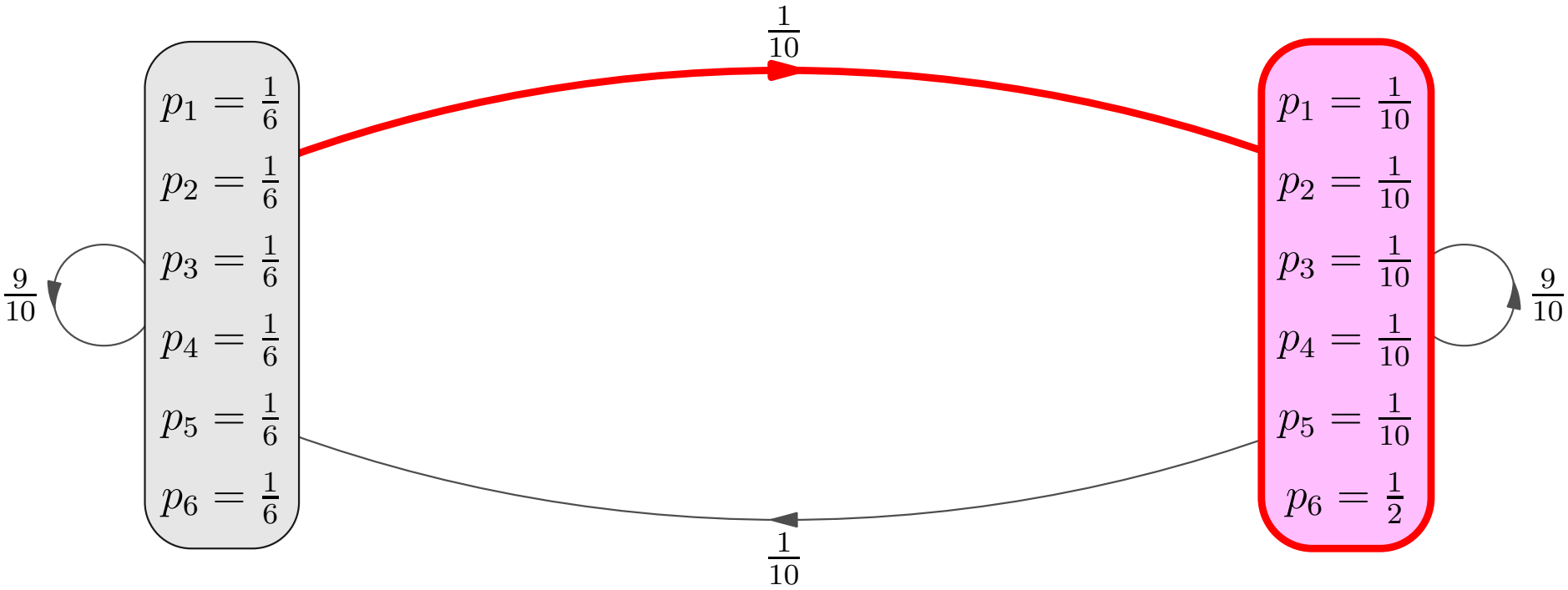

41566316666665652461
HHHDDDDDDDDDDHHHHHHH

# Instance of a Dishonest Casino



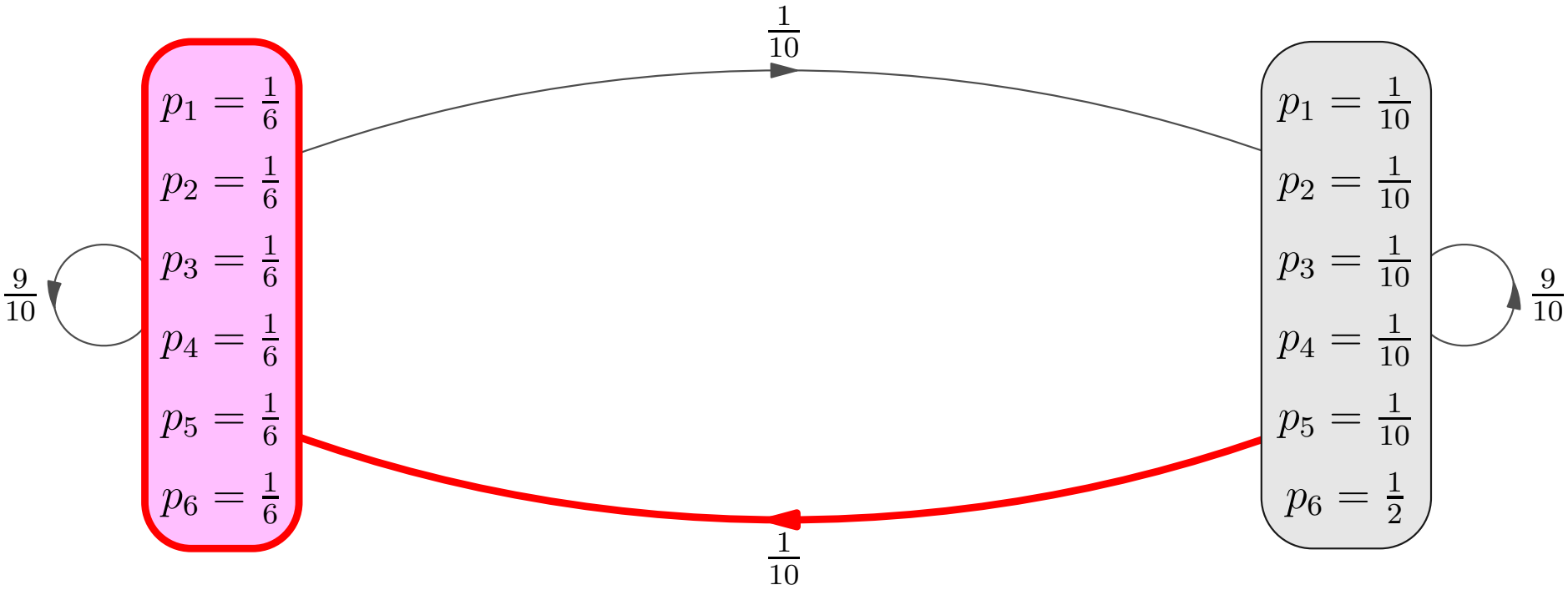

```
41566316666665652461 6
HHHDDDDDDDDDDHHHHHHHH
```

# Instance of a Dishonest Casino



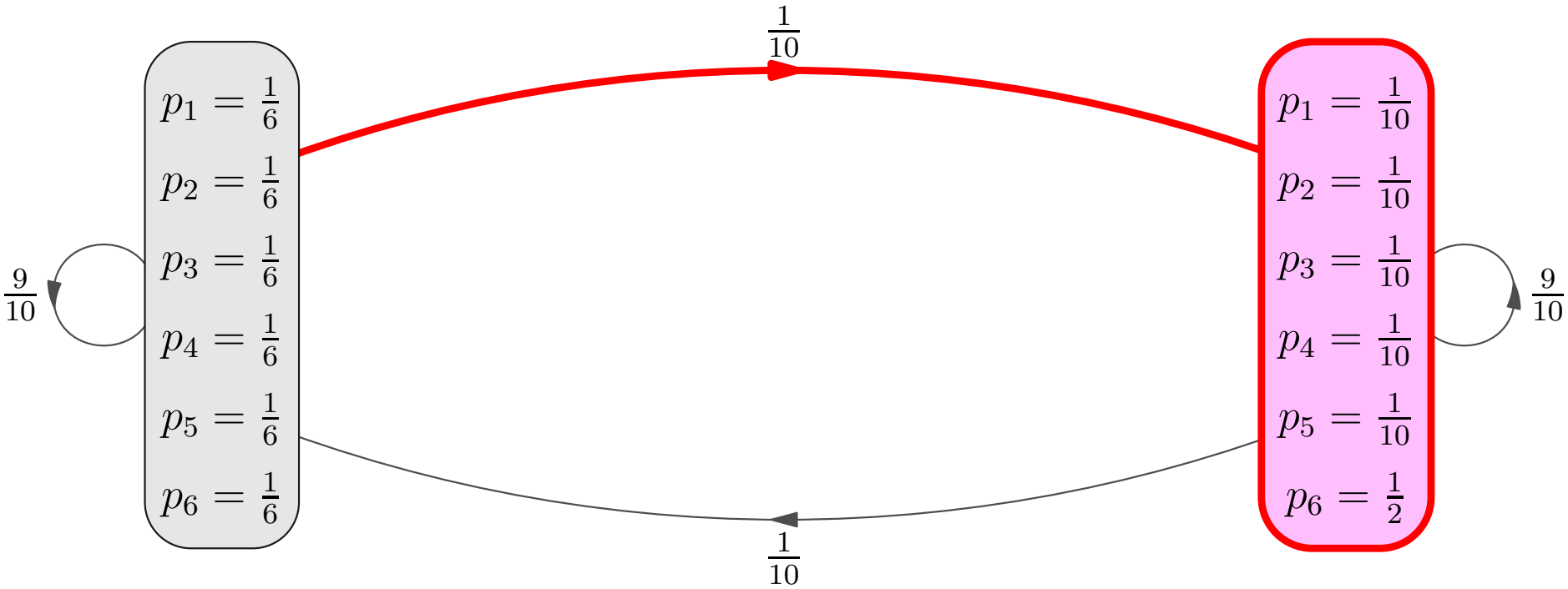

415663166666656524616436
HHHDDDDDDDDDDHHHHHHHHHHHD

# Instance of a Dishonest Casino



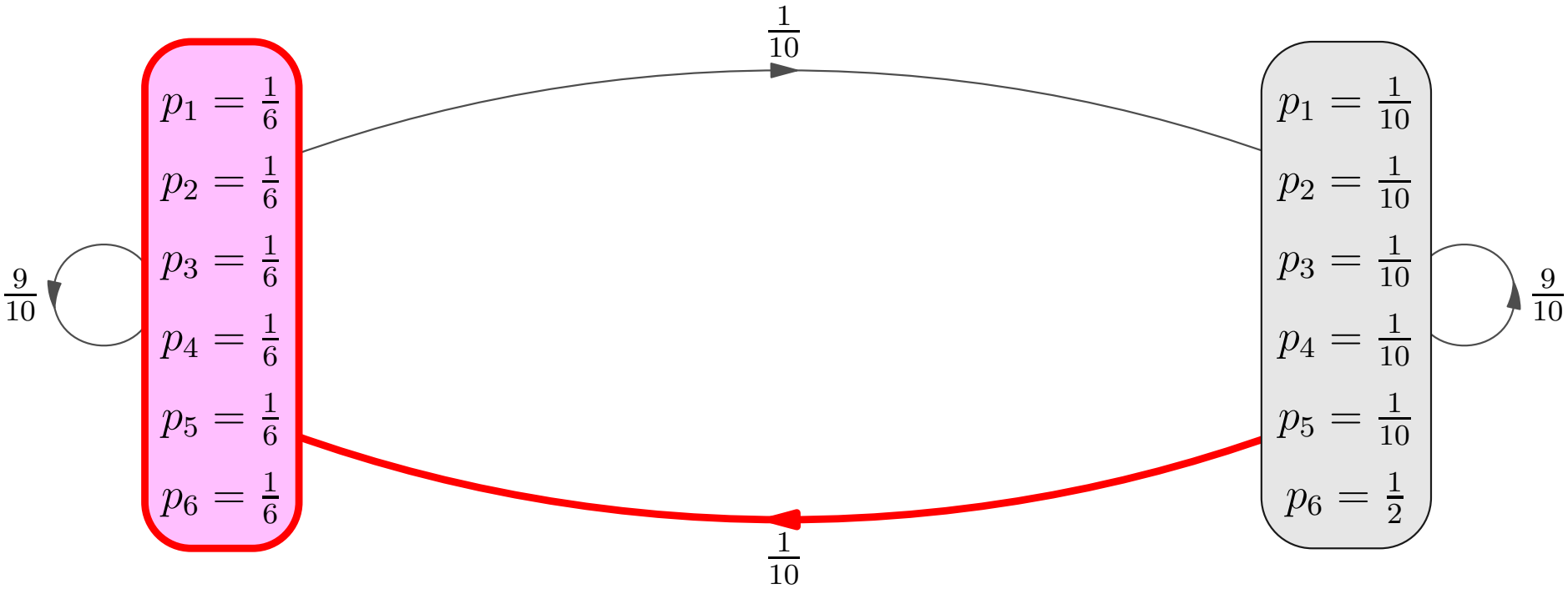

4156631666666565246164366663366563
HHHDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDH

# Instance of a Dishonest Casino



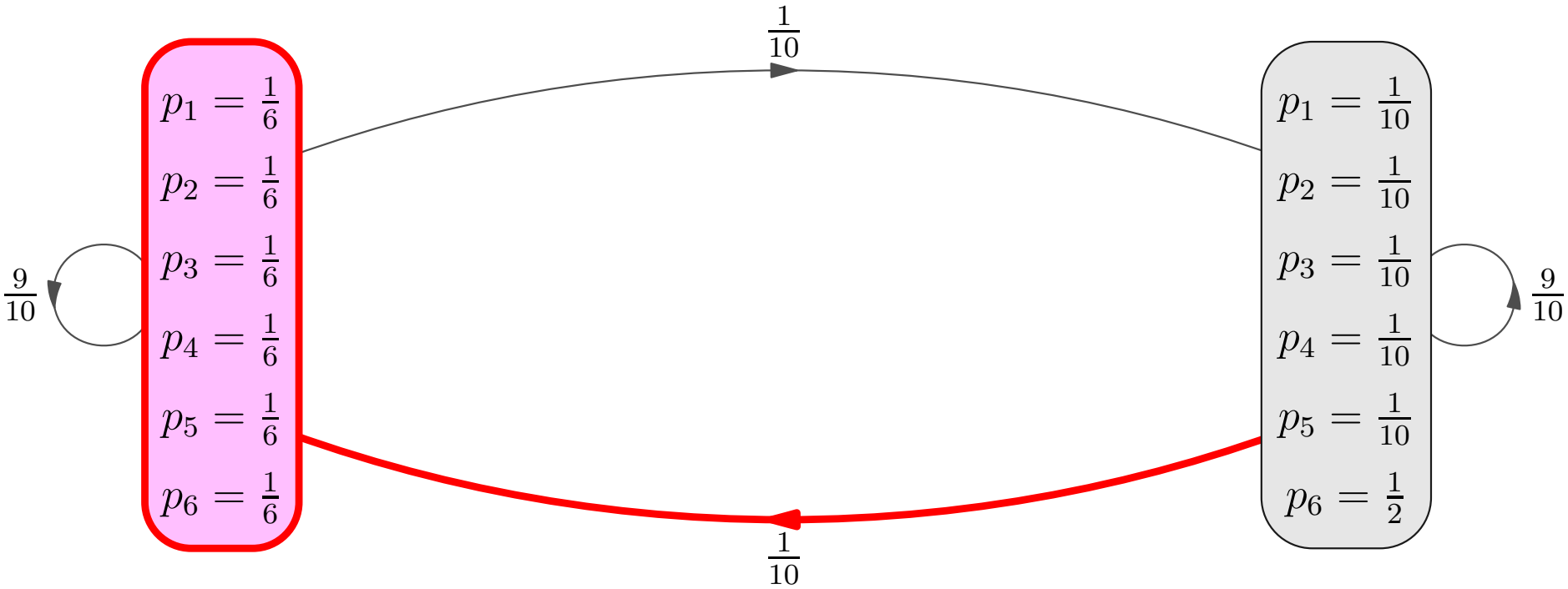

415663166666656524616436666336656321644666 16
HHHDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDHHHHHHHHHHD

# Instance of a Dishonest Casino




415663166666656524616436666336656321644666166626665145616656636 2
HHHDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDDDDDDDDDDDDH

# Instance of a Dishonest Casino




415663166666656524616436666336656321644666166626665145616656636 2
HHHDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDDDDDDDDDDDDH

# HMM Terminology

- We consider a sequence consisting of letters $\zeta$ from some alphabet $\mathcal{A}$
- We denote our sequence by $\boldsymbol{\xi} = (\xi_1, \xi_2, \ldots, \xi_T)$
- We let the set of states be denote by $\mathcal{Q}$
- The sequence of states we denote by $\boldsymbol{q} = (q_0, q_1, \ldots, q_T)$
- In HMMs we often have an initial state which we transition out of before doing anything else

# HMM Terminology

- We consider a sequence consisting of letters $\zeta$ from some alphabet $\mathcal{A}$
- We denote our sequence by $\boldsymbol{\xi} = (\xi_1, \xi_2, \ldots, \xi_T)$
- We let the set of states be denote by $\mathcal{Q}$
- The sequence of states we denote by $\boldsymbol{q} = (q_0, q_1, \ldots, q_T)$
- In HMMs we often have an initial state which we transition out of before doing anything else

# HMM Terminology

- We consider a sequence consisting of letters $\zeta$ from some alphabet $\mathcal{A}$
- We denote our sequence by $\boldsymbol{\xi} = (\xi_1, \xi_2, \ldots, \xi_T)$
- We let the set of states be denote by $\mathcal{Q}$
- The sequence of states we denote by $\boldsymbol{q} = (q_0, q_1, \ldots, q_T)$
- In HMMs we often have an initial state which we transition out of before doing anything else

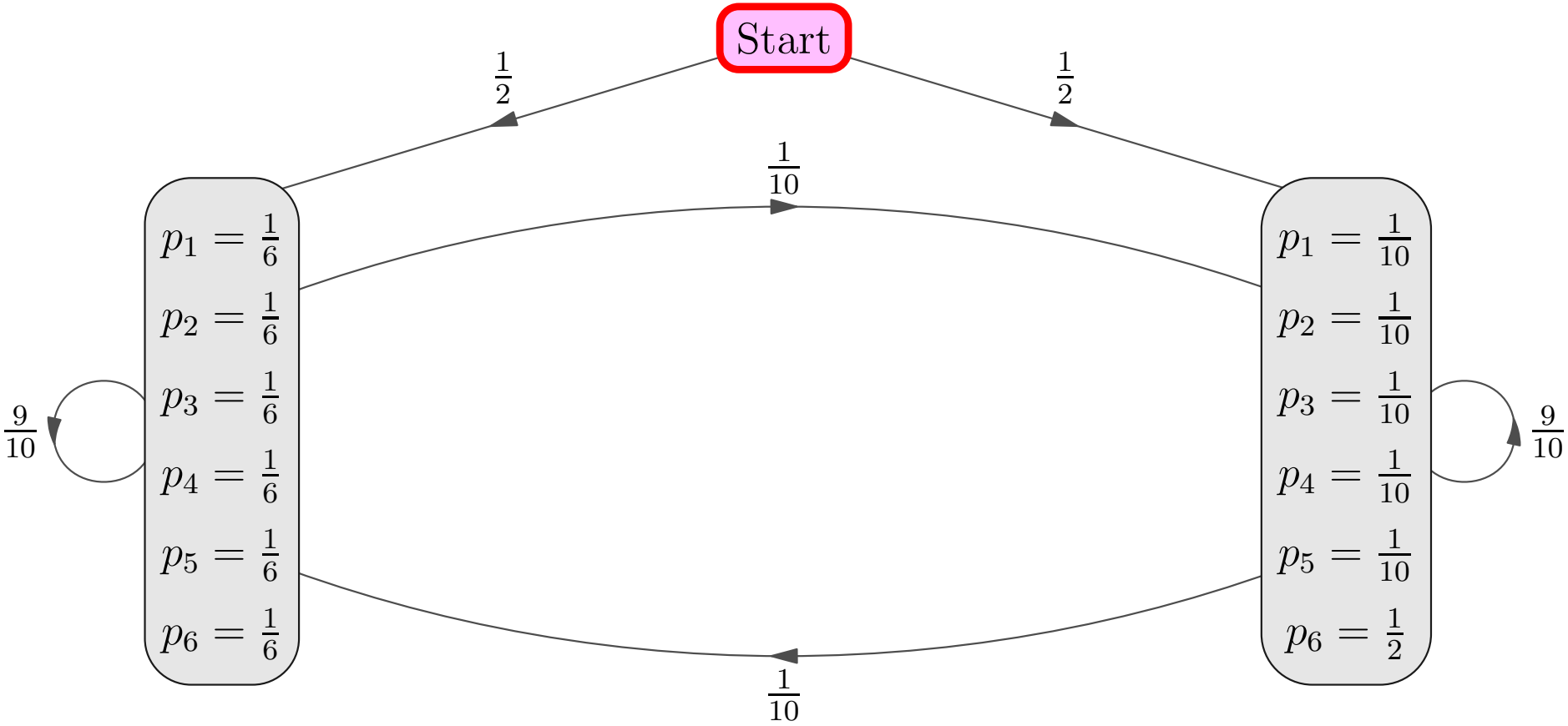
# HMM Terminology

- We consider a sequence consisting of letters $\zeta$ from some alphabet $\mathcal{A}$
- We denote our sequence by $\boldsymbol{\xi} = (\xi_1, \xi_2, \ldots, \xi_T)$
- We let the set of states be denote by $\mathcal{Q}$
- The sequence of states we denote by $\boldsymbol{q} = (q_0, q_1, \ldots, q_T)$
- In HMMs we often have an initial state which we transition out of before doing anything else

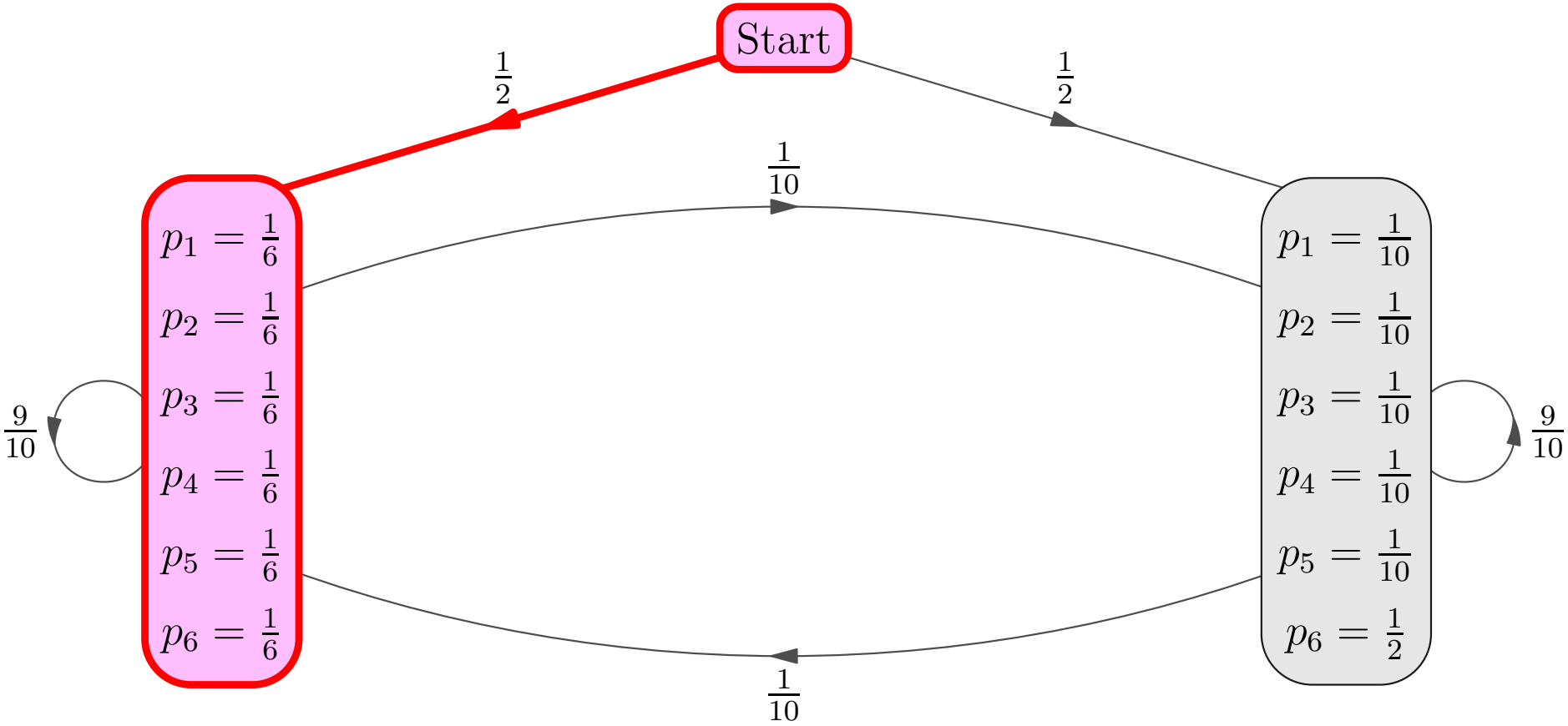
# HMM Terminology

- We consider a sequence consisting of letters $\zeta$ from some alphabet $\mathcal{A}$
- We denote our sequence by $\boldsymbol{\xi} = (\xi_1, \xi_2, \ldots, \xi_T)$
- We let the set of states be denote by $\mathcal{Q}$
- The sequence of states we denote by $\boldsymbol{q} = (q_0, q_1, \ldots, q_T)$
- In HMMs we often have an initial state which we transition out of before doing anything else

# Start State

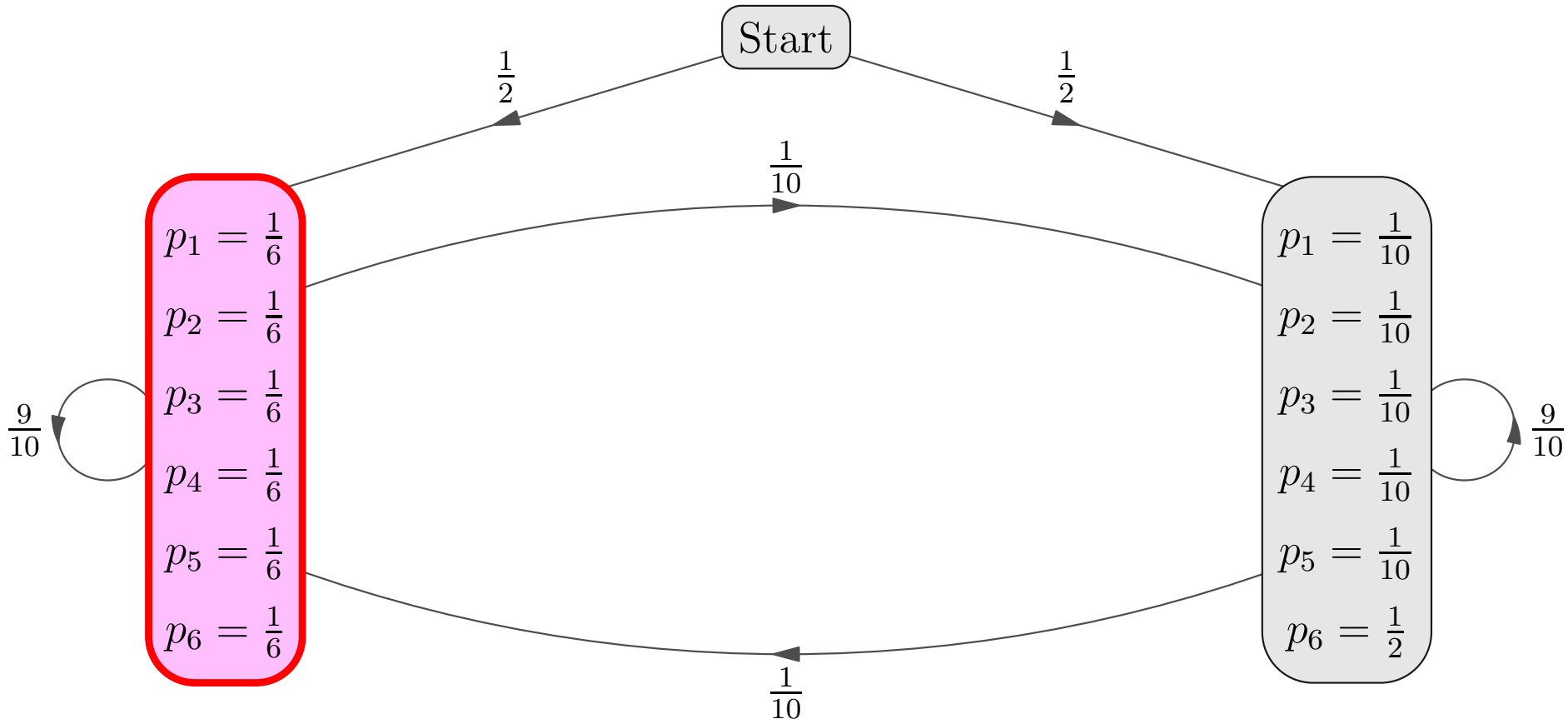

Start
$\frac{1}{2}$
$\frac{1}{2}$
$\frac{1}{10}$
$p_1 = \frac{1}{6}$
$p_2 = \frac{1}{6}$
$p_3 = \frac{1}{6}$
$p_4 = \frac{1}{6}$
$p_5 = \frac{1}{6}$
$p_6 = \frac{1}{6}$
$p_1 = \frac{1}{10}$
$p_2 = \frac{1}{10}$
$p_3 = \frac{1}{10}$
$p_4 = \frac{1}{10}$
$p_5 = \frac{1}{10}$
$p_6 = \frac{1}{2}$
$\frac{9}{10}$
$\frac{9}{10}$
$\frac{1}{10}$

# Start State



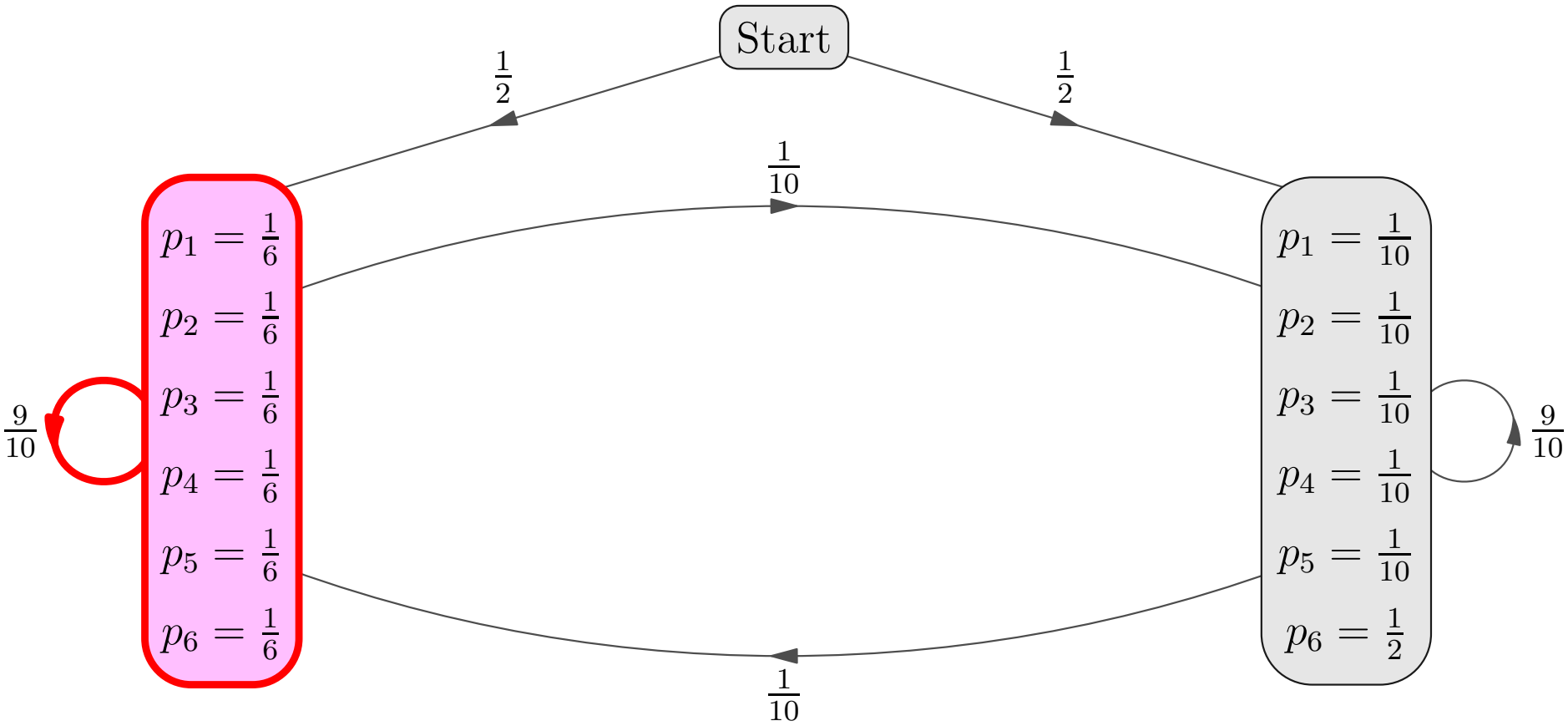

H

# Start State




4
H

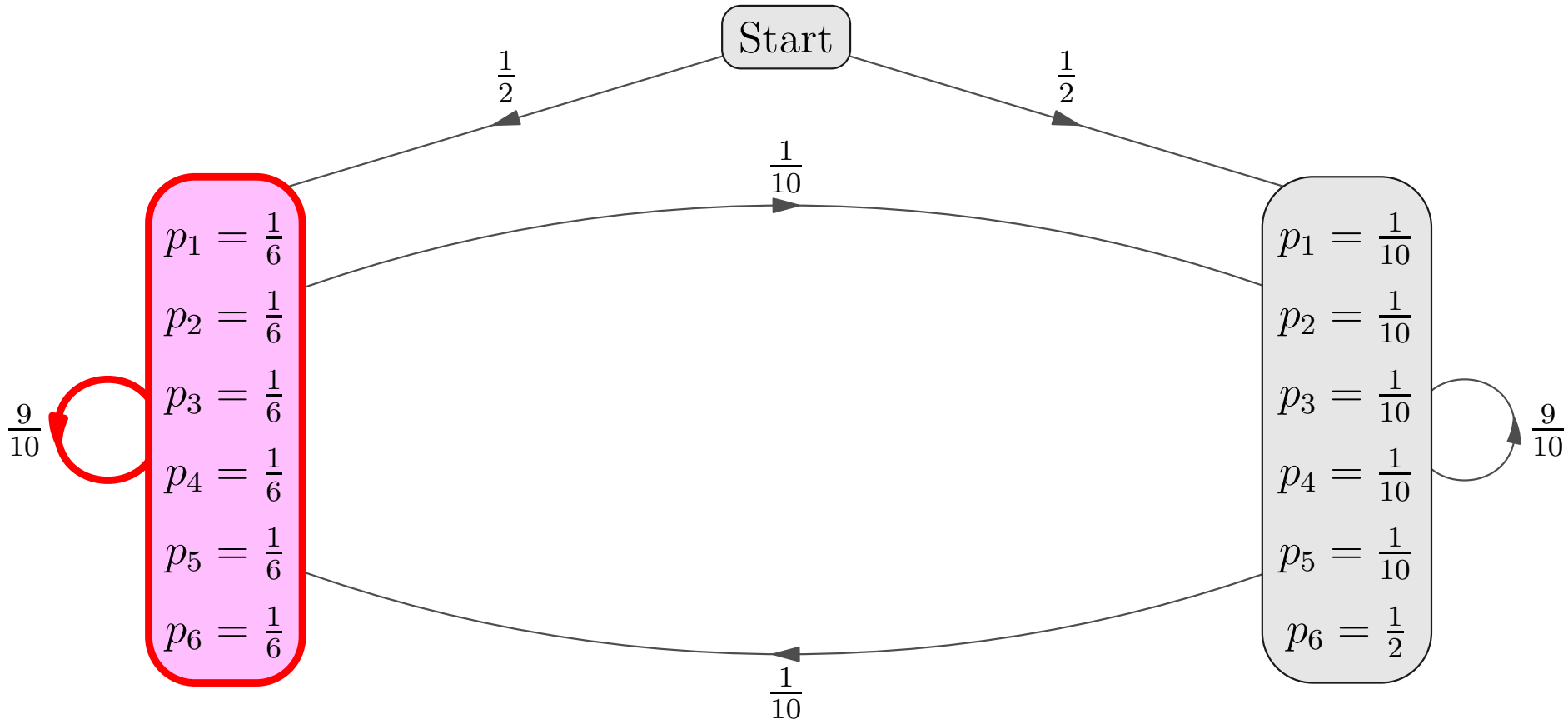
# Start State




4

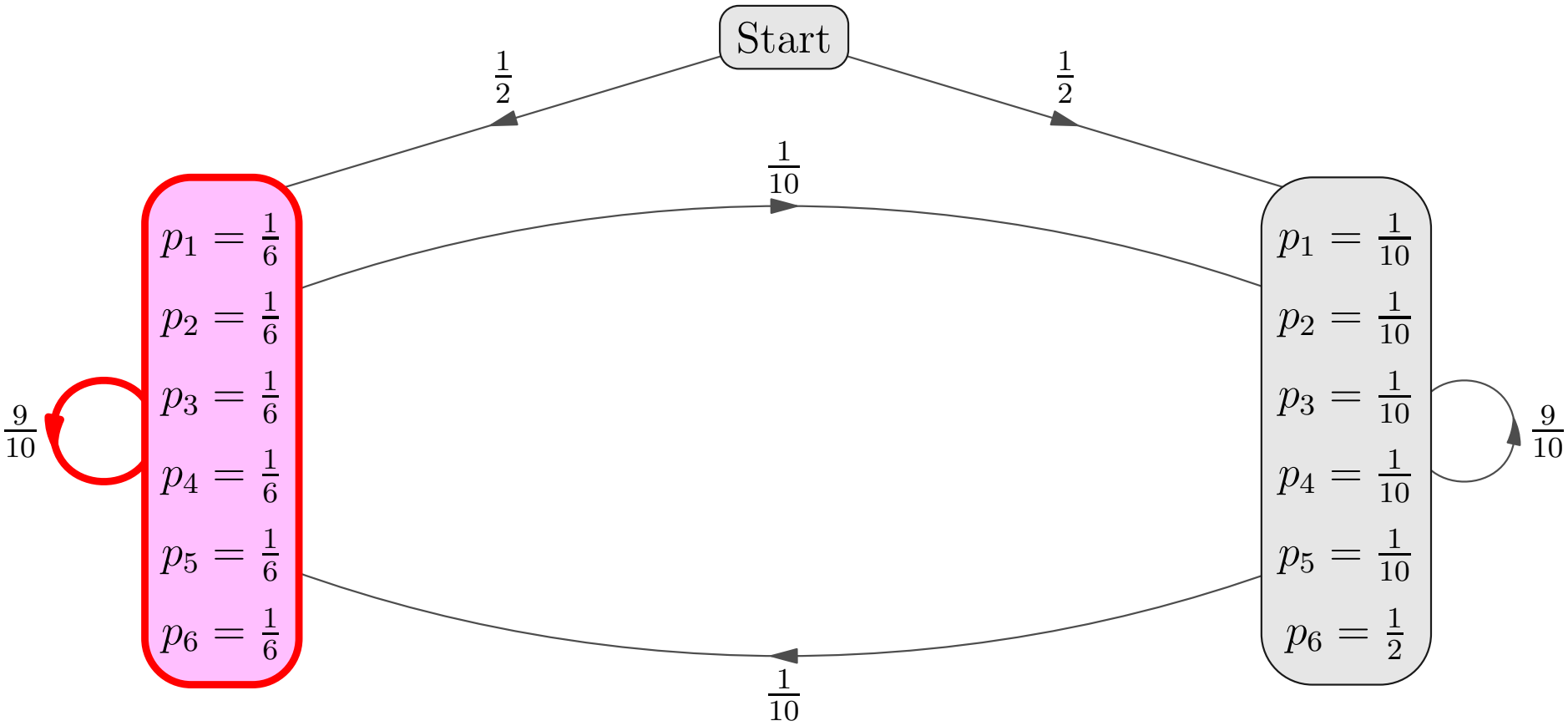
HH

# Start State



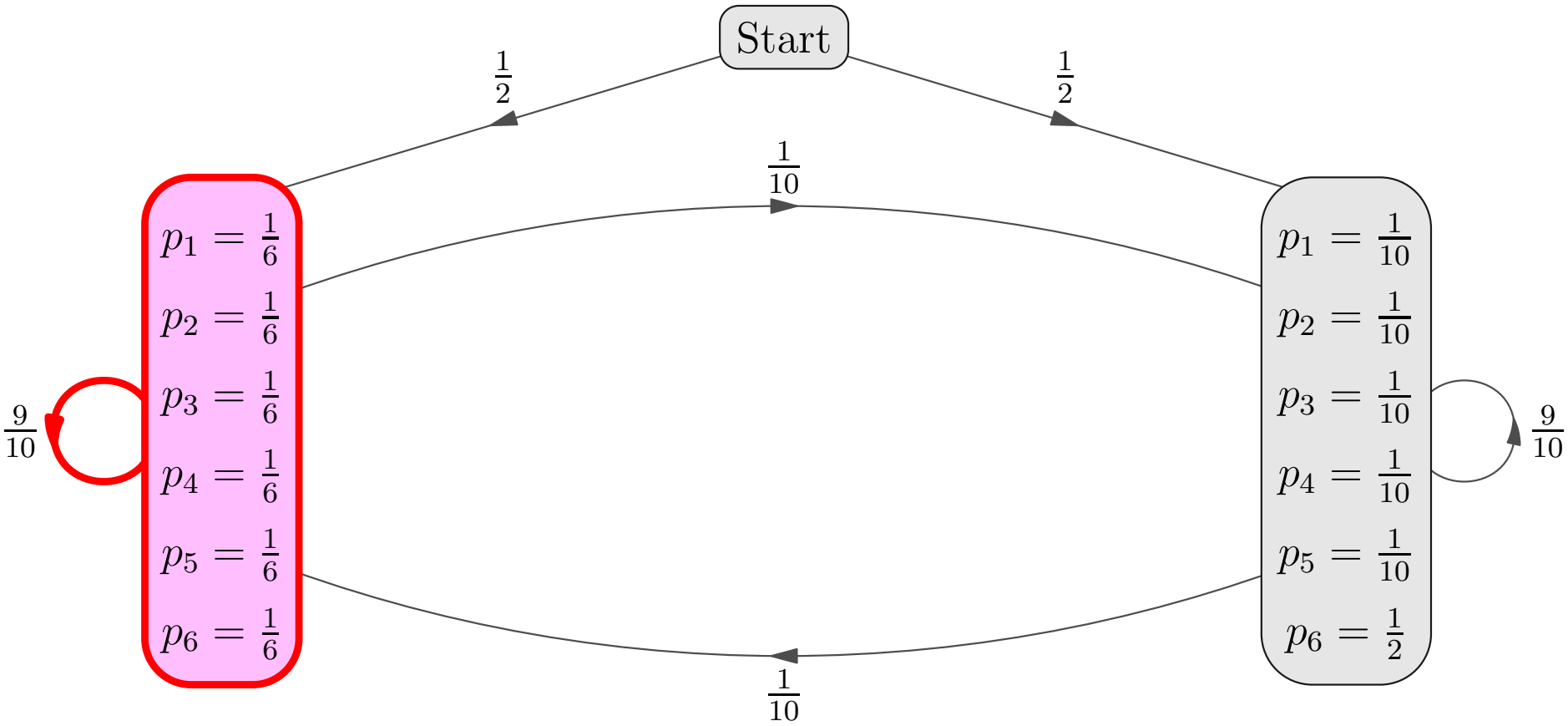

41
HH

# Start State



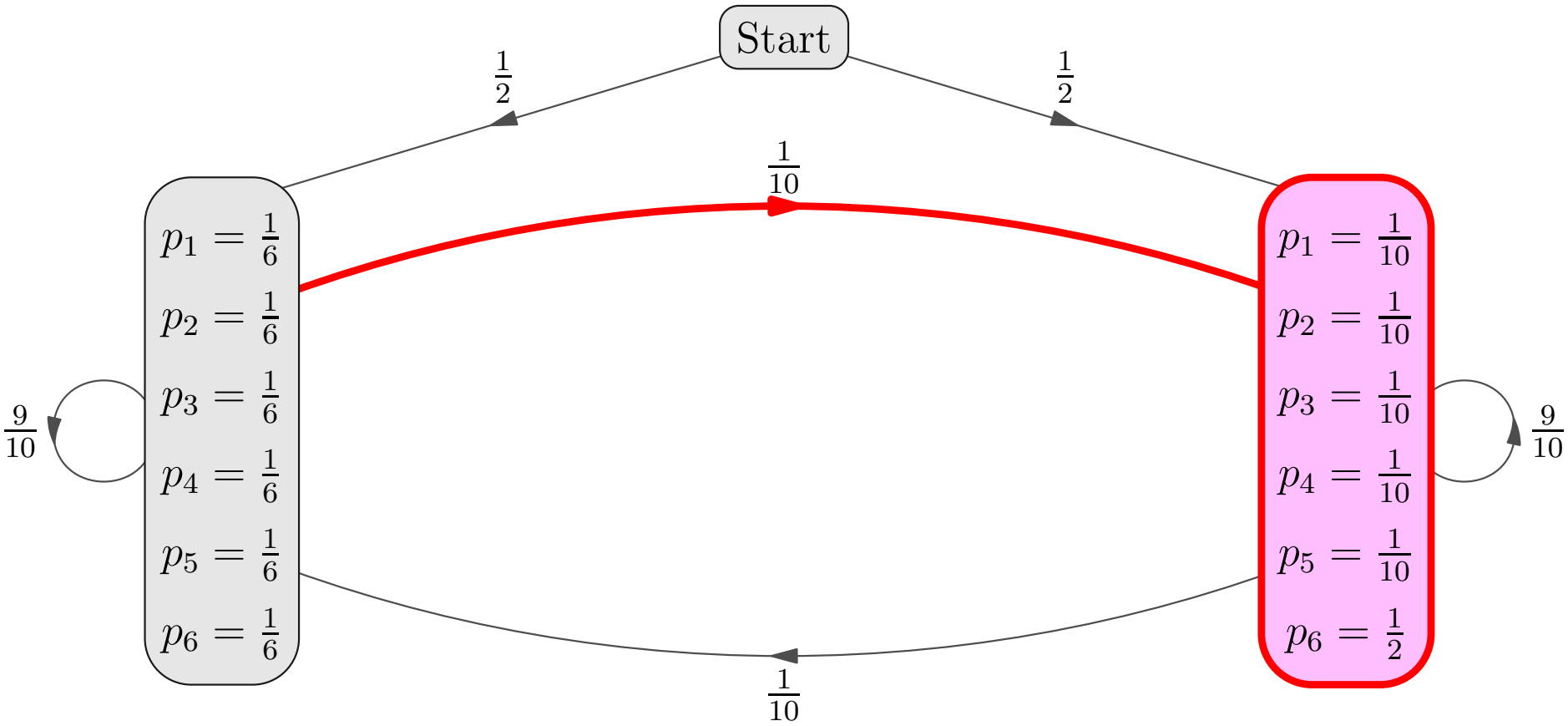

41
HHH

# Start State



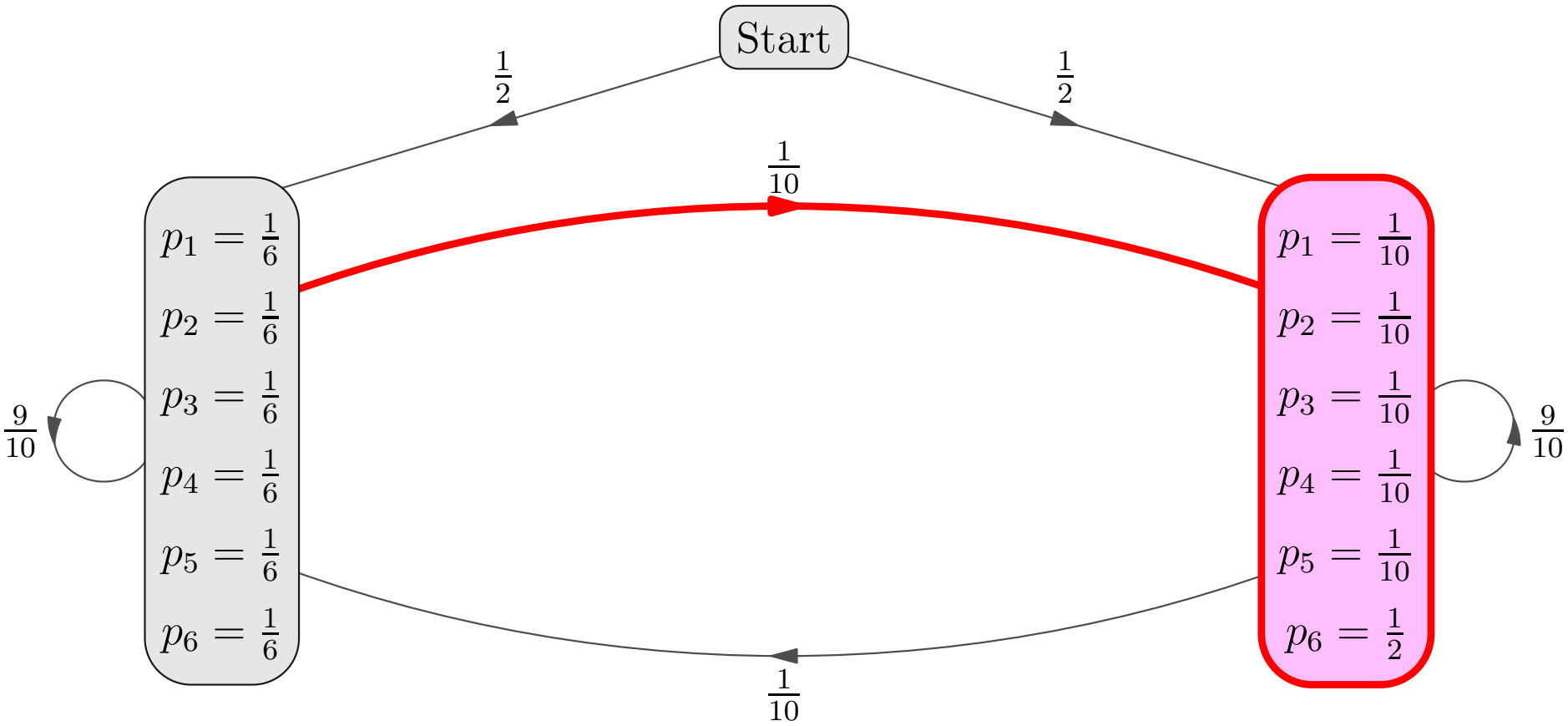

415
HHH

# Start State



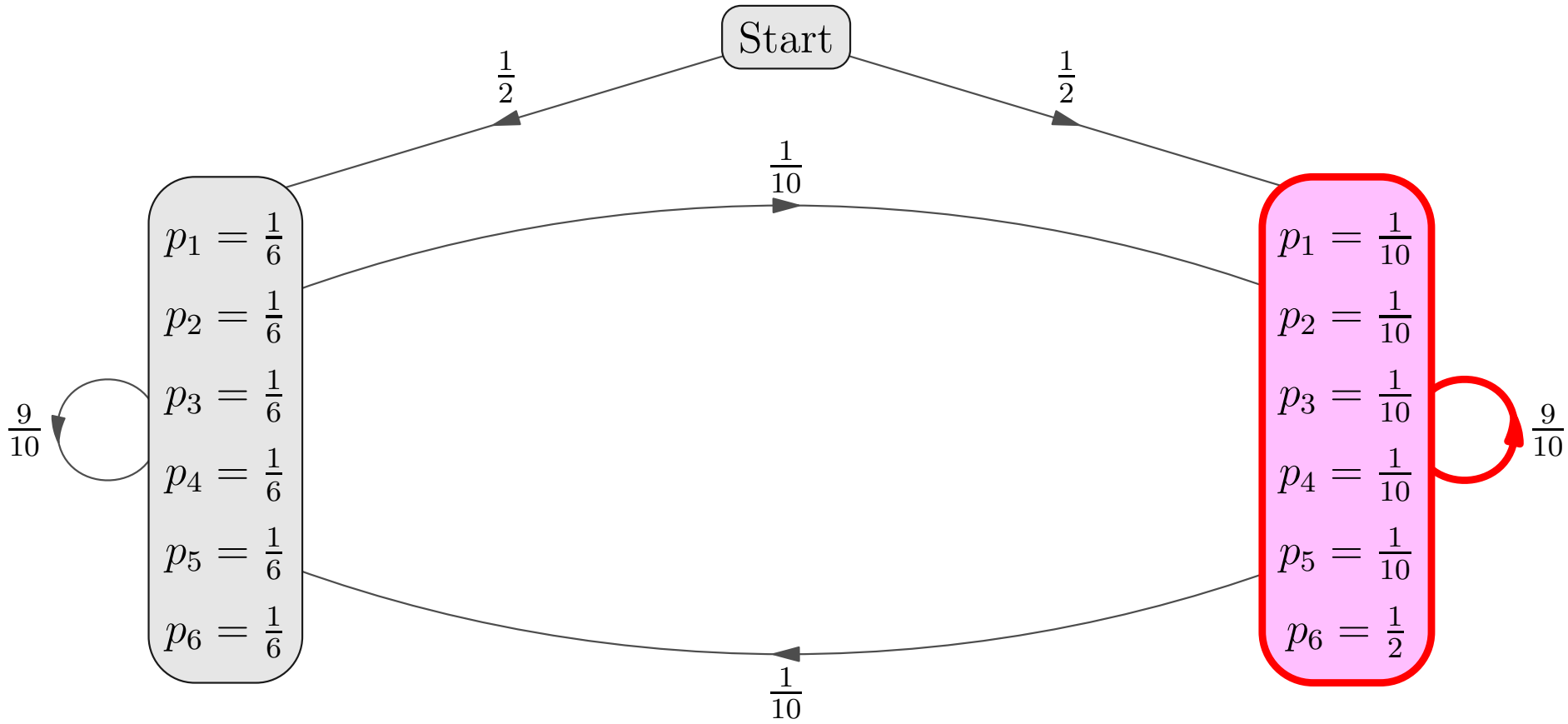

415
HHHD

# Start State



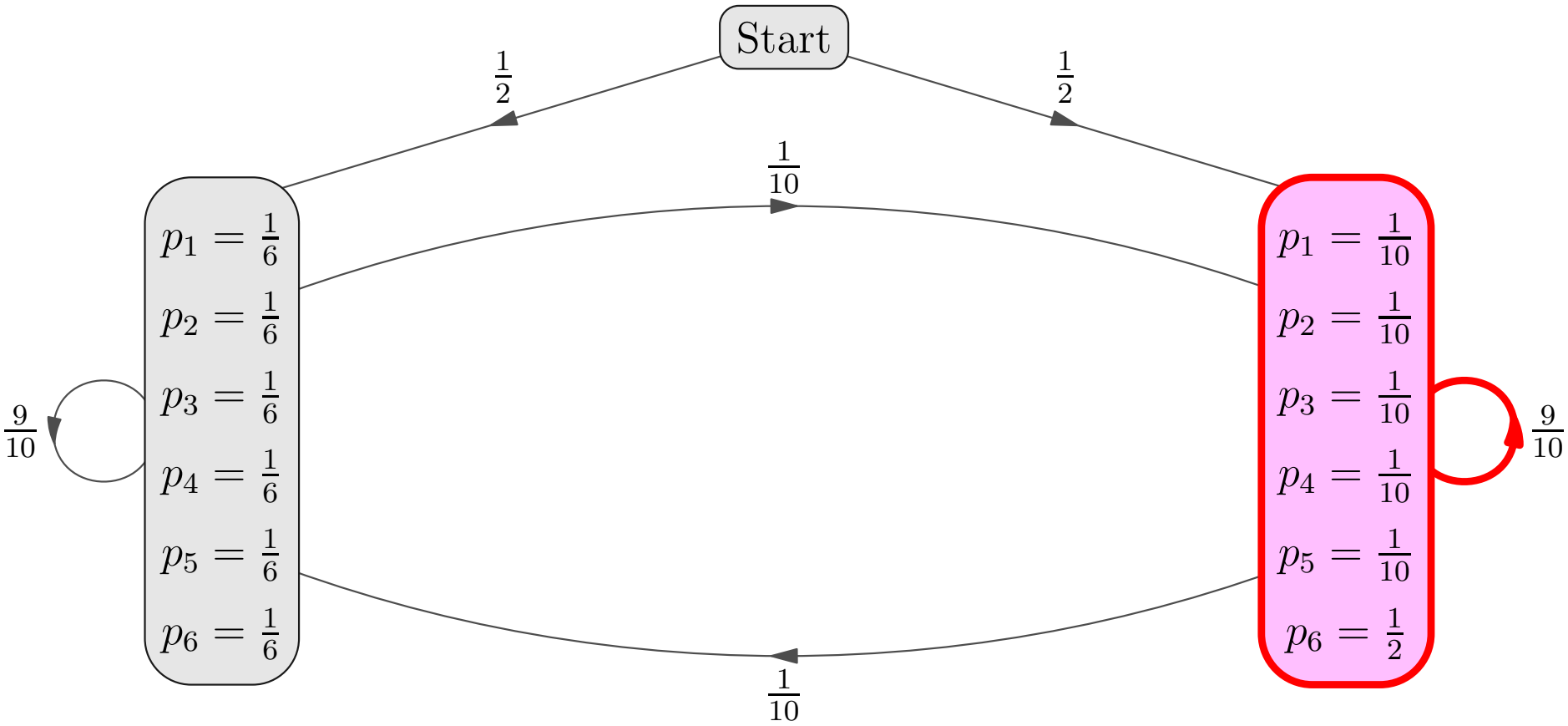

4156
HHHD

# Start State




41566
HHHDD

# Start State




415663
HHHDDD

# Parameter of HMM

- The **emission probability** is the probability of emitting a letter $\zeta$ in state $i$ and is denoted

$$e_i(\zeta) = \mathbb{P}\left(\zeta | q_t = i\right)$$

- The **transition probability** of moving from state $i$ to state $j$ is denoted

$$a_{ij} = \mathbb{P}\left(q_t = j | q_{t-1} = i\right)$$

- The set of emission and transition probabilities we denote by $\boldsymbol{\theta}$

# Parameter of HMM

- The **emission probability** is the probability of emitting a letter $\zeta$ in state $i$ and is denoted

$$e_i(\zeta) = \mathbb{P}\left(\zeta | q_t = i\right)$$

- The **transition probability** of moving from state $i$ to state $j$ is denoted

$$a_{ij} = \mathbb{P}\left(q_t = j | q_{t-1} = i\right)$$

- The set of emission and transition probabilities we denote by $\boldsymbol{\theta}$

# Parameter of HMM

- The **emission probability** is the probability of emitting a letter $\zeta$ in state $i$ and is denoted

$$e_i(\zeta) = \mathbb{P}\left(\zeta | q_t = i\right)$$

- The **transition probability** of moving from state $i$ to state $j$ is denoted

$$a_{ij} = \mathbb{P}\left(q_t = j | q_{t-1} = i\right)$$

- The set of emission and transition probabilities we denote by $\boldsymbol{\theta}$

# Viterbi Algorithm

- To find the most likely set of states associated with a sequence $\boldsymbol{\xi}$ we can use a classic dynamic programming algorithm
- This is known as the Viterbi algorithm
- Let $v_i(t)$ be the probability of being in state $i$ at time $t$
- Initially $v_0(0) = 1$ and $v_i(0) = 0$ where state 0 is the initial state
- We iteratively compute for each state $i \in \mathcal{Q}$

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

# Viterbi Algorithm

- To find the most likely set of states associated with a sequence $\boldsymbol{\xi}$ we can use a classic dynamic programming algorithm
- This is known as the Viterbi algorithm
- Let $v_i(t)$ be the probability of being in state $i$ at time $t$
- Initially $v_0(0) = 1$ and $v_i(0) = 0$ where state 0 is the initial state
- We iteratively compute for each state $i \in \mathcal{Q}$

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

# Viterbi Algorithm

- To find the most likely set of states associated with a sequence $\boldsymbol{\xi}$ we can use a classic dynamic programming algorithm
- This is known as the Viterbi algorithm
- Let $v_i(t)$ be the probability of being in state $i$ at time $t$
- Initially $v_0(0) = 1$ and $v_i(0) = 0$ where state 0 is the initial state
- We iteratively compute for each state $i \in \mathcal{Q}$

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

# Viterbi Algorithm

- To find the most likely set of states associated with a sequence $\boldsymbol{\xi}$ we can use a classic dynamic programming algorithm
- This is known as the Viterbi algorithm
- Let $v_i(t)$ be the probability of being in state $i$ at time $t$
- Initially $v_0(0) = 1$ and $v_i(0) = 0$ where state 0 is the initial state
- We iteratively compute for each state $i \in \mathcal{Q}$

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

# Viterbi Algorithm

- To find the most likely set of states associated with a sequence $\boldsymbol{\xi}$ we can use a classic dynamic programming algorithm
- This is known as the Viterbi algorithm
- Let $v_i(t)$ be the probability of being in state $i$ at time $t$
- Initially $v_0(0) = 1$ and $v_i(0) = 0$ where state 0 is the initial state
- We iteratively compute for each state $i \in \mathcal{Q}$

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

# Most Likely Path

- To compute the optimal path it helps to maintain a pointer

$$\mathrm{ptr}_t(i) = \operatorname*{argmax}_j \, v_j(t-1)\, a_{ji}$$

- The termination condition for Viterbi is

$$\mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q}^*\right) = \max_i v_i(T)$$

$$q_T^* = \operatorname*{argmax}_i v_i(T)$$

- The backward algorithm is

$$q_{t-1}^* = \mathrm{ptr}_t(q_t^*)$$

# Most Likely Path

- To compute the optimal path it helps to maintain a pointer

$$\mathrm{ptr}_t(i) = \operatorname*{argmax}_j \, v_j(t-1)\, a_{ji}$$

- The termination condition for Viterbi is

$$\mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q}^*\right) = \max_i v_i(T)$$

$$q_T^* = \operatorname*{argmax}_i v_i(T)$$

- The backward algorithm is

$$q_{t-1}^* = \mathrm{ptr}_t(q_t^*)$$

# Most Likely Path

- To compute the optimal path it helps to maintain a pointer

$$\mathrm{ptr}_t(i) = \operatorname*{argmax}_j \, v_j(t-1)\, a_{ji}$$

- The termination condition for Viterbi is

$$\mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q}^*\right) = \max_i v_i(T)$$

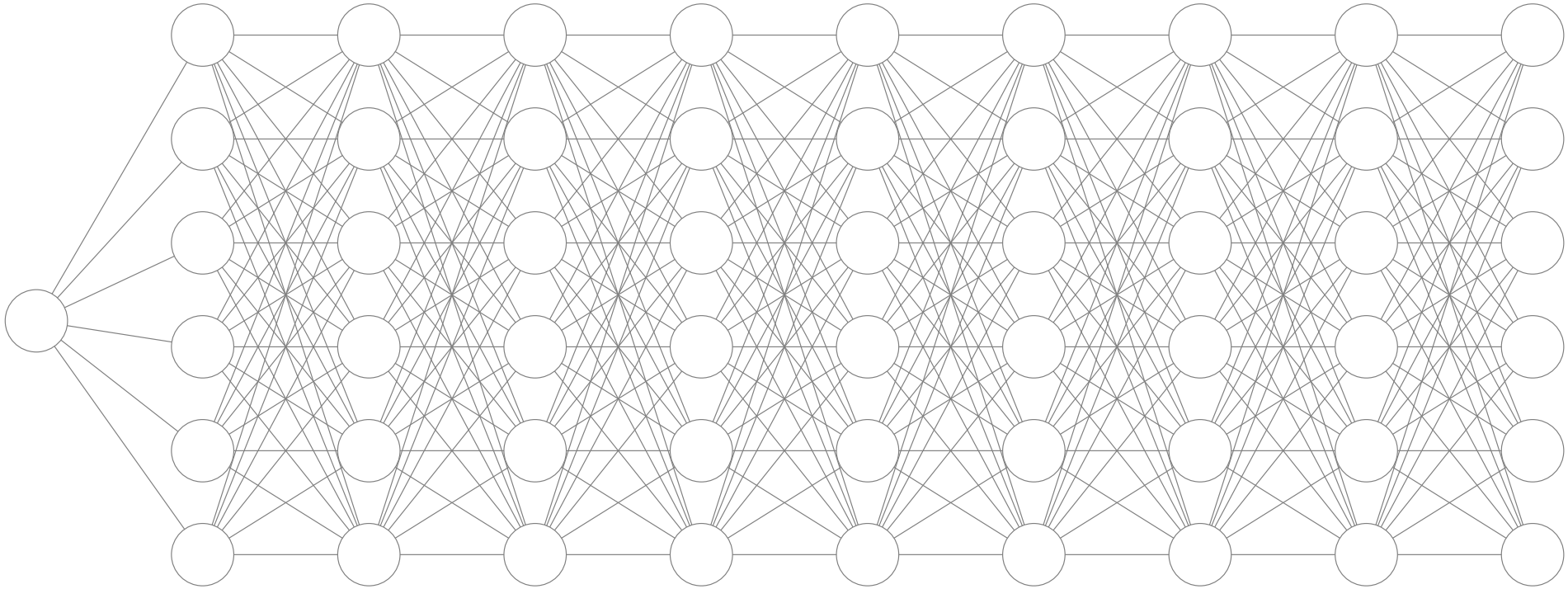
$$q_T^* = \operatorname*{argmax}_i v_i(T)$$

- The backward algorithm is

$$q_{t-1}^* = \mathrm{ptr}_t(q_t^*)$$

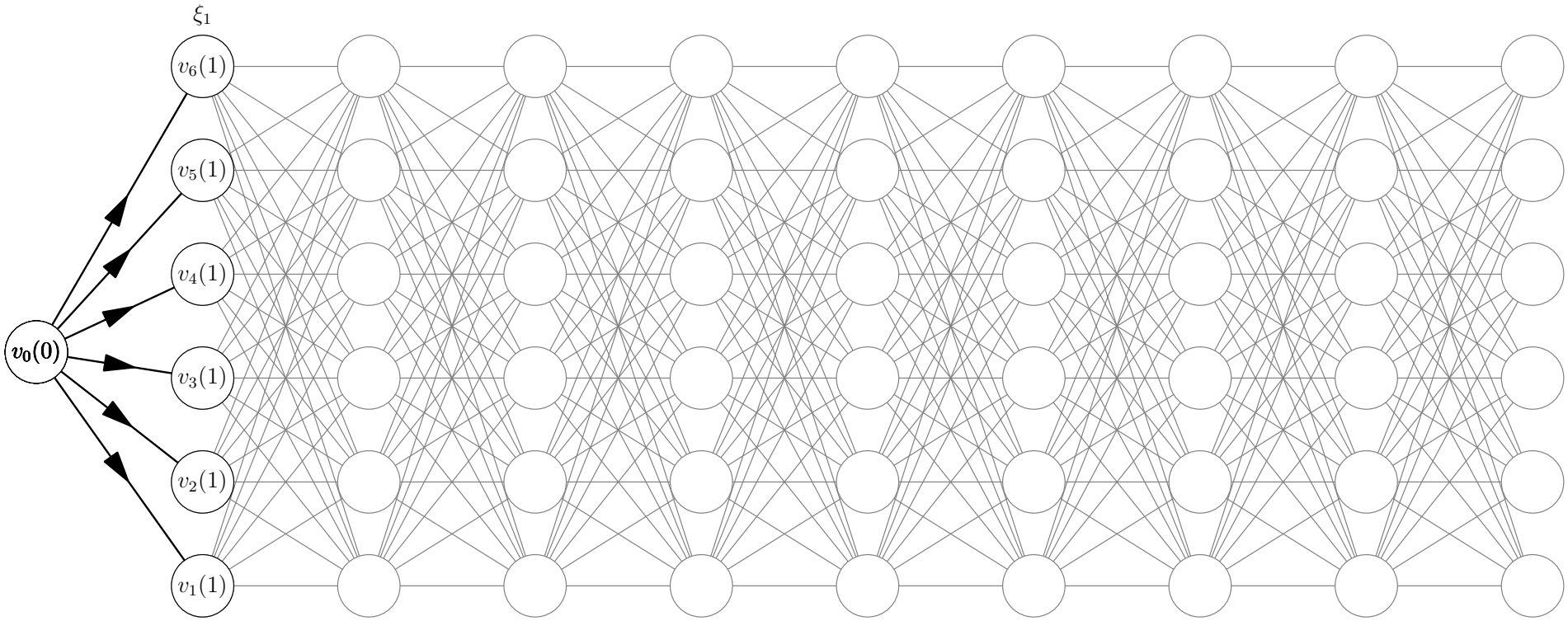
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



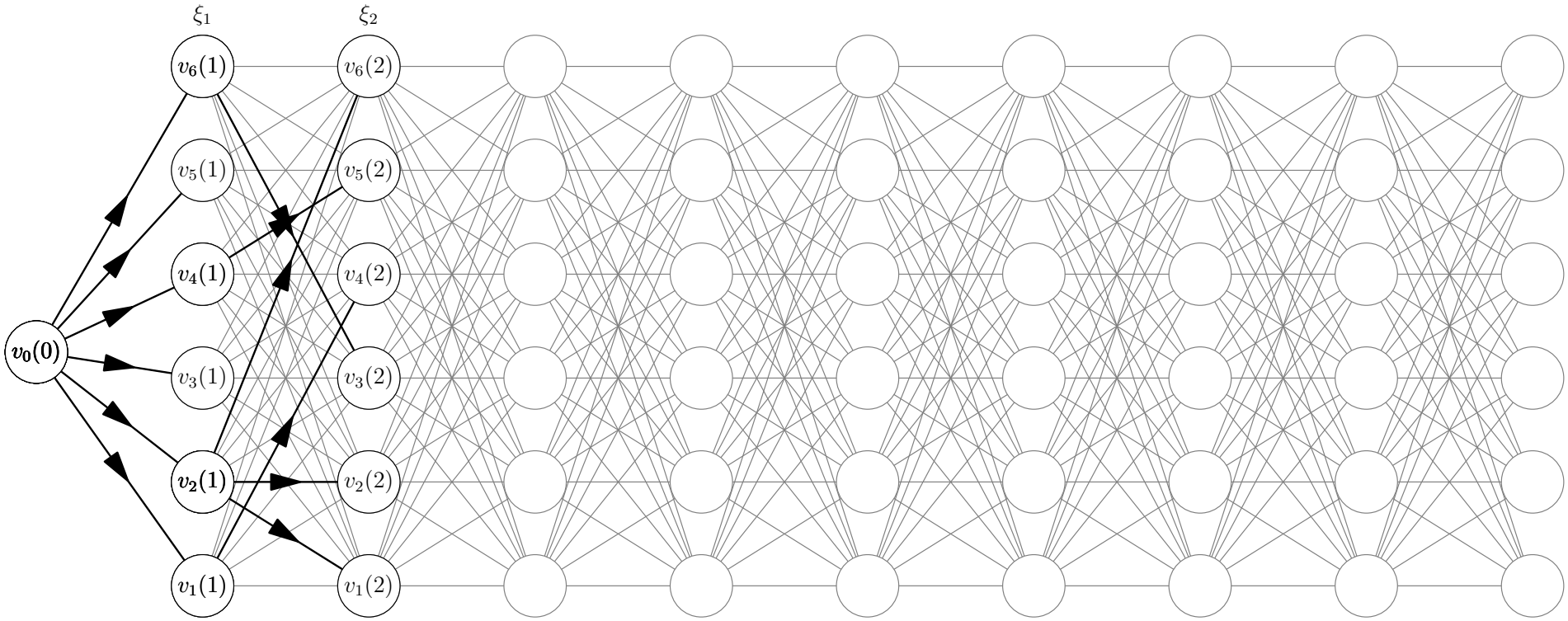
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



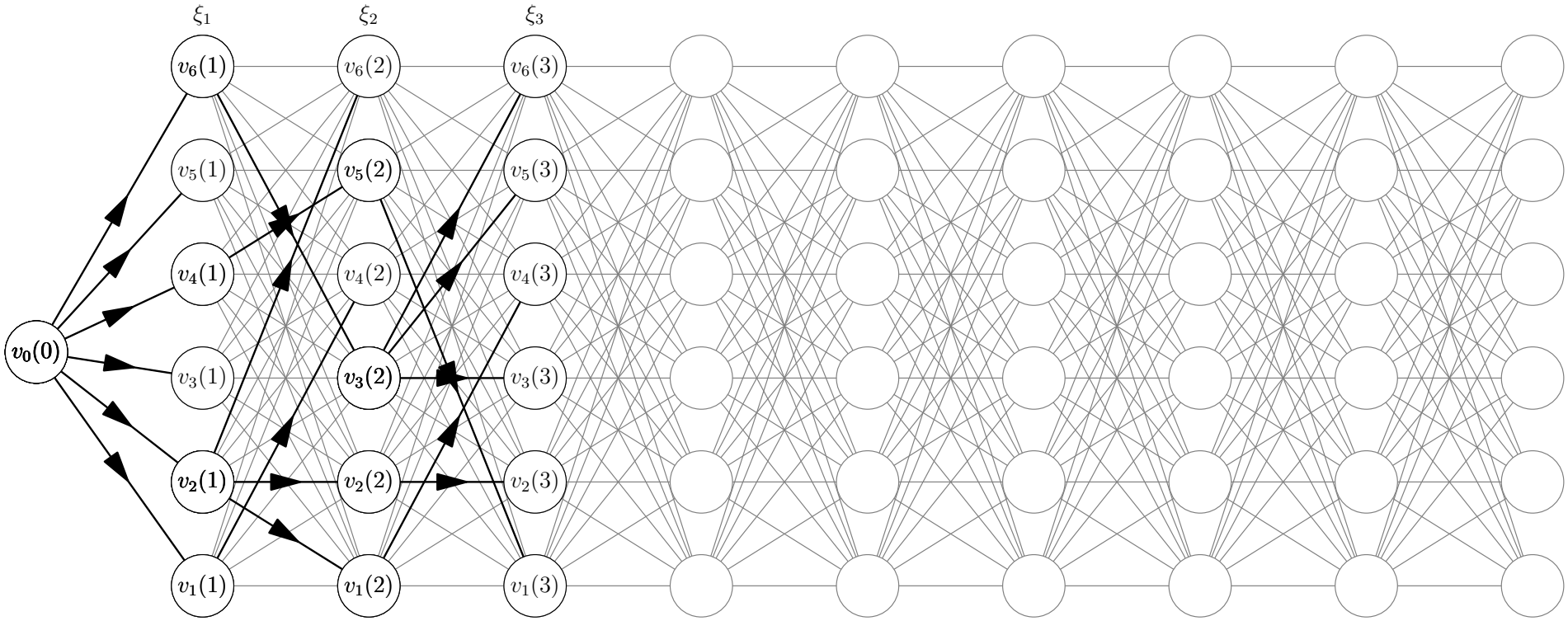
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



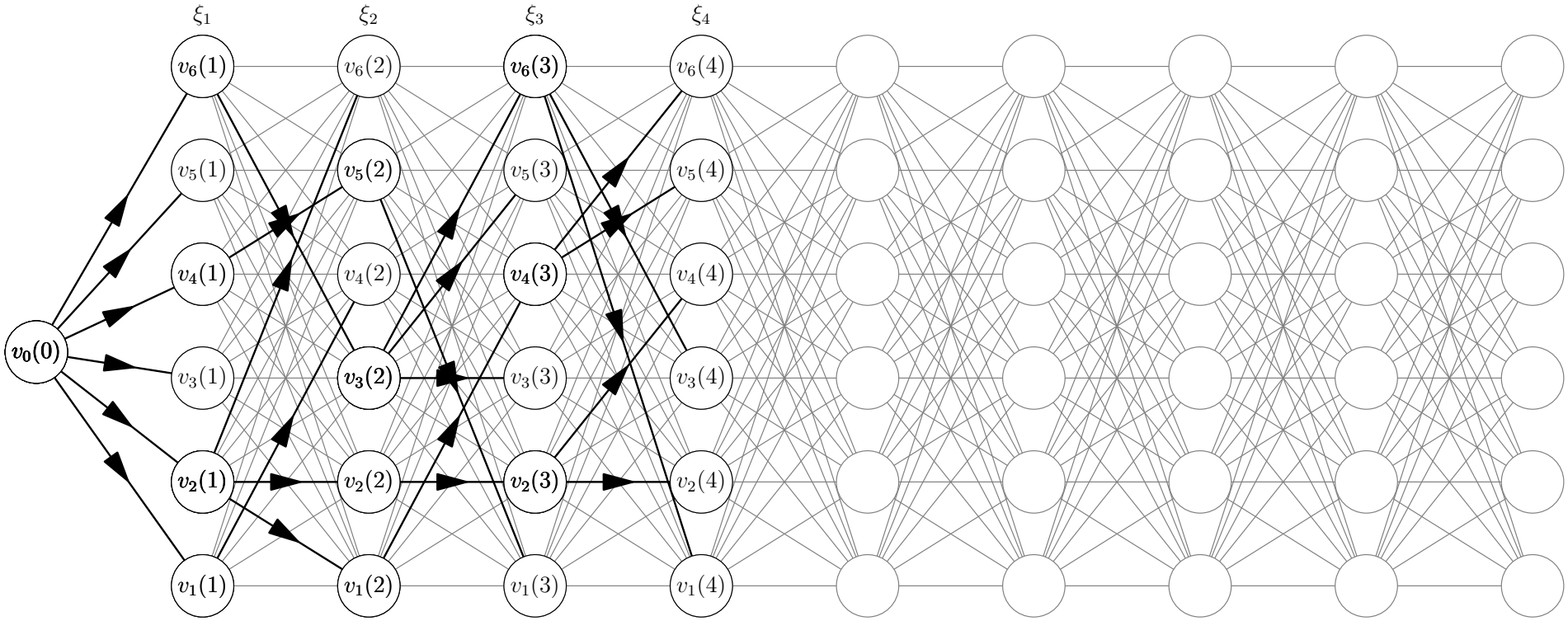
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



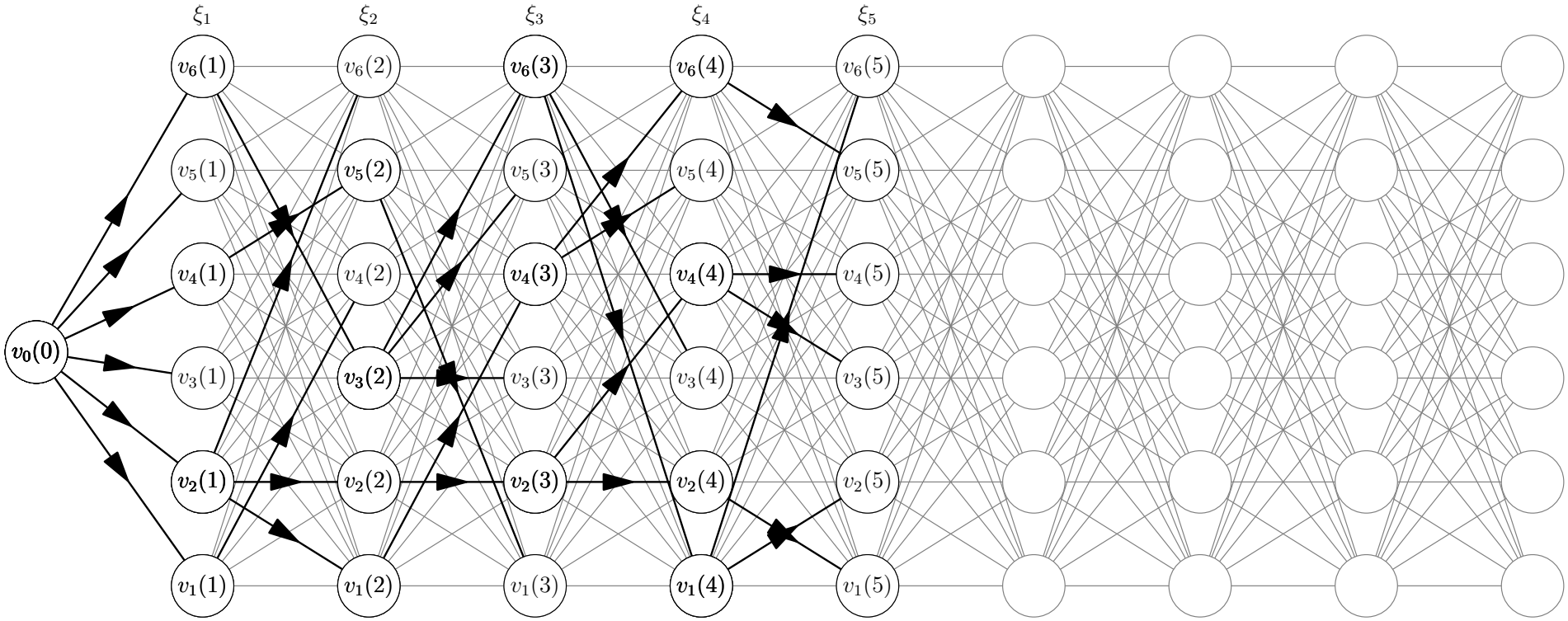
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



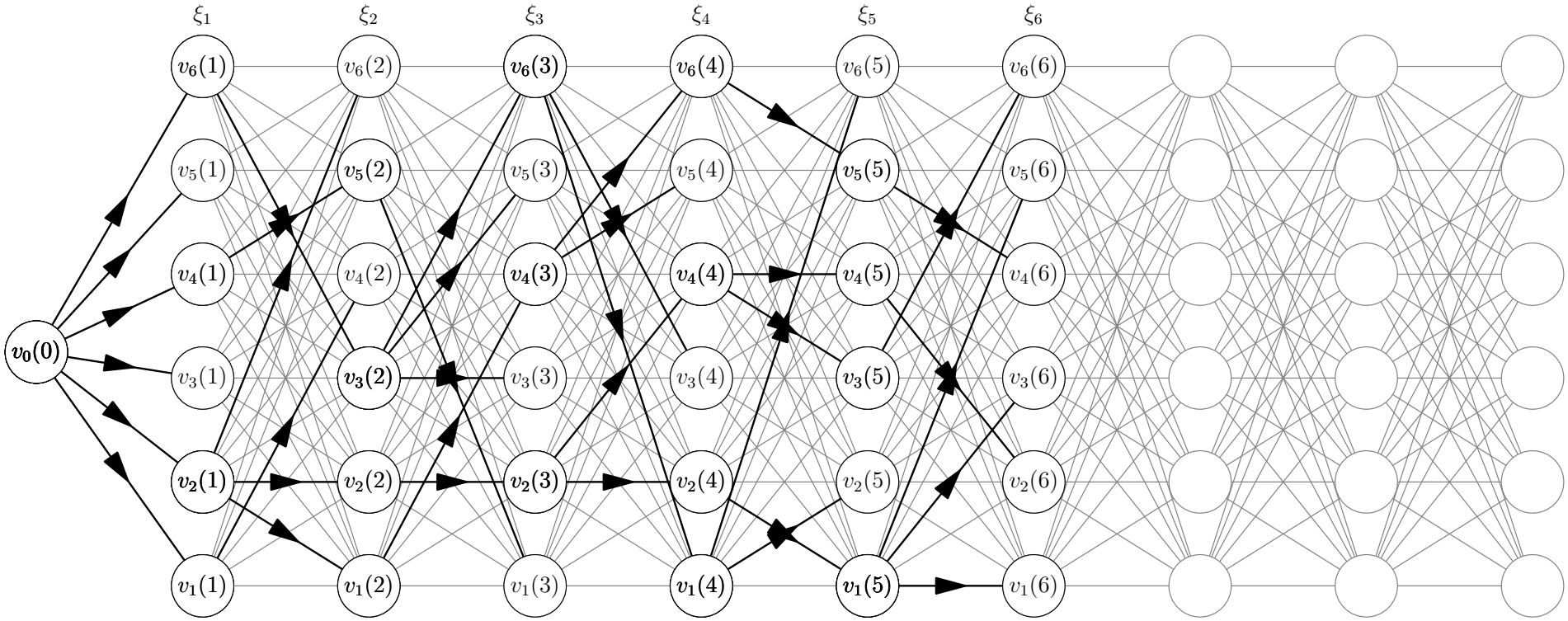
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



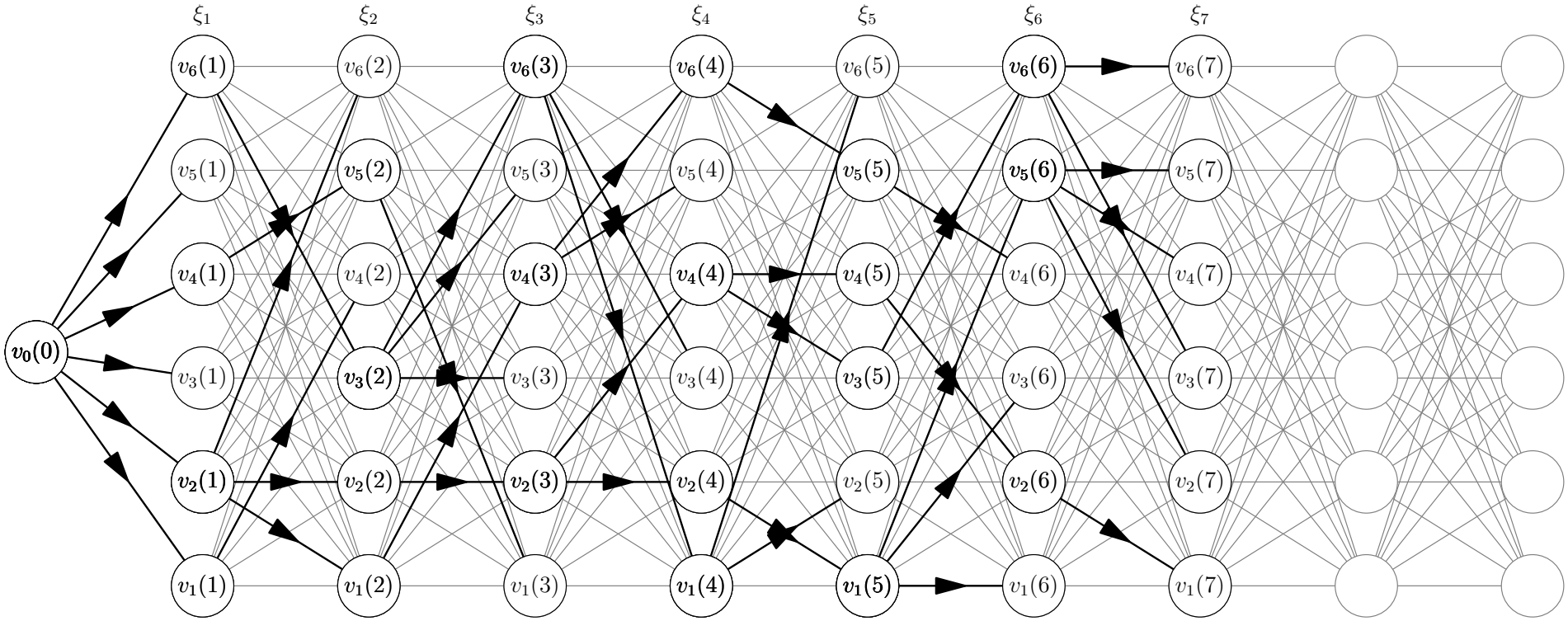
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



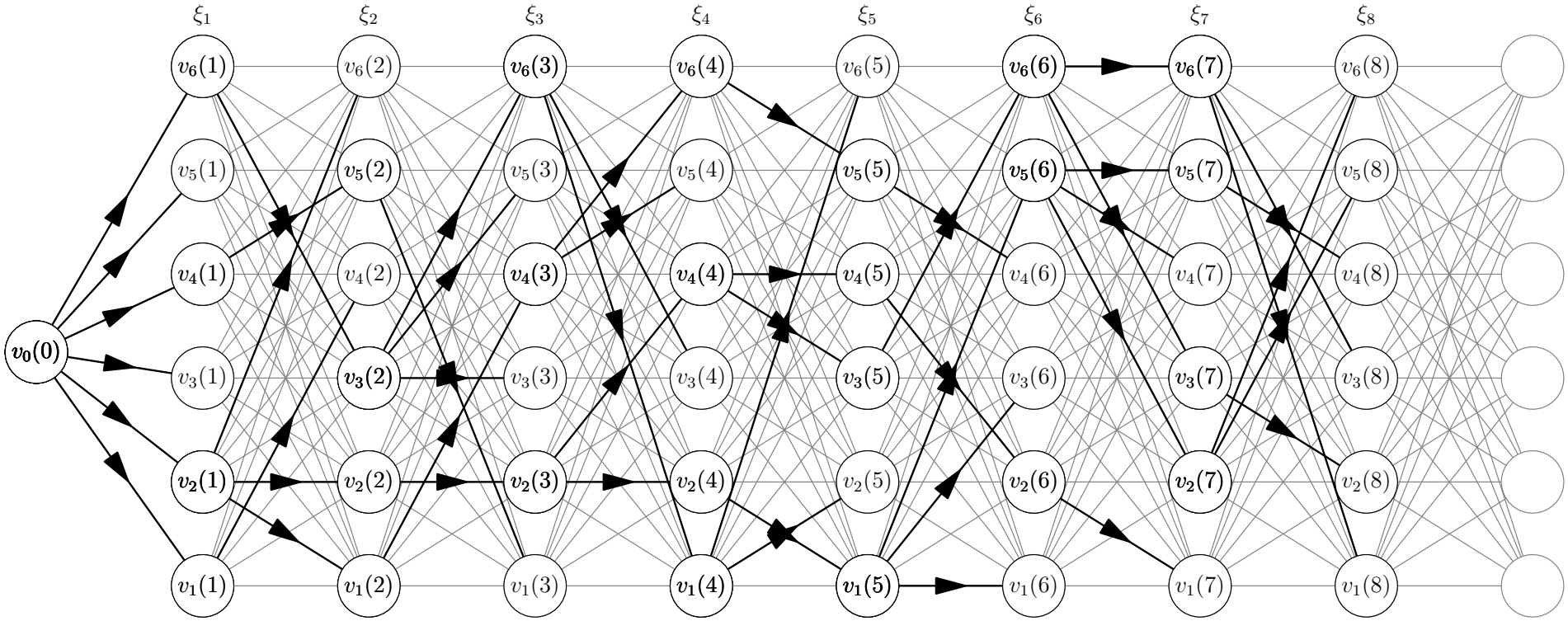
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



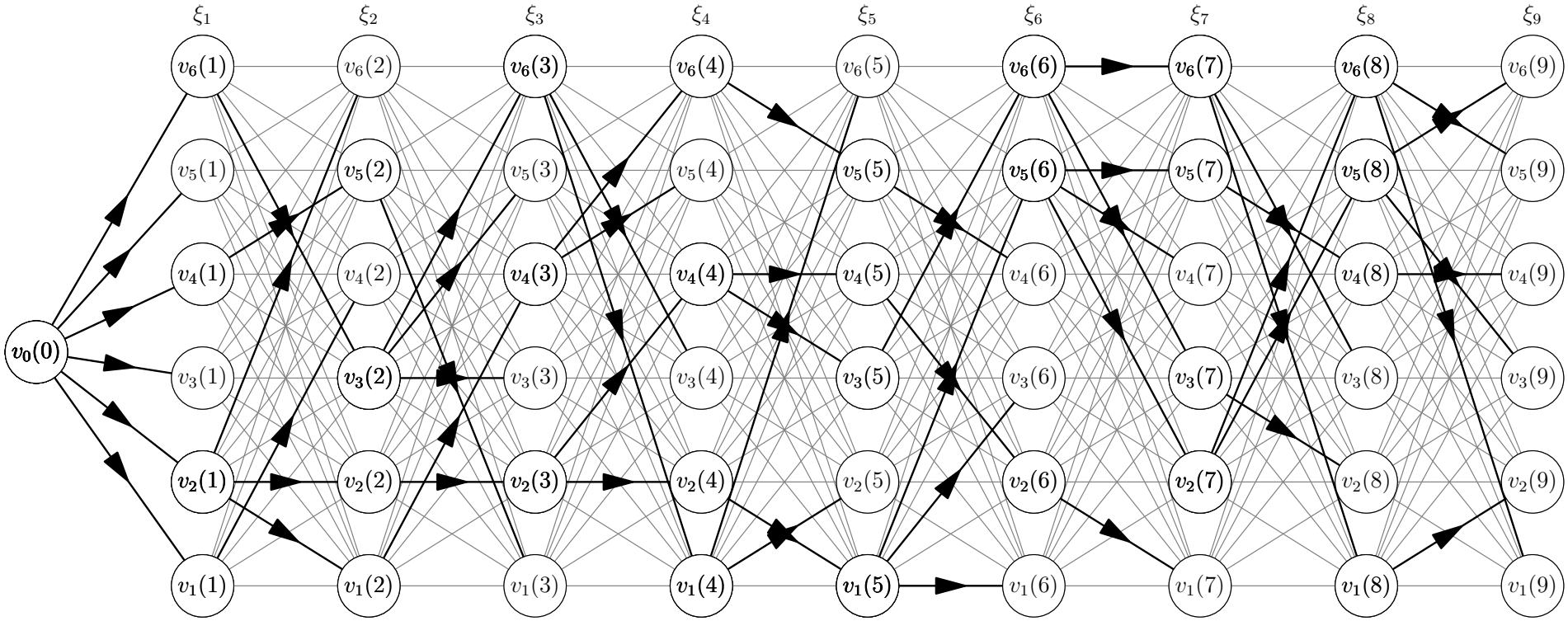
# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



# Viterbi Schematic

$$v_i(t) = e_i(\xi_t)\ \max_j v_j(t-1)\, a_{ji}, \quad v_0(0) = 1$$



# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400}$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1)$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$

# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400}$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1)$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$

# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400} = 0$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1)$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$

# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400} = 0$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1)$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$

# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400} = 0$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1)$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$

# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400} = 0$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1) = -921.03$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$

# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400} = 0$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1) = -921.03$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$

# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400} = 0$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1) = -921.03$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$

# Using Log-Probabilities

- Numerically products of probabilities underflow: if $P = \prod_{i=1}^{N} p_i$ with $p_i = 0.1$ and $n = 400$ then $P = (0.1)^{400} = 0$

- Now if $P = \prod_{i=1}^{N} p_i$ then $\log(P) = \sum_{i=1}^{N} \log(p_i)$ so with $p_i = 0.1$ and $n = 400$ then $\log(P) = 400 \times \log(0.1) = -921.03$

- If $S = p_1 + p_2$ with $p_1 > p_2$ then

$$\log(S) = \log\left(\mathrm{e}^{l_1} + \mathrm{e}^{l_2}\right) = l_1 + \log\left(1 + \mathrm{e}^{l_2 - l_1}\right)$$

  where $l_1 = \log(p_1)$ and $l_2 = \log(p_2)$

- Note that in many programming languages `log1p(x)` computes $\log(1 + x)$ see scipy.special.logsumexp

# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$

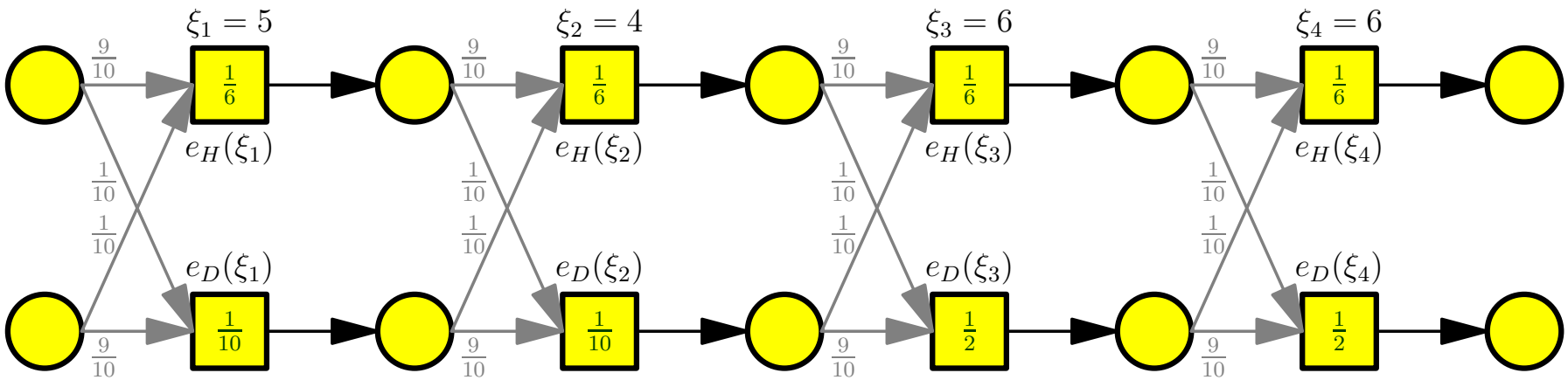
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



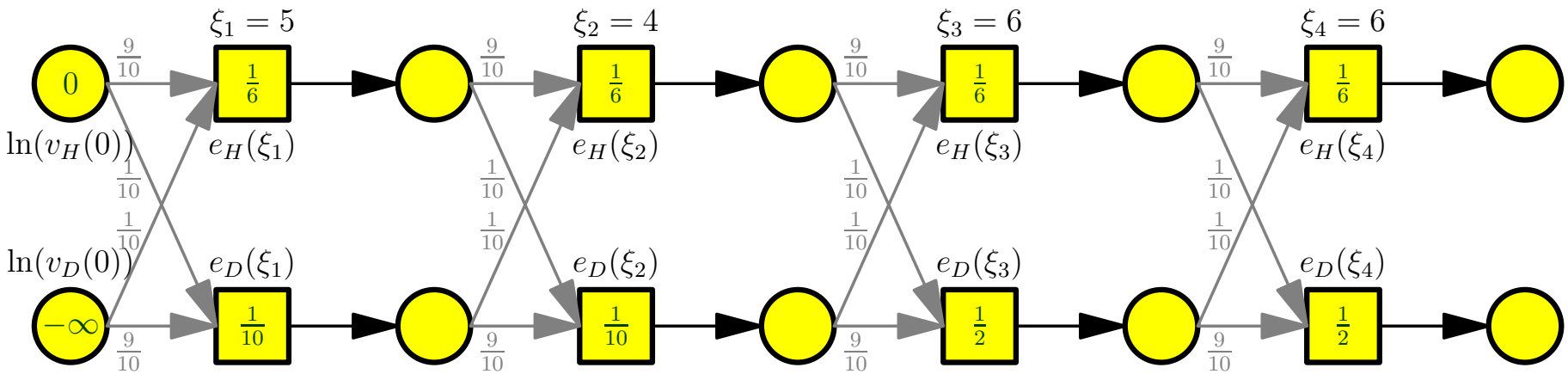
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



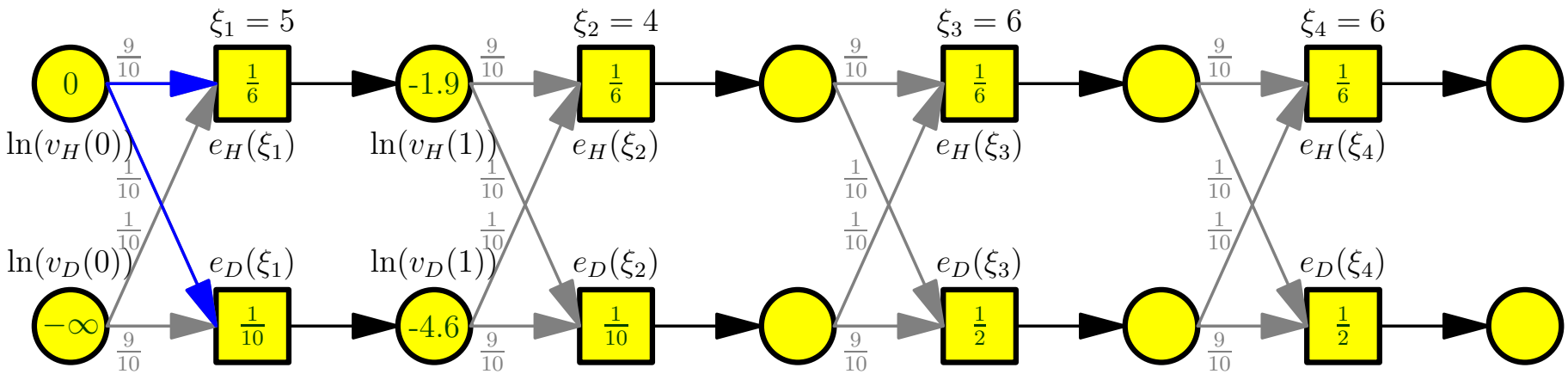
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



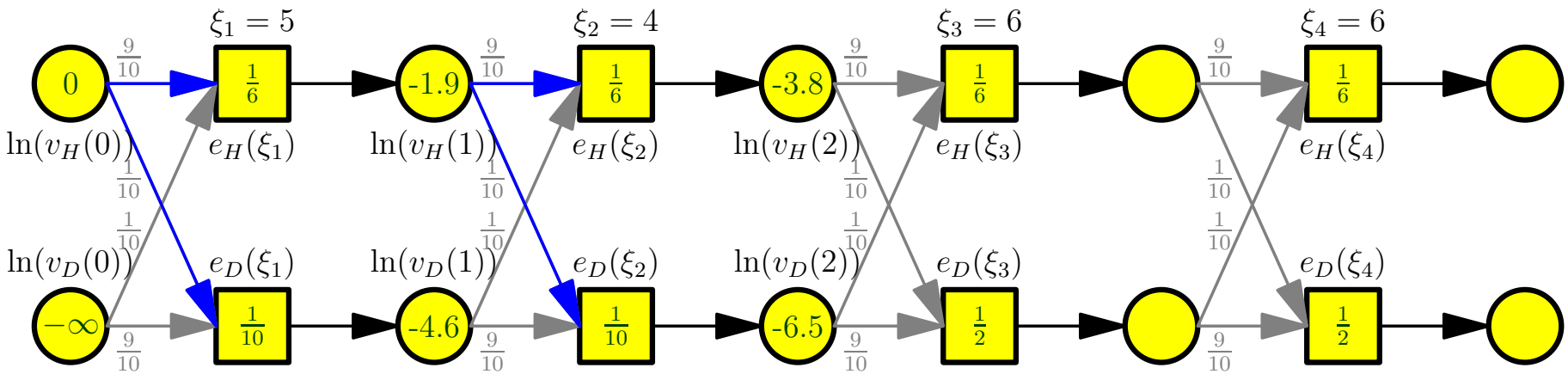
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



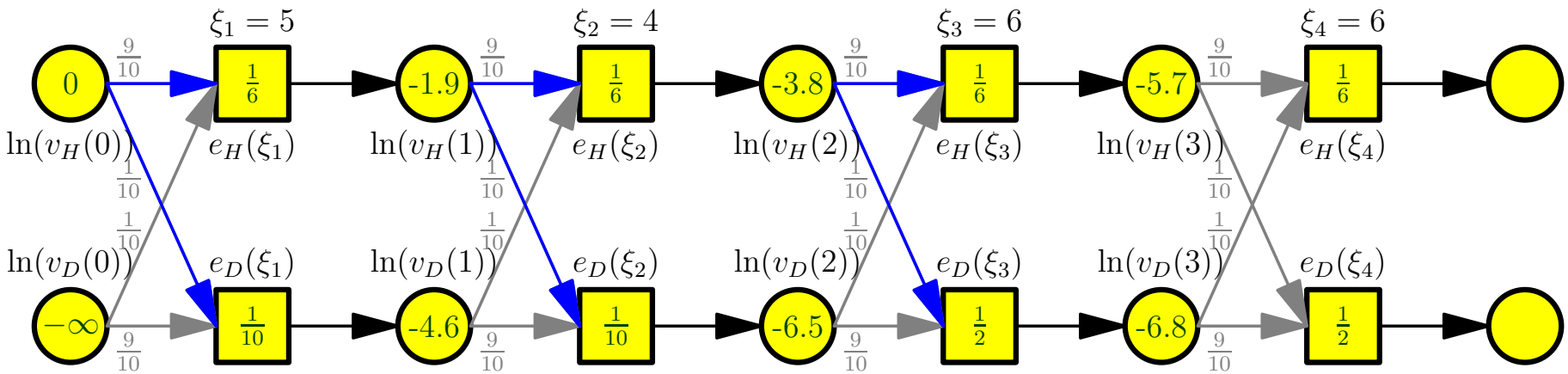
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



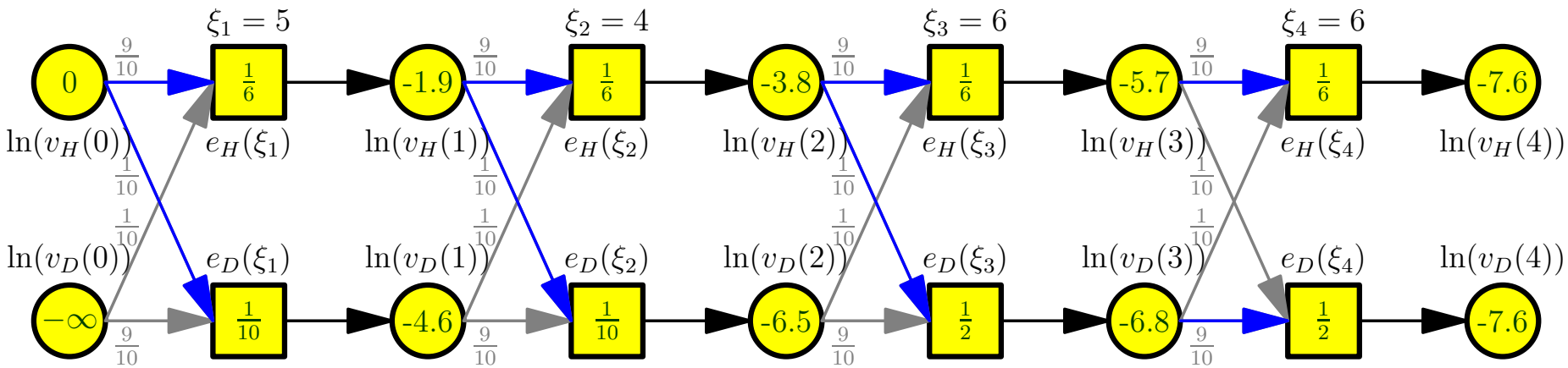
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



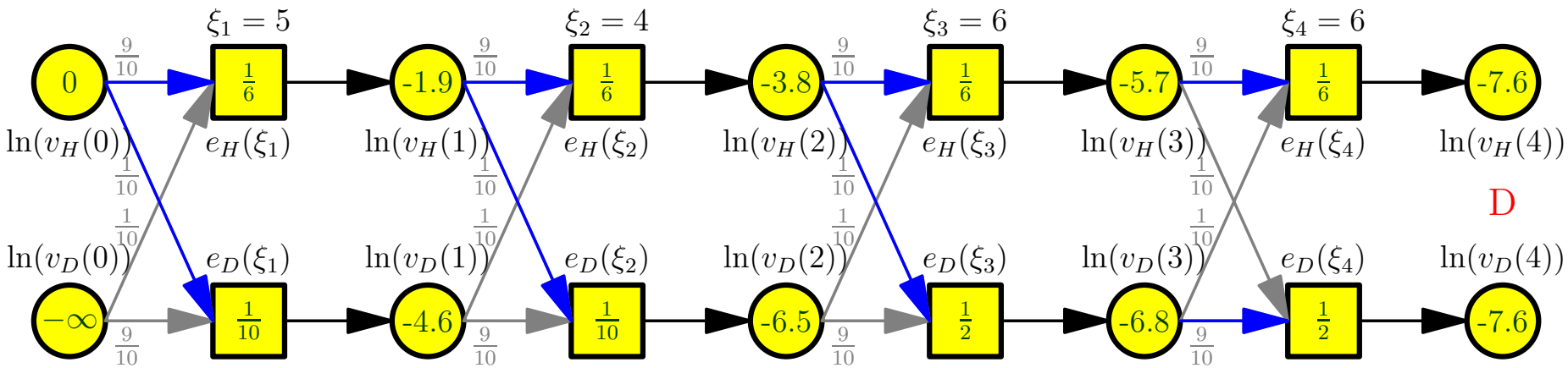
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



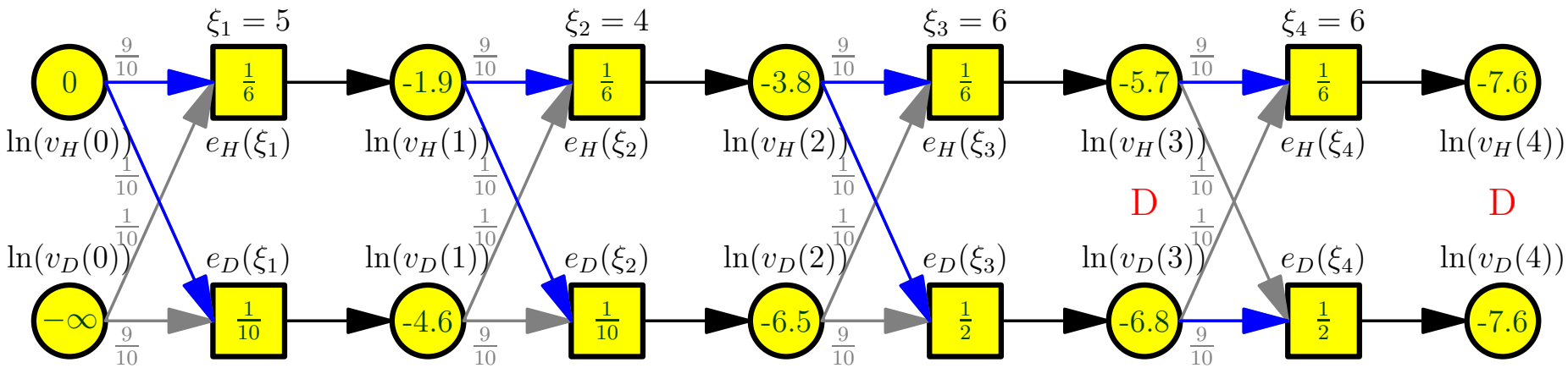
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



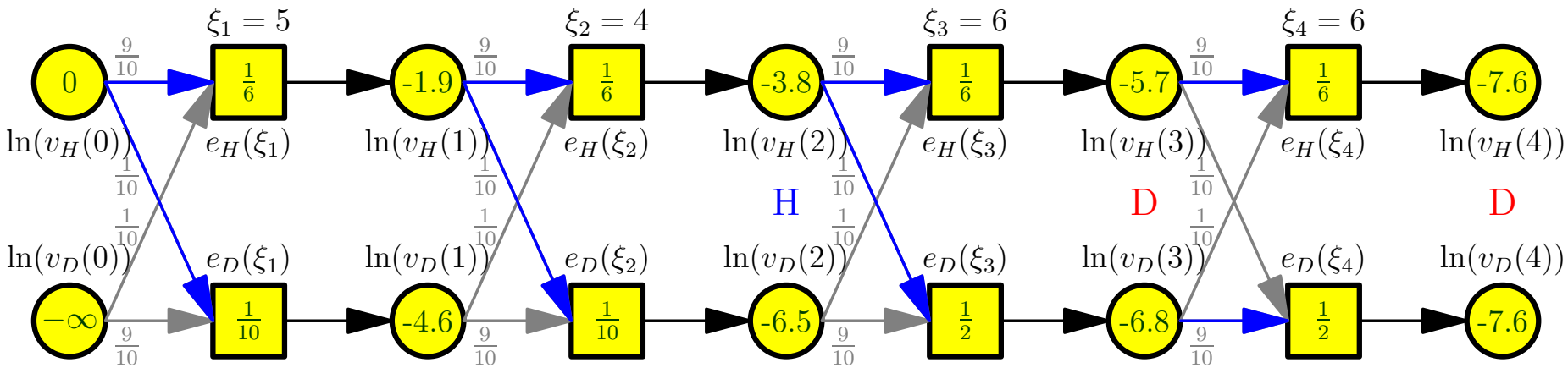
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



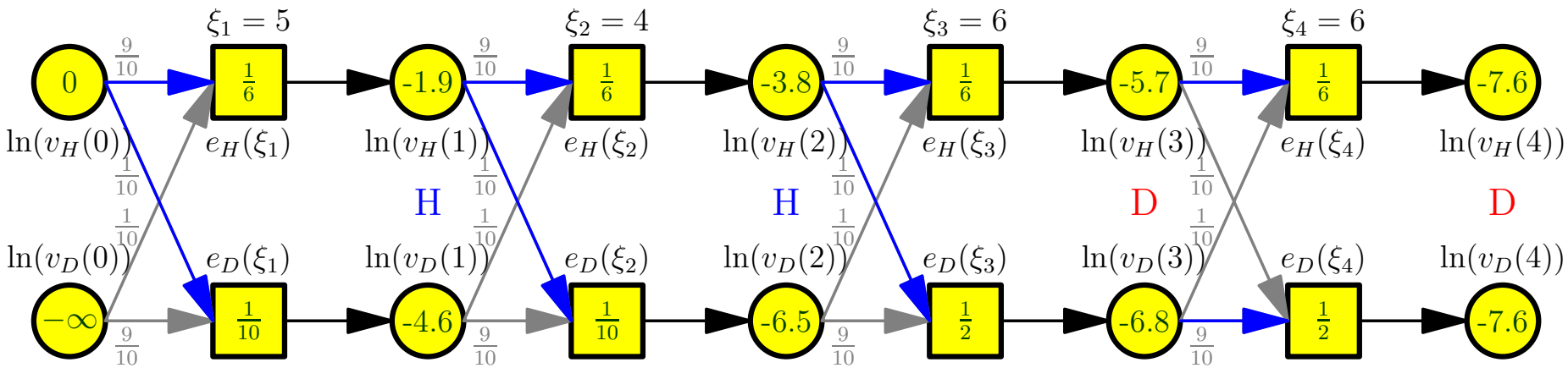
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



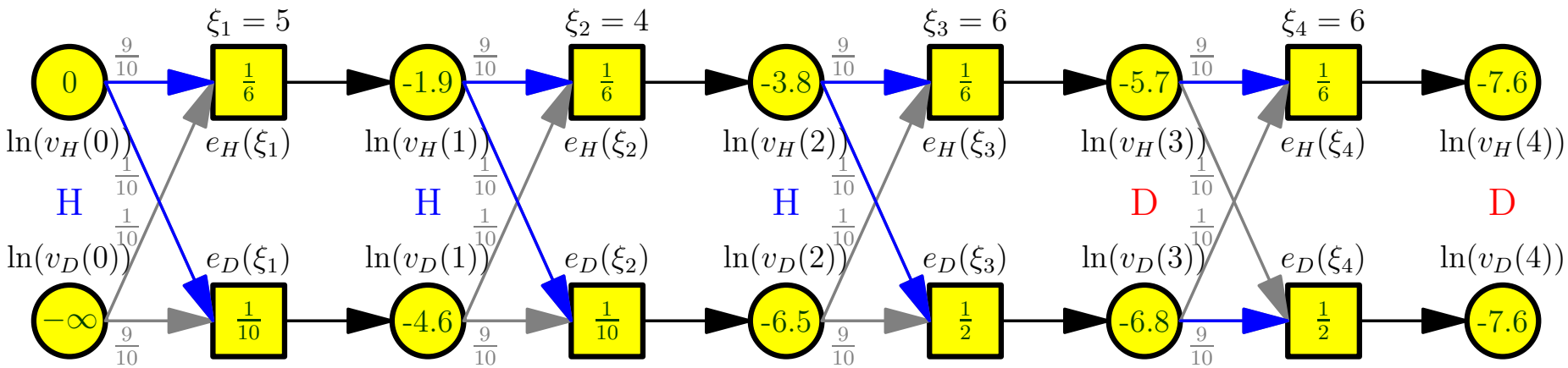
# Viterbi In Action

- Recall

$$v_i(t) = e_i(\xi_t) \max_j v_j(t-1)\, a_{ji}$$

- Or

$$\log(v_i(t)) = \log(e_i(\xi_t)) + \max_j \log(v_j(t-1)) + \log(a_{ji})$$



# Back to Our Dishonest Casino

- One application of the Viterbi algorithm is to make an informed guess whether the dice is fair or loaded in our dishonest casino
- Show the dice rolls, true states and Viterbi predictions

```
5453525456666664365666635661416626365666621166211311155566351166565663466653642535666662541345464155
HHHHHHHHHDDDDDDDDDDDDDDDDDHHDDDDDDDDDDDDDDHHHHHHHHHHHHHHDDDDDHDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHH
HHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHHHH
```

- Note that the transition occur at boundary between 6 and not 6
- The most likely path is not always very accurate. We often do better to ask what is the probability of being in a certain state

# Back to Our Dishonest Casino

- One application of the Viterbi algorithm is to make an informed guess whether the dice is fair or loaded in our dishonest casino
- Show the dice rolls, true states and Viterbi predictions

```
5453525456666664365666635661416626365666621166211311155566351166565663466653642535666662541345464155
HHHHHHHHHDDDDDDDDDDDDDDDDDHHDDDDDDDDDDDDDDHHHHHHHHHHHHHHDDDDDDHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHH
HHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHHHH
```

- Note that the transition occur at boundary between 6 and not 6
- The most likely path is not always very accurate. We often do better to ask what is the probability of being in a certain state

# Back to Our Dishonest Casino

- One application of the Viterbi algorithm is to make an informed guess whether the dice is fair or loaded in our dishonest casino

- Show the dice rolls, true states and Viterbi predictions

```
5453525456666664365666635661416626365666621166211311155566351166565663466653642535666662541345464155
HHHHHHHHHDDDDDDDDDDDDDDDDDHHDDDDDDDDDDDDDDHHHHHHHHHHHHHHDDDDDHDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHH
HHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHHHH
```

- Note that the transition occur at boundary between 6 and not 6

- The most likely path is not always very accurate. We often do better to ask what is the probability of being in a certain state

# Back to Our Dishonest Casino

- One application of the Viterbi algorithm is to make an informed guess whether the dice is fair or loaded in our dishonest casino
- Show the dice rolls, true states and Viterbi predictions

```
5453525456666664365666635661416626365666621166211311155566351166565663466653642535666662541345464155
HHHHHHHHHDDDDDDDDDDDDDDDDDHHDDDDDDDDDDDDDDHHHHHHHHHHHHHHDDDDDHDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHH
HHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHHHH
```

- Note that the transition occur at boundary between 6 and not 6
- The most likely path is not always very accurate. We often do better to ask what is the probability of being in a certain state

# Back to Our Dishonest Casino

- One application of the Viterbi algorithm is to make an informed guess whether the dice is fair or loaded in our dishonest casino

- Show the dice rolls, true states and Viterbi predictions

```
5453525456666664365666635661416626365666621166211311155566351166565663466653642535666662541345464155
HHHHHHHHHDDDDDDDDDDDDDDDDDHHDDDDDDDDDDDDDDHHHHHHHHHHHHHHDDDDDHDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHH
HHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDHHHHHHHHHHHHH
```

- Note that the transition occur at boundary between 6 and not 6

- The most likely path is not always very accurate. We often do better to ask what is the probability of being in a certain state

# Probabilities of a Sequence

- The joint probability of the HMM moving through a set of states $\boldsymbol{q}$ and emitting symbols $\boldsymbol{\xi}$ is given by

$$\mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q} | \boldsymbol{\theta}\right) = \prod_{t=1}^{T} a_{q_{t-1}\, q_t}\, e_{q_t}(\xi_t)$$

- Because the states are hidden from us we have to average them out

$$\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right) = \sum_{\boldsymbol{q} \in \mathcal{Q}^T} \mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q} | \boldsymbol{\theta}\right)$$

- $\mathcal{Q}^T$ signifies the set of all possible paths

# Probabilities of a Sequence

- The joint probability of the HMM moving through a set of states $\boldsymbol{q}$ and emitting symbols $\boldsymbol{\xi}$ is given by

$$\mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q} | \boldsymbol{\theta}\right) = \prod_{t=1}^{T} a_{q_{t-1}\, q_t}\, e_{q_t}(\xi_t)$$

- Because the states are hidden from us we have to average them out

$$\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right) = \sum_{\boldsymbol{q} \in \mathcal{Q}^T} \mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q} | \boldsymbol{\theta}\right)$$

- $\mathcal{Q}^T$ signifies the set of all possible paths

# Probabilities of a Sequence

- The joint probability of the HMM moving through a set of states $\boldsymbol{q}$ and emitting symbols $\boldsymbol{\xi}$ is given by

$$\mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q} | \boldsymbol{\theta}\right) = \prod_{t=1}^{T} a_{q_{t-1}\, q_t}\, e_{q_t}(\xi_t)$$

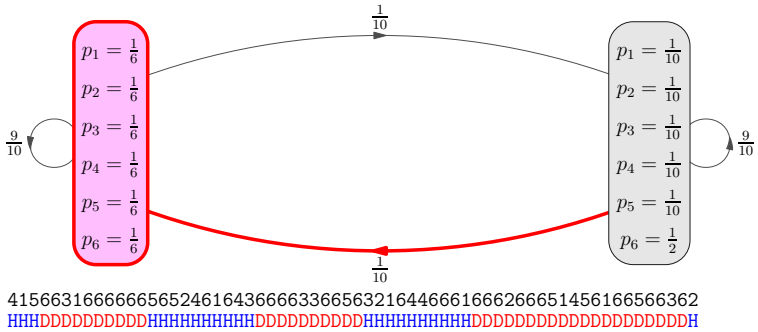
- Because the states are hidden from us we have to average them out

$$\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right) = \sum_{\boldsymbol{q} \in \mathcal{Q}^T} \mathbb{P}\left(\boldsymbol{\xi}, \boldsymbol{q} | \boldsymbol{\theta}\right)$$

- $\mathcal{Q}^T$ signifies the set of all possible paths

# Outline

1. Probabilistic Models
2. Hidden Markov Models
3. **Summing Over Paths**



# Summing Over Paths

- The sum of all paths involves $|\mathcal{Q}|^T$ terms which rapidly becomes intractable
- However, once again dynamic programming comes to the rescue
- Note, that previously we had used dynamic programming to find the optimal path—now we are using it to sum over all paths
- We define the forward and backward variables as

$$\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$$
$$\beta_t(i) = \mathbb{P}\left(\xi_{t+1}, \cdots, \xi_T | q_t = i, \boldsymbol{\theta}\right)$$

# Summing Over Paths

- The sum of all paths involves $|\mathcal{Q}|^T$ terms which rapidly becomes intractable
- However, once again dynamic programming comes to the rescue
- Note, that previously we had used dynamic programming to find the optimal path—now we are using it to sum over all paths
- We define the forward and backward variables as

$$\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$$
$$\beta_t(i) = \mathbb{P}\left(\xi_{t+1}, \cdots, \xi_T | q_t = i, \boldsymbol{\theta}\right)$$

# Summing Over Paths

- The sum of all paths involves $|\mathcal{Q}|^T$ terms which rapidly becomes intractable
- However, once again dynamic programming comes to the rescue
- Note, that previously we had used dynamic programming to find the optimal path—now we are using it to sum over all paths
- We define the forward and backward variables as

$$\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$$
$$\beta_t(i) = \mathbb{P}\left(\xi_{t+1}, \cdots, \xi_T | q_t = i, \boldsymbol{\theta}\right)$$

# Summing Over Paths

- The sum of all paths involves $|\mathcal{Q}|^T$ terms which rapidly becomes intractable
- However, once again dynamic programming comes to the rescue
- Note, that previously we had used dynamic programming to find the optimal path—now we are using it to sum over all paths
- We define the forward and backward variables as

$$\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$$
$$\beta_t(i) = \mathbb{P}\left(\xi_{t+1}, \cdots, \xi_T | q_t = i, \boldsymbol{\theta}\right)$$

# The Forward Variable

- The variable $\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$ is computed iteratively

$$\alpha_t(i) = \sum_{j \in \mathcal{Q}} \mathbb{P}\left(\xi_t, q_t = i | q_{t-1} = j, \boldsymbol{\theta}\right) \mathbb{P}\left(\xi_1, \cdots, \xi_{t-1}, q_{t-1} = j | \boldsymbol{\theta}\right)$$

$$= e_i(\xi_t) \sum_{j \in \mathcal{Q}} a_{ji}\, \alpha_{t-1}(j)$$

- With initial condition $\alpha_0(0) = 1$ (where state 0 is taken to be initial state) and $\alpha_i(0) = 0$ for $i \neq 0$

- Since $\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$

$$\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right) = \sum_{i \in \mathcal{Q}} \alpha_T(i)$$

# The Forward Variable

- The variable $\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$ is computed iteratively

$$\alpha_t(i) = \sum_{j \in \mathcal{Q}} \mathbb{P}\left(\xi_t, q_t = i | q_{t-1} = j, \boldsymbol{\theta}\right) \mathbb{P}\left(\xi_1, \cdots, \xi_{t-1}, q_{t-1} = j | \boldsymbol{\theta}\right)$$

$$= e_i(\xi_t) \sum_{j \in \mathcal{Q}} a_{ji}\, \alpha_{t-1}(j)$$

- With initial condition $\alpha_0(0) = 1$ (where state 0 is taken to be initial state) and $\alpha_i(0) = 0$ for $i \neq 0$

- Since $\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$

$$\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right) = \sum_{i \in \mathcal{Q}} \alpha_T(i)$$

# The Forward Variable

- The variable $\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$ is computed iteratively

$$\alpha_t(i) = \sum_{j \in \mathcal{Q}} \mathbb{P}\left(\xi_t, q_t = i | q_{t-1} = j, \boldsymbol{\theta}\right) \mathbb{P}\left(\xi_1, \cdots, \xi_{t-1}, q_{t-1} = j | \boldsymbol{\theta}\right)$$

$$= e_i(\xi_t) \sum_{j \in \mathcal{Q}} a_{ji}\, \alpha_{t-1}(j)$$

- With initial condition $\alpha_0(0) = 1$ (where state 0 is taken to be initial state) and $\alpha_i(0) = 0$ for $i \neq 0$

- Since $\alpha_t(i) = \mathbb{P}\left(\xi_1, \cdots, \xi_t, q_t = i | \boldsymbol{\theta}\right)$

$$\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right) = \sum_{i \in \mathcal{Q}} \alpha_T(i)$$

# The Backward Variable

- Similarly $\beta_t(i) = \mathbb{P}\left(\xi_{t+1}, \cdots, \xi_T | q_t = i, \boldsymbol{\theta}\right)$ is computed with the backward algorithm

$$\begin{aligned}\beta_{t-1}(i) &= \sum_{j \in \mathcal{Q}} \mathbb{P}\left(\xi_t, q_t = j | q_{t-1} = i\right) \mathbb{P}\left(\xi_{t+1}, \cdots, \xi_T | q_t = j\right) \\ &= \sum_{j \in \mathcal{Q}} a_{ij}\, e_j(\xi_t)\, \beta_t(j)\end{aligned}$$

- with $\beta_T(i) = 1$

- Some care is needed to make sure these quantities don't underflow (they are product of probabilities)—usually we would work with the logarithm of these quantities

# The Backward Variable

- Similarly $\beta_t(i) = \mathbb{P}\left(\xi_{t+1}, \cdots, \xi_T | q_t = i, \boldsymbol{\theta}\right)$ is computed with the backward algorithm

$$\begin{aligned}\beta_{t-1}(i) &= \sum_{j \in \mathcal{Q}} \mathbb{P}\left(\xi_t, q_t = j | q_{t-1} = i\right) \mathbb{P}\left(\xi_{t+1}, \cdots, \xi_T | q_t = j\right) \\ &= \sum_{j \in \mathcal{Q}} a_{ij}\, e_j(\xi_t)\, \beta_t(j)\end{aligned}$$

- with $\beta_T(i) = 1$

- Some care is needed to make sure these quantities don't underflow (they are product of probabilities)—usually we would work with the logarithm of these quantities

# Using Forward and Backward Variables

- With both forward and backward variables we can compute more complex quantities

- For example, the probability of being in a particular state given a sequence $\boldsymbol{\xi}$ is

$$\mathbb{P}\left(q_t = i | \boldsymbol{\xi}, \boldsymbol{\theta}\right) = \frac{\mathbb{P}\left(\boldsymbol{\xi}, q_t = i\right)}{\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right)} = \frac{\alpha_t(i)\,\beta_t(i)}{\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right)}$$

- We could use this to analyse our dishonest casino

# Using Forward and Backward Variables

- With both forward and backward variables we can compute more complex quantities

- For example, the probability of being in a particular state given a sequence $\boldsymbol{\xi}$ is

$$\mathbb{P}\left(q_t = i | \boldsymbol{\xi}, \boldsymbol{\theta}\right) = \frac{\mathbb{P}\left(\boldsymbol{\xi}, q_t = i\right)}{\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right)} = \frac{\alpha_t(i)\,\beta_t(i)}{\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right)}$$

- We could use this to analyse our dishonest casino

# Using Forward and Backward Variables

- With both forward and backward variables we can compute more complex quantities

- For example, the probability of being in a particular state given a sequence $\boldsymbol{\xi}$ is

$$\mathbb{P}\left(q_t = i | \boldsymbol{\xi}, \boldsymbol{\theta}\right) = \frac{\mathbb{P}\left(\boldsymbol{\xi}, q_t = i\right)}{\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right)} = \frac{\alpha_t(i)\,\beta_t(i)}{\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right)}$$

- We could use this to analyse our dishonest casino

# Using Forward and Backward Variables

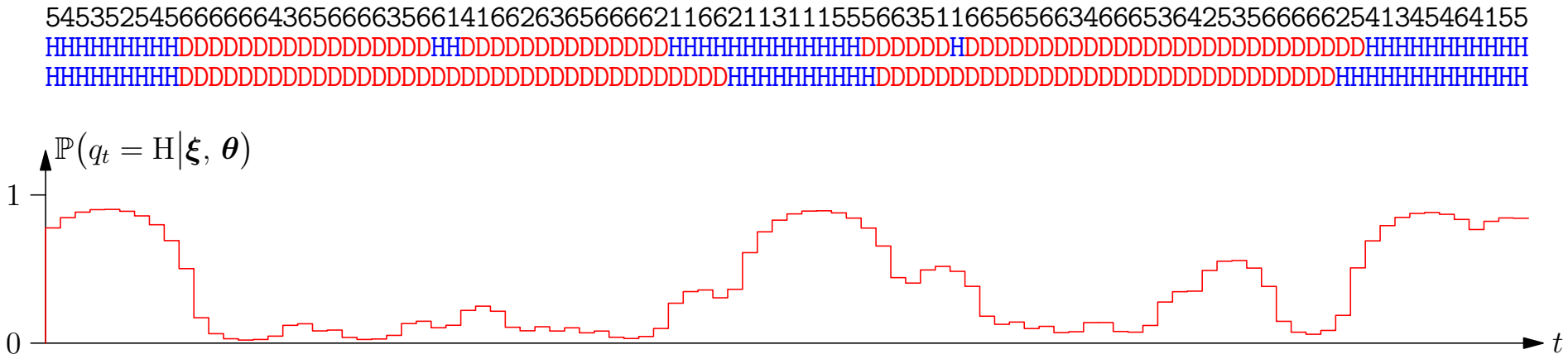
- With both forward and backward variables we can compute more complex quantities

- For example, the probability of being in a particular state given a sequence $\boldsymbol{\xi}$ is

$$\mathbb{P}\left(q_t = i | \boldsymbol{\xi}, \boldsymbol{\theta}\right) = \frac{\mathbb{P}\left(\boldsymbol{\xi}, q_t = i\right)}{\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right)} = \frac{\alpha_t(i)\,\beta_t(i)}{\mathbb{P}\left(\boldsymbol{\xi} | \boldsymbol{\theta}\right)}$$

- We could use this to analyse our dishonest casino

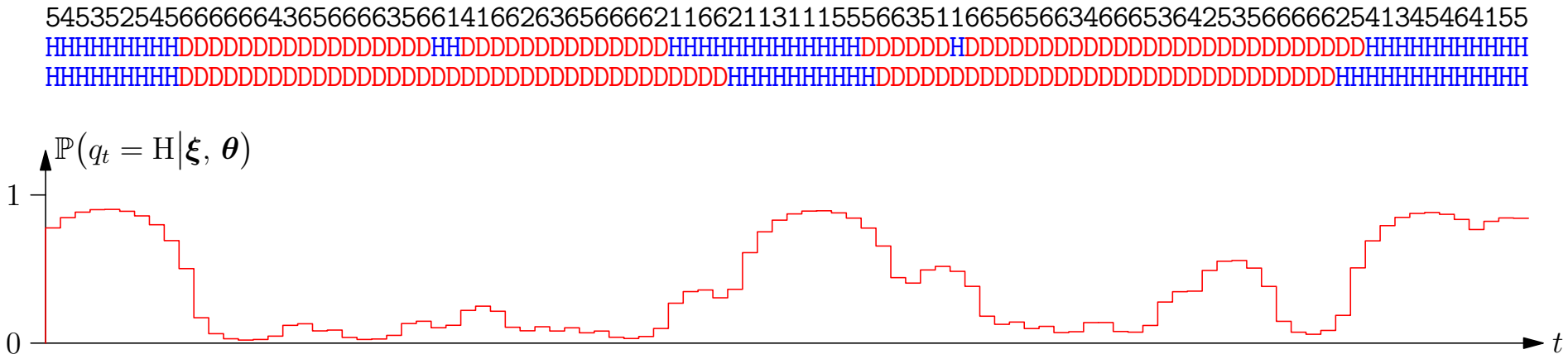
# Posterior Probability




- We suppose we can't detect the change of dice, but we can observe the sequence $\boldsymbol{\xi}$ of rolls

- $\mathbb{P}\left(q_t = H|\boldsymbol{\xi}, \boldsymbol{\theta}\right)$ is then our inferred probability that the dice is honest

# Posterior Probability




- We suppose we can't detect the change of dice, but we can observe the sequence $\boldsymbol{\xi}$ of rolls

- $\mathbb{P}\left(q_t = H|\boldsymbol{\xi}, \boldsymbol{\theta}\right)$ is then our inferred probability that the dice is honest

# Dynamic Programming for Averaging

- Dynamic programming is often used to find optima
- E.g. the most likely path as computed by Viterbi
- However, with a minor modification the same strategy can be used for averaging over all paths
- Both strategies are commonly referred to as dynamic programming

# Dynamic Programming for Averaging

- Dynamic programming is often used to find optima
- E.g. the most likely path as computed by Viterbi
- However, with a minor modification the same strategy can be used for averaging over all paths
- Both strategies are commonly referred to as dynamic programming

# Dynamic Programming for Averaging

- Dynamic programming is often used to find optima
- E.g. the most likely path as computed by Viterbi
- However, with a minor modification the same strategy can be used for averaging over all paths
- Both strategies are commonly referred to as dynamic programming

# Dynamic Programming for Averaging

- Dynamic programming is often used to find optima
- E.g. the most likely path as computed by Viterbi
- However, with a minor modification the same strategy can be used for averaging over all paths
- Both strategies are commonly referred to as dynamic programming

# Using HMMs in Biology

- In biological sequence analysis we can use an HMM, for example, to tell if a protein is a kinase protein
- If we associate different states to either an alpha helix, a beta sheet or a coil then we can calculate the probability of an amino acid being one of the above
- Of course to do this we need that our HMM has the right structure
- If we knew a lot about biology we might be able to carefully design the HMM
- In practice we very rarely do and we have to train our HMM on data—we discuss this in the next lecture

# Using HMMs in Biology

- In biological sequence analysis we can use an HMM, for example, to tell if a protein is a kinase protein
- If we associate different states to either an alpha helix, a beta sheet or a coil then we can calculate the probability of an amino acid being one of the above
- Of course to do this we need that our HMM has the right structure
- If we knew a lot about biology we might be able to carefully design the HMM
- In practice we very rarely do and we have to train our HMM on data—we discuss this in the next lecture

# Using HMMs in Biology

- In biological sequence analysis we can use an HMM, for example, to tell if a protein is a kinase protein
- If we associate different states to either an alpha helix, a beta sheet or a coil then we can calculate the probability of an amino acid being one of the above
- Of course to do this we need that our HMM has the right structure
- If we knew a lot about biology we might be able to carefully design the HMM
- In practice we very rarely do and we have to train our HMM on data—we discuss this in the next lecture

# Using HMMs in Biology

- In biological sequence analysis we can use an HMM, for example, to tell if a protein is a kinase protein
- If we associate different states to either an alpha helix, a beta sheet or a coil then we can calculate the probability of an amino acid being one of the above
- Of course to do this we need that our HMM has the right structure
- If we knew a lot about biology we might be able to carefully design the HMM
- In practice we very rarely do and we have to train our HMM on data—we discuss this in the next lecture

# Using HMMs in Biology

- In biological sequence analysis we can use an HMM, for example, to tell if a protein is a kinase protein
- If we associate different states to either an alpha helix, a beta sheet or a coil then we can calculate the probability of an amino acid being one of the above
- Of course to do this we need that our HMM has the right structure
- If we knew a lot about biology we might be able to carefully design the HMM
- In practice we very rarely do and we have to train our HMM on data—we discuss this in the next lecture

# Using HMMs in Biology

- In biological sequence analysis we can use an HMM, for example, to tell if a protein is a kinase protein
- If we associate different states to either an alpha helix, a beta sheet or a coil then we can calculate the probability of an amino acid being one of the above
- Of course to do this we need that our HMM has the right structure
- If we knew a lot about biology we might be able to carefully design the HMM
- In practice we very rarely do and we have to train our HMM on data—we discuss this in the next lecture

# Summary

- HMMs are a very important structure in biological sequence analysis
- They are probabilistic Finite State Automata
- They are a generative model for sequence
- Of course, they are massively simplified generative models
- We can use dynamic programming to sum over all the unseen state

# Summary

- HMMs are a very important structure in biological sequence analysis
- They are probabilistic Finite State Automata
- They are a generative model for sequence
- Of course, they are massively simplified generative models
- We can use dynamic programming to sum over all the unseen state

# Summary

- HMMs are a very important structure in biological sequence analysis
- They are probabilistic Finite State Automata
- They are a generative model for sequence
- Of course, they are massively simplified generative models
- We can use dynamic programming to sum over all the unseen state

# Summary

- HMMs are a very important structure in biological sequence analysis
- They are probabilistic Finite State Automata
- They are a generative model for sequence
- Of course, they are massively simplified generative models
- We can use dynamic programming to sum over all the unseen state

# Summary

- HMMs are a very important structure in biological sequence analysis
- They are probabilistic Finite State Automata
- They are a generative model for sequence
- Of course, they are massively simplified generative models
- We can use dynamic programming to sum over all the unseen state